Tecnologias Web – Resumo Teórico

Gonçalo Sousa Atualizado: 6 out. 2026 (toda a matéria do site da disciplina)

Conteúdo

Tema 1 – A Web e o modelo cliente-servidor **2**
Da Internet à Web . . . 2
O modelo cliente-servidor . . . 2
Navegadores e servidores . . . 3
Panorama de tecnologias da Web . . . 3

Tema 2 – HTML **4**
Introdução . . . 4
Morfologia dos elementos . . . 5
Estrutura do documento . . . 6
Validação . . . 9
Formulários . . . 9
Tabelas . . . 10

Tema 3 – CSS: regras, seletores e padrões **12**
Regras CSS . . . 12
A cascata . . . 12
Associar CSS ao HTML . . . 14
Padrões de seletores . . . 16
Exemplos completos . . . 18

Tema 4 – CSS: propriedades e layout **21**
O modelo de caixa . . . 21
Cor, texto e tipo de letra . . . 22
Posicionamento clássico . . . 23
Flexbox . . . 25
CSS Grid . . . 27

Tema 5 – JavaScript: a linguagem **28**
Executar código e mostrar resultados . . . 28
Sintaxe básica . . . 28
Objetos e arrays . . . 32
Programação orientada a objetos . . . 35

Tema 6 – JavaScript: DOM, CSSOM e eventos **40**
Reflexão . . . 40
A árvore DOM . . . 41
Eventos . . . 44
Exemplos completos . . . 47

Tema 7 – O protocolo HTTP **50**
Introdução . . . 50
Recursos . . . 52
Métodos . . . 53
Códigos de estado . . . 55
Cabeçalhos . . . 56
Segurança . . . 58

Tema 8 – Comunicação assíncrona cliente-servidor **59**
A forma clássica: formulários e páginas inteiras . . . 59
Serialização de dados . . . 60
XMLHttpRequest . . . 60
Encadear pedidos . . . 62
Tempo real: SSE e WebSockets . . . 65
Exemplos completos . . . 67

Tema 9 – Outras APIs do navegador **70**
Web Storage . . . 70
Canvas: fundamentos . . . 71
Canvas: texto . . . 72
Canvas: caminhos . . . 72
Canvas: propriedades de linha . . . 73
Canvas: curvas . . . 74
Canvas: animação . . . 75
Canvas: embeber e eventos . . . 75
Canvas: transformações . . . 76
Exemplos completos . . . 78

Tema 10 – O servidor: Node.js **82**
Primeiro servidor . . . 82
Módulos . . . 82
Módulos internos . . . 83
Exemplos completos . . . 87

Tema 11 – CGI e alternativas **96**
O modelo CGI . . . 96
Variáveis de ambiente . . . 96
Entrada e saída . . . 97

O sinal † marca matéria que não está nos slides mas faz falta para as práticas.

1 Tema 1
A Web e o modelo cliente-servidor

Da Internet à Web

A **Internet** é a rede de redes que transporta qualquer tipo de dados. A **World Wide Web** (WWW, ou só **Web**) é **um** dos serviços que correm sobre ela, feito para organizar e disponibilizar informação. Email (SMTP), FTP ou IRC também usam a Internet, mas não são Web, porque não organizam a informação com URLs, HTTP e hipertexto.

> **Definição**
>
> A **Web** é um **espaço de informação global** onde:
>
> - os recursos (páginas, imagens, documentos) são **identificados por URLs**;
> - a **gestão** dos recursos é feita por servidores espalhados pela Internet;
> - a **navegação** entre recursos faz-se por **hipertexto**: conteúdo com referências clicáveis para outros recursos.

A Web tornou-se de utilização **massiva e ubíqua**, a **base das aplicações distribuídas** (redes sociais, email, bancos) e a **interface da "nuvem"**.

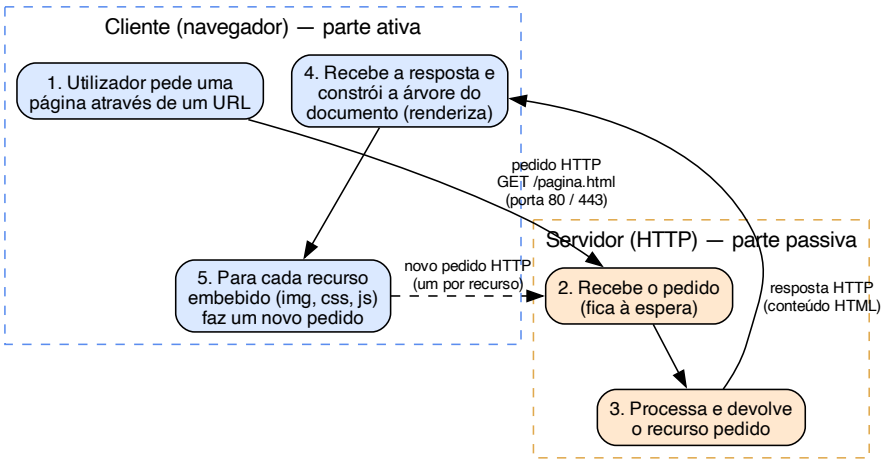
O modelo cliente-servidor

> **Definição**
>
> - O **cliente** é a parte **ativa**: toma a iniciativa, **pede** um recurso e depois **processa-o** (por exemplo, mostra-o).
> - O **servidor** é a parte **passiva**: fica **à espera**, **recebe** o pedido e **fornece** o recurso como resposta.

O modelo é comum a toda a Internet. Na Web, o cliente é o **navegador** e o servidor é um **servidor web** (Apache, Nginx, IIS). Comunicam sobre **TCP/IP**, com um protocolo aplicacional por cima: o **HTTP** (*HyperText Transfer Protocol*).

Um **pedido HTTP**† leva um **método** (`GET` para pedir um recurso, `POST` para enviar dados, como num formulário) e o **caminho** do recurso. A **resposta** leva um **código de estado** (200 = sucesso, 404 = não encontrado) e o **conteúdo** (por exemplo, o HTML da página). O HTTP é estudado a fundo no Tema 7.

Uma página dispara normalmente **vários** pedidos: um pelo HTML e depois um por cada recurso que o HTML referencia (imagens, CSS, scripts).



Exemplo resolvido

O que acontece ao escrever `http://www.dcc.fc.up.pt/pagina.html` e carregar Enter:

1. O **navegador** interpreta o URL: tem de contactar `www.dcc.fc.up.pt` na porta por omissão do HTTP (80).
2. Envia o pedido `GET /pagina.html` a esse servidor.
3. O **servidor** localiza `pagina.html` no seu sistema de ficheiros.
4. Devolve uma **resposta HTTP** com o HTML de `pagina.html`.
5. O navegador **constrói a árvore do documento** a partir do HTML.
6. Encontra `<img src="logo.png">`: a imagem **não** veio na resposta, é só uma referência. Faz um **novo pedido** `GET /logo.png`.
7. O servidor responde com os bytes da imagem, que o navegador **insere** no sítio certo da página.
8. O ciclo repete-se para cada recurso (CSS, JS, imagens) até a página estar completa.

Navegadores e servidores

Os **navegadores** pedem páginas, processam o HTML/CSS/JS recebido e mostram o resultado. Passaram por três fases:

- **pioneiros**: Mosaic, tkWWW, Lynx (modo texto);
- **guerras de browsers**: Netscape vs IE, IE vs Firefox, e Firefox vs IE vs Chrome;
- **últimos anos**: fim da hegemonia do IE, **vários** navegadores com quota relevante (Chrome, Firefox, Safari, Edge) e **preponderância do acesso móvel**.

Cada navegador interpretou certas construções de forma diferente: é daí que vem o problema da **interoperabilidade**, e a importância da standardização (W3C) e da **validação** (Tema 2).

Os **servidores web** passaram por fases parecidas:

- **pioneiros**: NCSA HTTPd e depois **Apache**;
- **guerra de servidores**: Apache vs Microsoft (IIS);
- **últimos anos**: **menos** concorrência e **menos** competidores do que nos navegadores, e uma **perceção muito menor** pelos utilizadores (o servidor é a parte passiva e invisível; o navegador é a parte ativa, que o utilizador escolhe e vê).

Panorama de tecnologias da Web

A Web é um conjunto de tecnologias (HTML, CSS, JavaScript, HTTP, …) que evoluem em conjunto. Duas ideias:

- **criação ou co-evolução?** Uma tecnologia nova surge muitas vezes em resposta a outra (o CSS surgiu para separar a formatação do HTML);
- **profusão de versões, nem sempre alinhadas**: cada tecnologia tem as suas versões, e os navegadores não as adotam todos ao mesmo tempo (o HTML5 já não tem DTD; alguns `input` do HTML5, como `tel` e `url`, não são suportados em todos os navegadores).

2 Tema 2 HTML

Introdução

HTML (*HyperText Markup Language*) descreve o conteúdo e a estrutura das páginas Web. Um documento HTML é um **ficheiro de texto** com **anotações** que dizem ao navegador como o interpretar.

> **Definição**
>
> O HTML serve para:
>
> 1. **apresentar conteúdo** (texto, imagens, vídeos);
> 2. **anotar o texto**, dando-lhe estrutura (títulos, parágrafos, listas);
> 3. **navegar por hipertexto** (hiperligações para outras páginas ou secções);
> 4. **embeber conteúdo externo** (imagens, CSS, JavaScript, vídeo).

```
<!DOCTYPE html>
<html>
  <head>
    <title>Olá</title>
  </head>
  <body>
    <h1>Olá HTML 5</h1>
    <img alt="HTML5" src="logo.png">
    <a href="https://www.w3.org/TR/html5/">
      Mais informação
    </a>
  </body>
</html>
```

Este documento mínimo mostra um título "Olá HTML 5", a imagem `logo.png` e uma hiperligação "Mais informação". As **anotações** são as construções entre < e >; a opção **"Ver código fonte"** do navegador mostra o HTML tal como veio do servidor.

- `<a href="...">Mais informação</a>` é um **elemento de navegação**: ao clicar, o navegador carrega **outra página**, identificada pelo URL do `href`.
- `<img src="logo.png">` é **conteúdo embebido**: a imagem é um ficheiro à parte, que o navegador vai buscar com outro pedido e insere **na página atual**. Também se embebem folhas de estilo **CSS** (formatação) e programas **JavaScript** (comportamento).

Ambos usam URLs, mas a navegação troca de página e o embeber traz conteúdo para dentro da página atual.

URLs absolutos e relativos

Um **URL** (*Uniform Resource Locator*) identifica um recurso na Internet (não só na Web: há URLs de email e FTP) e serve para o **carregar**.

Um **URL absoluto** tem três partes: **protocolo** (`http` ou `https`), **servidor** (nome ou IP, ex.: `dcc.fc.up.pt`) e **caminho** dentro do servidor (como os caminhos do Linux, ex.: `/~zp/pagina.html`). O separador é `://`. Opcionalmente indica-se a **porta** depois do servidor: `http://www.dcc.fc.up.pt:80/`. Por omissão usa-se a 80 (HTTP) ou a 443 (HTTPS).

Um **URL relativo** só tem o caminho, **relativo a um URL base** (a diretoria da página que faz a referência). Facilita a **recolocação** de um conjunto de páginas (mudar o site de servidor sem mexer nas ligações internas) e é mais **curto**.

Exemplo resolvido

A página `http://dcc.fc.up.pt/~zp/index.html` contém `<a href="logo.png">`.

1. URL base = diretoria da página atual: `http://dcc.fc.up.pt/~zp/`.
2. Concatena-se o caminho relativo: `http://dcc.fc.up.pt/~zp/` + `logo.png`.
3. URL absoluto: `http://dcc.fc.up.pt/~zp/logo.png`, que é o que o navegador pede.

Se a página estiver numa subdiretoria, o URL base muda com ela:

- página: `http://dcc.fc.up.pt/~zp/galeria/index.html`
- `logo.png` resolve para `http://dcc.fc.up.pt/~zp/galeria/logo.png`

Morfologia dos elementos

Anotações e contentores

Definição

As **anotações** são delimitadas por < e > e começam por um **nome** (o vocabulário do HTML), muitas vezes mnemónica do inglês: `br` = *BReak*, `hr` = *Horizontal Rule*. Funcionam como comandos.
Uma **anotação vazia** (`<br>`, `<hr>`) não tem conteúdo nem fecho. Um **contentor** é delimitado por um **par** de anotações com o mesmo nome: início (`<p>`) e **fecho**, que começa por **barra** (`</p>`).
Os contentores podem ser **aninhados**. Uma anotação de fecho termina a **última** anotação aberta com esse nome (ordem de pilha).

Em `<p> Este contentor tem um <b>pequeno</b> texto </p>`, o `</b>` fecha o `<b>`, aberto por último, e só depois o `</p>` fecha o parágrafo.

Atributos

Os **atributos** são as "opções" de uma anotação. Só aparecem na anotação de **início**, cada um é uma palavra sem espaços e a **ordem é irrelevante**.

Definição

Atributos simples (booleanos): só o nome, ex. `<input disabled>`, `<option selected>`. **Verdadeiro** se presente, **falso** se ausente.
Pares nome-valor `nome=valor`, ex. `<a href='http://www.up.pt/'>`, `<img src=casa.png width=90 height=50 alt="Casa d'avó">`:

- os valores são sempre *strings* (`width=90` é texto que representa um número);
- delimitam-se por **aspas** ou **plicas**, que não fazem parte do valor;
- os delimitadores são **obrigatórios** se o valor tiver espaços ou outros carateres especiais (`alt=HTML5` pode ir sem aspas);
- um valor pode conter o outro delimitador: `"Casa d'avó"`.

Entidades

Definição

Uma **entidade** é uma substituição de carateres (uma espécie de macro) feita durante o *parsing*. Começa por `&` e termina em `;`. Serve para escrever carateres que seriam lidos como formatação, como < e >.

Entidade	Carácter	Significado
`&`	&	o próprio &, que iniciaria uma entidade
`<`	<	*less than*: não abre uma anotação
`>`	>	*greater than*: não fecha uma anotação
`"`	"	aspas dentro de um valor delimitado por aspas
`'`	'	plica dentro de um valor delimitado por plicas
` `	espaço	*non-breaking space*: espaço que não é colapsado
`…`	…	reticências

É com `<` e `>` que se **mostra código HTML como texto** numa página.

Texto, CDATA e comentários

O **texto** é tudo o que está fora de anotações e de entidades.

> **Definição**
>
> Uma **secção CDATA**, delimitada por `<![CDATA[` e `]]>`, **inibe** a formatação: tudo lá dentro é texto literal. Pode ter **todos** os carateres exceto `]]>`, **não** se aninha, e serve para incluir **XML embebido** ou programas. É pouco usada.
>
> Um **comentário** tem a forma `<!-- Isto é um comentário -->`:
>
> - as anotações lá dentro ficam **inativas**;
> - **não** pode ocorrer dentro de outras anotações;
> - a *string* `--` **não** pode aparecer no texto comentado, por isso os comentários **não se aninham**.

Em `<![CDATA[ <p> Isto é <u> tudo </u> texto </p> ]]>` o conteúdo aparece tal como está escrito, com os `<` e `>`, sem precisar de `</>`.

A declaração `<!DOCTYPE html>`

Aparece sempre no **início** e **declara o tipo** do documento como HTML 5. **Parece** uma anotação, mas não é (tem `!` antes do nome), e **parece** um comentário, mas também não é (tem um nome em vez de `--`). Outras linguagens e versões têm declarações próprias: as do HTML 4 e do XHTML eram longas e remetiam para um DTD.

Estrutura do documento

Em linha e de bloco

Os contentores formam uma **árvore**; nas folhas estão anotações vazias (`<br>`, `<img>`) ou texto.

> **Definição**
>
> Cada contentor é desenhado como uma **caixa** retangular, com **dimensão** e **posição**, que contém as caixas dos descendentes. Há dois géneros:
>
> - **em linha** (*inline*): colocadas **no fluxo do texto**, lado a lado, sem quebra de linha;
> - **de bloco** (*block*): ocupam **toda a largura disponível**, com quebra de linha antes e depois.

Categoria	Elemento	Nome	Efeito
Em linha	`b`, `i`, `u`, `s`	Bold, Italic, Underlined, Striketrough	negrito, itálico, sublinhado, cortado
Em linha	`sup`, `sub`	Superscript, Subscript	expoente, índice
Em linha	`a`	Anchor	âncora / hiperligação
Em linha	`br`	BReak	quebra de linha (caixa achatada)
Em linha	`code`	—	carateres mono-espaçados
Bloco	`hr`	Horizontal Rule	régua horizontal (sem altura nem conteúdo)
Bloco	`p`	Paragraph	parágrafo
Bloco	`h1`…`h6`	Heading	título, destaque conforme o número
Bloco	`ul`, `ol`, `li`	Unordered/Ordered List, List Item	listas e os seus itens
Bloco	`dl`, `dt`, `dd`	Definition List, Defined Text, Definition	lista descritiva
Bloco	`img`	IMaGe	imagem embebida

Elementos em linha

Fluxo do texto. O texto corre da esquerda para a direita e quebra conforme o espaço. Os **carateres brancos são irrelevantes**: vários espaços seguidos valem **um** espaço, e mudanças de linha e tabulações valem um espaço. Por isso a indentação do código fonte não tem efeito visual. Os carateres têm **largura variável** (`|m| = 3·|i|`, `|a| = 2·|i|`); o `<code>` usa carateres **mono-espaçados**, mas também ignora os brancos extra.

Para **forçar** espaços e quebras, com **parcimónia**: `<br>` dá uma **quebra de linha** e ` ` um **espaço** que não é colapsado.

Formatação de texto. `b`, `i`, `u` e `s` combinam-se encaixando contentores: `<b><i>Negrito+Itálico</i></b>`. **Índices e expoentes**: `<sup>` e `<sub>` deslocam o texto em relação à linha de base e reduzem-no (∼75%), ex.: `e = m.c<sup>2</sup>` dá e = m.c². Em `<b>Negrito <sup>expoente <sub>índice</sub></sup></b>`, tudo fica a negrito, "expoente" sobe e, dentro dele, "índice" fica ainda mais pequeno e desce.

> **Definição**
>
> A **âncora** `<a>` é a base das hiperligações. Tem dois atributos, usados em alternativa:
>
> - `name` **cria um alvo** (um ponto endereçável na página);
> - `href` **cria a hiperligação** para um alvo. Se o alvo é um `name` da mesma página, o valor começa por `#`.
>
> O contentor de um `<a href>` é o **ponto ativo** que se clica, e por omissão aparece **a azul e sublinhado**.

```html
<a name="top"> Vai ser muiiito longo ... </a>
<!-- ... muito conteúdo ... -->
<a href="#top">Início</a>
```

Clicar em "Início" não carrega página nova: salta para o ponto `top` da mesma página.

Elementos de bloco

- `<hr>`: **régua horizontal**, uma caixa achatada a toda a largura; separa mais do que um `<br>`.
- `<p>`: **parágrafo**, com texto e elementos em linha; reserva espaço entre parágrafos consecutivos.
- `<h1>`…`<h6>`: **cabeçalhos**, `h1` o mais destacado, `h6` o menos.

Definição

Listas: `<ul>` é **itemizada** (*bullets*) e `<ol>` é **ordenada** (números). Os itens são blocos `<li>`. O fecho `</li>` pode ser **omitido** (o item seguinte ou o fim da lista fecha-o), e numa `<ol>` o atributo `value` **força** o número do item (`<li value=4>` dá "4.").
Lista descritiva `<dl>`, como um dicionário: cada entrada é um termo `<dt>` (*Defined Text*) e a sua definição `<dd>` (*Definition*).

```
<dl>
  <dt> <b>Itemizada</b>
  <dd>
    <ul>
      <li> Primeiro
      <li> Segundo
    </ul>
  <dt> <b>Ordenada</b>
  <dd>
    <ul>
      <li> Primeiro
      <li> Segundo
    </ul>
</dl>
```

Cada `<dd>` contém uma lista itemizada inteira: blocos aninham-se livremente dentro de blocos.

Imagens. `<img>` insere **conteúdo embebido**, com as dimensões do próprio ficheiro, a não ser que os atributos as mudem: `src` (URL da imagem), `alt` (texto alternativo, se a imagem não carregar, e para leitores de ecrã), `width` e `height` (em pixels).

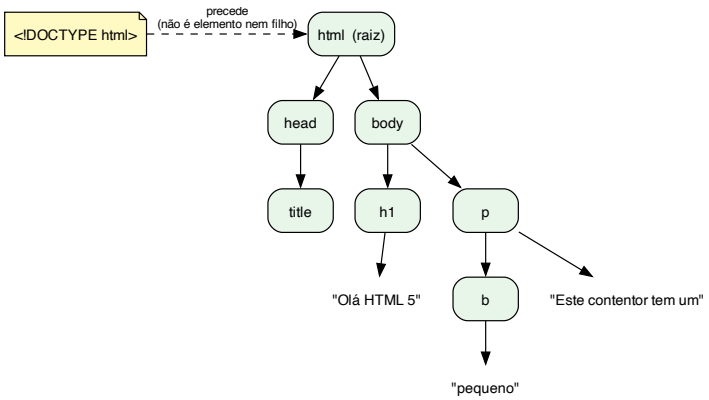
A estrutura de topo

Definição

Um documento HTML:

- começa pela **declaração de tipo** `<!DOCTYPE html>`;
- tem uma **única raiz**, `<html>` (o DOCTYPE e comentários antes dela não fazem parte da árvore);
- a raiz tem exatamente **dois** filhos: `<head>`, o **cabeçalho** (metadados, como o `<title>`; **não** é mostrado na área de trabalho), e `<body>`, o **corpo** (o que é mostrado).

Dentro do `<body>` não há estrutura rígida, mas certos elementos têm de estar dentro de outros (`<li>` dentro de `<ul>`/`<ol>`, `<td>` dentro de `<tr>`).



Validação

> **Definição**
>
> O HTML é um **tipo de documento anotado**: há um conjunto definido de anotações válidas, e cada uma pode ter **certos atributos** e conter **certos tipos de anotações** (eventualmente nenhum). No HTML 4 esta gramática era um **DTD** (*Document Type Definition*); o **HTML5 não tem DTD**, mas a ideia mantém-se.

Interoperabilidade. Os navegadores são **tolerantes**: mostram HTML inválido em vez de o recusar. Mas cada um pode interpretá-lo **de forma diferente**, e a página varia de navegador para navegador. Exemplos de HTML inválido tolerado:

- **anotações de fecho omitidas** (`</head>`, `</body>`, `</html>`; o `</li>` é permitido omitir no HTML5);
- **fechos fora de ordem**: `<b><i> ... </b></i>` em vez de `<b><i> ... </i></b>`;
- **valores de atributos sem aspas** quando deviam ter.

Uma página que "parece funcionar" num navegador pode não estar correta. Os **validadores** verificam o código; o de referência é o **Validador do W3C** (`https://validator.w3.org/`), que valida páginas publicadas (pelo URL), ficheiros carregados ou texto colado.

Formulários

Os **formulários** permitem ao utilizador introduzir valores através de **widgets** que restringem ou validam o que se escreve, e submeter esses valores a um servidor.

> **Definição**
>
> O elemento `<form>` agrega *widgets* a submeter ao servidor. Pode ser **omitido** se os *widgets* só forem usados no cliente, e pode haver **vários** numa página. Atributos:
>
> - `action`: o URL do recurso que recebe os dados;
> - `method`: o método HTTP (`get` ou `post`).
>
> Com `method=get`†, os pares nome-valor vão **no URL** (`?Nome=Ana&Idade=20`), visíveis na barra de endereços: serve para pesquisas. Com `method=post`, vão **no corpo** do pedido: para dados sensíveis ou para criar/alterar algo no servidor.
> Todos os *widgets* têm `name` (a chave do par enviado) e `value` (o valor atual). Os dois ligados à submissão são `reset` (repõe os valores iniciais) e `submit` (envia os pares ao `action`).

O `<input>` é o *widget* mais versátil; o atributo `type` escolhe o tipo. Os tipos especializados do HTML5 só aceitam **valores válidos** (ex.: um mês 13 não) e mostram a **estrutura esperada** e **seletores** de ajuda (calendário, seletor de cor).

`type`	Widget	`type`	Widget
`text`	texto (omissão)	`date`	data
`password`	texto mascarado	`datetime-local`	data e hora, sem fuso horário
`radio`	escolha única entre os de mesmo `name`	`month`	mês
`checkbox`	ligado/desligado, independente	`time`	hora
`button`	botão genérico, sem submissão	`week`	semana
`file`	escolher um ficheiro local	`color`	seletor de cor
`number`	só números	`image`	imagem (`src`); envia as coordenadas do clique
`range`	*slider* num intervalo	`email`	valida o formato de email
`search`	campo de pesquisa	`tel`, `url`	telefone, URL (**não suportados no Chrome**)

Definição

`<select>` agrega opções `<option>`. Com `size=1` (omissão) é um **menu suspenso**; com `size=n > 1` é uma **lista** com *n* valores visíveis. `<optgroup label="...">` **agrupa** opções com um cabeçalho. O `value` de uma `<option>` pode ser **diferente** do texto mostrado (mostra "XXXL", envia `9`). O fecho `</option>` pode ser omitido.
`<textarea>` serve para texto **multi-linha** (o `<input type=text>` é de uma linha). `rows` dá o número de linhas visíveis e `cols` a largura em carateres. O texto entre `<textarea>` e `</textarea>` é o **valor inicial**.

Exemplo resolvido

```
<form action=registar method=post>
  Nome <input name=Nome><br>
  Idade <input type=number name=Idade> <br>
  Email <input type=email name=Email> <br>
  Data de nascimento <input name=Nasce type=date>
  <input type=reset>
  <input type=submit value=Registar>
</form>
```

1. `action=registar method=post`: os dados vão para o recurso `registar` do servidor atual, no corpo do pedido.
2. `<input name=Nome>`: texto simples (tipo por omissão), enviado como `Nome=...`.
3. `type=number name=Idade`: só aceita números; envia `Idade=...`.
4. `type=email name=Email`: o navegador valida o formato; envia `Email=...`.
5. `name=Nasce type=date`: seletor de data; envia `Nasce=...` (a ordem dos atributos é irrelevante).
6. `type=reset`: repõe todos os campos (aqui, vazios).
7. `type=submit value=Registar`: o `value` é o **texto do botão**.

Ao clicar "Registar", o navegador junta os pares `nome=valor` e envia-os por `POST` para `registar`.

Tabelas

Uma tabela organiza conteúdo em **linhas e colunas**, mas no HTML é uma **árvore**: a tabela contém linhas e cada linha contém células. As células podem ter texto, imagens ou outras tabelas.

Definição

- `<table>`: a **tabela**;
- `<tr>` (*Table Row*): uma **linha**;
- `<td>` (*Table Data*): uma **célula**;
- `<th>` (*Table Header*): célula de **cabeçalho**, em vez de `<td>`; por omissão **centrada** e **a negrito**;
- `colspan` / `rowspan`: a célula **ocupa** esse número de **colunas** / **linhas** contíguas.

```
<table>
  <tr> <td>X</td> <td> </td> <td> </td> </tr>
  <tr> <td>O</td> <td>X</td>      <td>O</td>      </tr>
  <tr> <td>X</td> <td>O</td>      <td> </td> </tr>
</table>
```

Um jogo do galo: cada `<tr>` é uma linha da grelha 3×3. As células vazias levam ` ` para não colapsarem. Os bordos **não** vêm do HTML: uma tabela sem CSS não tem bordos.

Exemplo resolvido

```
<table>
  <tr>
    <th colspan=3>Dados</th>
  </tr>
  <tr>
    <th>Sexo</th> <th>Nome</th> <th>Idade</th>
  </tr>
  <tr>
    <td rowspan=2>&#9792;</td>
    <td>Sofia Costa<td>19
  </tr>
  <tr>
    <td>Joana Dias<td>20
  </tr>
</table>
```

1. `<th colspan=3>Dados</th>` **substitui** as três células da primeira linha: é um título a toda a largura.
2. A segunda linha tem os três cabeçalhos de coluna.
3. `<td rowspan=2>♀</td>` (o símbolo feminino) ocupa a coluna "Sexo" na terceira **e** na quarta linha. Por isso a quarta linha só tem **duas** células (Nome e Idade).
4. Os `</td>` foram omitidos (`<td>Sofia Costa<td>19`): um `<td>` novo fecha o anterior, como no `</li>` e no `</option>`.

Posicionamento com tabelas. Antes do CSS, usava-se `<table>` como grelha invisível para posicionar texto, imagens ou outras tabelas. Hoje é má prática† (mistura dados com apresentação): o layout faz-se em CSS (Tema 4).

> **Definição**
> No HTML5 a formatação das tabelas faz-se só em **CSS** (o HTML4 tinha atributos de formatação nas anotações). Três elementos **agrupam linhas** para o CSS as formatar de forma diferente: `<thead>` (**cabeçalho**), `<tbody>` (**corpo**) e `<tfoot>` (**rodapé**, ex.: totais).

```
<table>
  <thead>
    <tr><th>NCD</th><th>Nome</th><th>Classificação</th></tr>
  </thead>
  <tbody>
    <tr><td>20999999</td><td>Fulano de Tal</td><td>18</td></tr>
    <tr><td>20999999</td><td>Beltrano de Qual</td><td>17</td></tr>
  </tbody>
  <tfoot>
    <tr>
      <th colspan=2>Média</th><td>13.5</td>
      <th colspan=2>Inscritos</th><td>75</td>
    </tr>
  </tfoot>
</table>
```

Numa pauta, o `<thead>` tem os nomes das colunas, o `<tbody>` uma linha por aluno e o `<tfoot>` a média e os inscritos, que o CSS pode destacar (fundo escuro, negrito).

3 Tema 3
CSS: regras, seletores e padrões

Regras CSS

O **CSS** (*Cascading Style Sheets*) descreve a apresentação visual de documentos estruturados (XML, mas sobretudo HTML5). Uma **folha de estilos** é um conjunto de **regras**.

> **Definição**
>
> Uma **regra** associa **valores de propriedades** a elementos:
>
> ```
> seletor { propriedade: valor; }
> ```
>
> O **seletor** diz *a que elementos* se aplica; a **declaração** (entre chavetas) diz *que propriedade* muda e *para que valor*, separados por `:`. Ex.: `H1 { font-weight: bold }` põe todos os `<h1>` a negrito.
> Os **comentários** são `/* ... */` (não há `//`). Servem para **documentar** e para **inativar** temporariamente uma regra.

Agrupar seletores: separados por **vírgula**, recebem **todas** as declarações do bloco. **Agrupar declarações**: separadas por **ponto e vírgula**. É boa prática pôr `;` também depois da última declaração (ver o `;` como **terminador**, não como separador). As duas formas combinam-se: uma regra com n seletores e m declarações equivale a $n \times m$ pares seletor-declaração.

```
H1, H2, H3 {
    color:          gray;
    text-transform: capitalize;
    font-family:    arial, helvetica;
}
```

Esta regra vale por nove: cada um de `H1`, `H2`, `H3` recebe as três declarações.

A cascata

As propriedades de um elemento podem estar **dispersas por várias regras**, com valores **em conflito**. A **cascata** (o *C* de CSS) resolve-os.

> **Definição** **Ordem de resolução da cascata**
>
> 1. **Origem**: as regras do **autor** do documento vencem as do **leitor**/navegador.
> 2. **Especificidade**: na mesma origem, vence a regra **mais específica**.
> 3. **Ordem**: com especificidade igual, vence a **última** declarada.
> 4. `!important`: força a precedência de uma declaração, passando à frente dos três critérios anteriores.

```css
/* author */
h3.other   { font-style: italic; }
h1, h2, h3 {
    font-family: san-serif;
    font-weight: bold;
    font-style:  normal;
}
h1 { font-size: xx-large; }
h2 { font-size: x-large;  }
h3 { font-size: large; font-style: italic; }
h2 { font-size: 110%;     }

/* user agent (navegador) */
h1, h2, h3 {
    font-family: serif;
    font-weight: 500 !important;
}
```

Neste exemplo, `font-family`, `font-weight`, `font-style` e `font-size` aparecem várias vezes: a cascata resolve cada propriedade em separado.

1. Origem. O **navegador** também define regras (valores por omissão e preferências do **leitor**, ex.: letra maior); o **documento** traz as do **autor**. As do autor vencem. A especificação real† tem **três** origens, e o `!important` inverte a ordem entre elas, da menor para a maior prioridade:

`user agent` < `utilizador` < `autor` < `autor !important` < `utilizador !important` < `user agent !important`

2. Especificidade. Há quatro categorias de seletores, com pesos crescentes: tipo < classe < ID < estilo em linha (`style=`). O peso de um seletor depende de **quantos** seletores de cada categoria tem, e vence o de maior peso.

> **Definição** **Calcular a especificidade †**
>
> A especificidade é um tuplo (b, c, d):
>
> - b = número de **IDs** (`#x`);
> - c = número de **classes** (`.x`), **atributos** (`[x]`) e **pseudo-classes** (`:hover`);
> - d = número de **tipos** (`h1`, `div`) e **pseudo-elementos** (`::before`).
>
> Compara-se **posição a posição**: primeiro b; só se empatar, c; só se empatar, d. Nunca se soma: 1 ID vence 50 classes. Um `style=` em linha é um quarto número $a = 1$ à frente, que vence qualquer seletor (exceto `!important`).

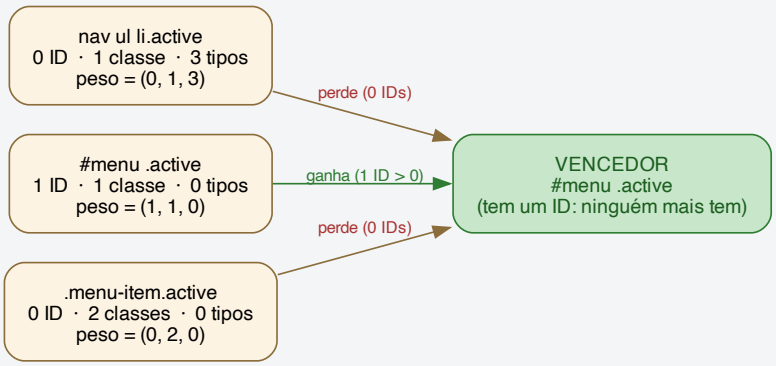
Exemplo resolvido

Três regras para o mesmo `<li id="menu" class="menu-item active">`:

```
nav ul li.active   { background: yellow; }   /* A */
#menu .active      { background: orange; }   /* B */
.menu-item.active  { background: green;  }   /* C */
```

Seletor	IDs b	Classes c	Tipos d	(b, c, d)
`nav ul li.active`	0	1 (`.active`)	3 (`nav`, `ul`, `li`)	$(0, 1, 3)$
`#menu .active`	1	1 (`.active`)	0	$(1, 1, 0)$
`.menu-item.active`	0	2 (`.menu-item`, `.active`)	0	$(0, 2, 0)$

Primeira posição: B tem $b = 1$, A e C têm $b = 0$. B vence logo, sem olhar para c nem d: o fundo é **laranja**. C tem mais classes do que B, mas um ID pesa mais do que qualquer número de classes.



3. Ordem de declaração. Com a mesma origem e especificidade, vence a **última** declarada (mais abaixo na folha, ou na última folha carregada). No exemplo da cascata, `h2 { font-size: x-large; }` e `h2 { font-size: 110%; }` têm a mesma especificidade $(0, 0, 1)$; a segunda vem depois, por isso `h2` fica com `110%`. Por isso importa a ordem entre `<link>` e `<style>`† no documento.

4. `!important`. Escrito depois do valor (`font-weight: 500 !important;`), **força** a precedência dessa declaração, mesmo com menor especificidade ou declarada antes. Usar com **parcimónia**: só outro `!important` o sobrepõe, o que torna o CSS difícil de depurar.

Associar CSS ao HTML

Definição

- **Folha externa**: um ficheiro `.css` só com regras, ligado no `<head>` com `<link rel="stylesheet" href="..." type="text/css">`. **Reutiliza** as regras entre páginas e dá **consistência** a um site.
- **Folha interna**: regras dentro de `<style>` no `<head>`, só para **essa** página. Os navegadores aceitam `<style>` no `<body>`, mas não é válido.
- **Estilo local**: o **atributo** `style` de um elemento, só com pares propriedade-valor, sem seletor. Formata um único elemento, ou **substitui** pontualmente uma regra geral (tem a maior especificidade; só um `!important` o vence).

```
<!DOCTYPE html>
<html>
<head>
  <title>Olá mundo</title>
  <link rel="stylesheet" href="style.css" type="text/css">
  <style>
     h1 { color: orange; }
  </style>
</head>
<body>
    <h1 style="font-weight: 500;">Olá mundo</h1>
</body>
</html>
```

Seletores por tipo, classe e id

```
<style>
   p { font-size: large; }
   .strong { font-weight: bold; }
   #important { color: red; }
</style>
<p class=strong> Lorem ipsum dolor sit amet, ... </p>
<p> Praesent vel elementum nunc...  </p>
<p class=strong id=important> Donec cursus orci ... </p>
```

> **Definição**
>
> - **Tipo** (`p`): **todos** os elementos desse tipo (os três parágrafos).
> - **Classe** (`.strong`, com **ponto**): todos os elementos cujo atributo `class` contém esse nome (o 1.º e o 3.º).
> - **Id** (`#important`, com **cardinal**): o **único** elemento com esse `id` (o 3.º). O `id` deve ser **único** no documento†; repeti-lo não dá erro, mas `document.getElementById` só devolve o primeiro.

Contentores genéricos: `div` e `span`

Quando nenhum elemento tem o significado certo, há dois contentores **sem significado**, só para pendurar um `id`, uma `class` ou um `style`:

```
<div style="background: orange;">
  Em fundo laranja e
  <span style="color: white;"> texto com cor branca </span> !!
</div>
```

- `div` é **de bloco**: permite formatar a **caixa** inteira.
- `span` é **em linha**: não quebra o texto; muitas propriedades (como a cor) são **herdadas** pelo que contém.

Combinar seletores

O atributo `class` aceita **vários** nomes, separados por espaço: em `<p class="strong important">` aplicam-se as regras de `.strong` **e** de `.important`.

Um **tipo colado a uma classe** restringe a classe a esse tipo:

```
p.important { background: orange; }
.important  { color: red; }
```

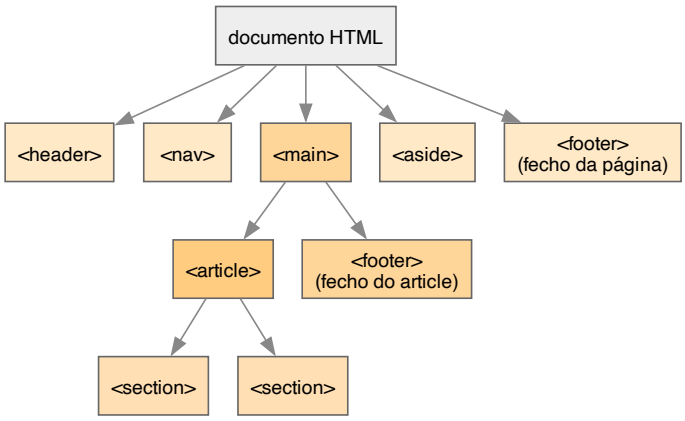
```
<p class=important> Lorem ipsum dolor sit amet, ... </p>
<p> Donec <span class=important>cursus</span> orci ...  </p>
```

O `<p class=important>` fica vermelho e com fundo laranja; o `<span class=important>` só fica vermelho. Atenção ao espaço: `p.important` são os `p` com a classe `important`, e `p .important` (com espaço) são os elementos com a classe `important` **dentro** de um `p` (descendência).

Elementos semânticos do HTML5

> **Definição**
>
> Um **elemento semântico** é um contentor cujo **nome** diz o papel do conteúdo. Tal como `div/span`, em geral não tem formatação própria, mas só se usa **quando o significado se adequa**. Captura papéis recorrentes da publicação eletrónica e **melhora a acessibilidade** (um leitor de ecrã anuncia a navegação ao encontrar um `<nav>`).

- **`<header>`**: bloco de cabeçalhos (`<h1>`, `<img>`, `<a>`). Pode haver **vários** (um da página, um por `<article>`), mas nunca dentro de `<header>`, `<footer>` ou `<address>`.
- **`<nav>`**: *links* da navegação **principal** (listas `<ul>`, `<li>`, `<a>`).
- **`<main>`**: o conteúdo **mais relevante**; costuma conter `<article>` e ter um título (`<h2>`).
- **`<article>`**: um tema **autónomo**, que faz sentido isolado (um post, uma notícia); contém `<section>` ou parágrafos e tem título (`<h3>`).
- **`<section>`**: **uma secção** de um documento; costuma haver várias num `<article>` ou `<main>`; tem título (`<h4>`).
- **`<aside>`**: informação **complementar**, muitas vezes ao lado.
- **`<footer>`**: **fecho** da página ou de um `<article>`; pode haver vários.



Outros elementos semânticos: **sem** formatação própria, `<figure>` (uma figura e a sua legenda) e `<figcaption>` (a legenda); **com** formatação própria, `<mark>` (texto destacado, fundo amarelo), `<button>` (botão para uma ação), `<details>` (informação que se expande e colapsa) e `<summary>` (a parte sempre visível de um `<details>`).

Padrões de seletores

Os **padrões** selecionam elementos pelo **contexto** na árvore, pelo **conteúdo**, pelos **atributos** ou pelo **estado**. Foram introduzidos ao longo dos níveis CSS 1, 2 e 3.

Definição

- Uma **pseudo-classe** (`seletor:nome`) seleciona elementos por uma condição (posição na árvore, estado) que não é uma classe do HTML.
- Um **pseudo-elemento** (`seletor::nome`) refere-se a uma **parte** de um elemento (a primeira letra) ou **insere** conteúdo. O `::` é do CSS3†; a forma antiga com um `:` (`:before`, `:first-letter`) continua a funcionar.

Padrão	Seleciona
Contexto	
`e1, e2`	ambos (agrupamento)
`ancestral e`	e com esse **ancestral**, a qualquer profundidade
`*`	**todos** os elementos
`:not(seletor)`	tudo o que **não** corresponde a `seletor`
Conteúdo	
`s:first-letter`, `s:first-line`	a **primeira letra** / **linha**
`s::before`, `s::after`	ponto **antes** / **depois** do conteúdo
Atributos	
`[atr]`	com o atributo (qualquer valor)
`[atr=valor]`	com o atributo **exatamente** igual a `valor`
`[atr~=valor]`	com `valor` **entre** as palavras do atributo
`[atr^=valor]`	com o atributo a **começar por** `valor`
`e:lang(língua)`	com esse atributo `lang`
Estado de links	
`:link`, `:visited`	*links* **não visitados** / **visitados**
`:active`	*link* a ser clicado
Estado de widgets	
`e:focus`	com o **foco** (recebe o teclado)
`:target`	alvo de um *link* interno (`#nome`)
`:enabled`, `:disabled`	*widget* **ativo** / **inativo**
`:checked`	*widget* **escolhido** (ex.: *checkbox* marcada)
`::selection`	texto **selecionado** pelo utilizador

`h2 b { color: orange; }` pinta o `<b>` de `<h2>Lorem <b>ipsum</b></h2>`, mas não o de `<p>Lorem <b>ipsum</b></p>`, que não tem `<h2>` como ancestral. `p:first-letter { font-size: larger; font-weight: bolder; }` faz uma letra capitular.

`::before` e `::after` sozinhos não mostram nada: precisam da propriedade **`content`**†. `a::after { content: "\2197"; }` põe uma seta diagonal (U+2197) a seguir a todos os *links*, sem mexer no HTML.

`[class=strong]` só apanha `class="strong"` exato, não `class="strong important"`; `[class~=strong]` apanha os dois (é o que `.strong` faz). `a[href^="http"]` seleciona os *links* externos.

O `:hover`† (rato sobre o elemento) é do mesmo género que `:focus`: dá retorno visual em botões e *links* (`a:hover { text-decoration: underline; }`).

```
a:link    { color: red; }
a:visited { color: orange; }
a:active  { font-weight: bold; }
```

Padrão de estrutura	Seleciona
`progenitor > e`	e **filho direto** de `progenitor`
`anterior + e`	e **imediatamente** a seguir a `anterior` (irmão adjacente)
`anterior ~ e`	e a seguir a `anterior`, não necessariamente logo a seguir
`:root`	a **raiz** (`<html>`, com mais especificidade)
`e:empty`	e **vazio** (nem espaços)
`s:first-child`, `e:last-child`	**primeiro** / **último** filho do progenitor
`e:first-of-type`, `e:last-of-type`	primeiro / último **deste tipo** no progenitor
`e:only-child`	**único** filho do progenitor
`e:nth-child(n)`	o **n-ésimo** filho (de qualquer tipo)
`e:nth-last-child(n)`	o n-ésimo, **a contar do fim**

`div p` apanha qualquer `<p>` dentro de um `<div>`; `div > p` só os **filhos imediatos** (um `<p>` dentro de um `<span>` dentro do `<div>` não conta).

O argumento de `:nth-child` pode ser `an+b`†, com `n` = 0, 1, 2, …: `:nth-child(2n)` ou `(even)` são os **pares** (linhas alternadas de uma tabela, efeito *zebra*), `(2n+1)` ou `(odd)` os **ímpares**, `(3n)` o 3.º, 6.º, 9.º, …

Exemplos completos

Dois componentes feitos só com `div`s e classes. Usam propriedades de *layout* explicadas no Tema 4: `float: left/right` tira o elemento do fluxo e encosta-o a um lado, com o resto a correr à volta; `clear: right` impede um elemento de subir para o lado de um flutuante anterior; `display: flex` com `flex-wrap: wrap` arruma os filhos em linhas; `flex-basis` é o tamanho "ideal" de um filho.

Exemplo resolvido **Caixa de diálogo**

```html
<div class="dialog">
    <div class="message"> Mesmo sair? </div>
    <div class="button"> Ok </div>
    <div class="button"> Cancelar </div>
</div>
```

```css
.dialog {
    width: 90%; height: 120px; padding: 10px;
    margin: auto;
    border-width: 4px; border-style: double; border-color: #228;
    background: white;
    box-shadow: 3px 3px #AAA;
}
```

- `width`, `height`, `padding`: as **dimensões** da caixa.
- `margin: auto`: **centra horizontalmente** um bloco de largura definida (as margens esquerda e direita ficam iguais).
- `border-style: double`: contorno de **traço duplo**.
- `box-shadow: 3px 3px #AAA`: **sombra** 3px à direita e 3px abaixo, cinzenta.

Exemplo resolvido **Caixa de diálogo (continuação)**

```
.message {
    width: 50%; height: 100%;
    float: left;
    line-height: 120px;
    color: orange;
}
.button {
    float: right; clear: right;
    width: 145px; height: 40px; margin: 10px;
    line-height: 40px; text-align: center;
    background: orange; color: white;
    border-radius: 15px; border-width: 2px; border-style: outset;
}
```

- `.message` flutua à **esquerda** e ocupa 50% da largura. Para a **centrar verticalmente**, a altura da linha (`line-height: 120px`) é igual à altura da caixa: uma única linha fica a meio.
- `.button` flutua à **direita**; o `clear: right` obriga o segundo botão a ficar **por baixo** do primeiro. `line-height` igual a `height` centra o texto na vertical, `text-align: center` na horizontal.
- `border-radius: 15px` arredonda os cantos; `border-style: outset` dá relevo 3D.

Exemplo resolvido **Calculadora**

```
<div class="calculator">
    <div class="display">0</div>
    <div class="key">7</div> <div class="key">8</div> <div class="key">9</div>
    <div class="key op"> / </div>
    <div class="key">4</div> <div class="key">5</div> <div class="key">6</div>
    <div class="key op"> * </div>
    <div class="key">1</div> <div class="key">2</div> <div class="key">3</div>
    <div class="key op"> + </div>
    <div class="key">,</div> <div class="key">0</div> <div class="key">C</div>
    <div class="key op"> = </div>
</div>
```

```
.calculator {
    display: flex; flex-wrap: wrap;
    width: 200px;
    border-style: outset; border-width: 2px;
    padding: 10px; background: #DDD; margin: auto;
}
```

As teclas de operação têm duas classes (`class="key op"`). O painel tem **largura fixa** e **altura omitida** (ajusta-se ao conteúdo). `display: flex` com `flex-wrap: wrap` arruma o visor e as 16 teclas em **linhas** de 4, passando à linha seguinte quando a atual enche. `margin: auto` centra o painel.

Exemplo resolvido **Calculadora (continuação)**

```css
.display {
    flex-basis: 90%; height: 1em;
    border-width: 2px; border-style: inset;
    margin: 2px; margin-bottom: 20px; padding: 2px;
    text-align: right;
    background: white; color: orange;
}
.key {
    width: 40px; height: 40px;
    border-width: 2px; margin-bottom: 4px; border-style: outset;
    text-align: center; line-height: 40px;
    background: orange; color: white;
}
.op { background: white; color: orange; }
```

- `.display`: `flex-basis: 90%` faz o visor ocupar quase toda a primeira linha, por isso as teclas começam na linha seguinte. `height: 1em` é a altura de uma linha de texto; `text-align: right` alinha os números à direita.
- `.key`: teclas de 40×40px, texto centrado na horizontal e na vertical (`line-height` = `height`). As mudanças de linha vêm do `flex-wrap` do contentor.
- `.op` troca as cores (fundo branco, texto laranja). `.op` e `.key` têm a mesma especificidade $(0, 1, 0)$, e `.op` vem depois: vence em `background` e `color`, e as outras propriedades de `.key` mantêm-se.

4 Tema 4 CSS: propriedades e layout

Cada propriedade CSS tem um valor de **omissão**. Algumas são **herdadas** pelos descendentes (as de texto e tipo de letra); outras (caixa, posicionamento) aplicam-se só ao elemento onde são declaradas.

O modelo de caixa

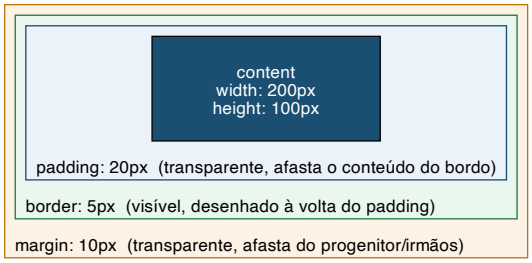
Todo o elemento é uma **caixa retangular**, **contida** na caixa do progenitor, **rodeada** pelos irmãos, e que **contém** as caixas dos descendentes. `width` e `height` dão a largura e a altura do conteúdo, mas o espaço ocupado depende também das camadas à volta.

Definição

Quatro camadas, de dentro para fora:

- **`content`**: o conteúdo, com as dimensões `width`/`height`;
- **`padding`**: preenchimento entre o conteúdo e o bordo, **transparente** (mostra o fundo do elemento);
- **`border`**: o bordo, a única camada que **se desenha**;
- **`margin`**: margem até às caixas vizinhas, **transparente** e nunca desenhada.

O `background-color` pinta o `content` e o `padding`, nunca a `margin`.



Bordo: `border-width` (espessura), `border-style` (`solid`, `dotted`, …) e `border-radius` (cantos arredondados; um só canto com, por exemplo, `border-top-left-radius`). O bordo só **aparece** depois de definir `border-style`†: por omissão é `none`, e `border-width` com `border-color` sozinhos não desenham nada.

Um lado só: junta-se `top`, `bottom`, `left` ou `right` ao nome: `margin-top`, `border-right`, `padding-left`, `border-top-style`.

Definição

Uma dimensão leva sempre **unidade**.
Absolutas: `cm`, `mm`, `in` (polegadas, `1in = 2,54cm`), `pt` (pontos, `1pt = 1/96in`; **não** são pixels), `pc` (picas, `1pc = 12pt`).
Relativas: `%` (da dimensão do **progenitor**), `px` (depende da **resolução**), `em` (do **tamanho de letra** do elemento), `rem` (do tamanho de letra da **raiz** `html`), `vw` e `vh` (percentagem da **largura** e da **altura** do *viewport*, a área visível da janela).
Com elementos aninhados, os `em` **multiplicam-se**† (`1.5em` dentro de `1.5em` dá 2,25 vezes); o `rem` refere-se sempre à raiz e evita isso.

Exemplo resolvido

Largura total ocupada por:

```
div { width: 200px; padding: 20px;
      border-width: 5px; border-style: solid; margin: 10px; }
```

Camada	Soma (esquerda + direita)	Largura acumulada
content	`width: 200px`	200px
+ padding	20 + 20	240px (limite interior do bordo)
+ border	5 + 5	250px (largura **visível**)
+ margin	10 + 10	**270px** (espaço reservado no layout)

`padding`, `border` e `margin` sem lado valem para os quatro lados, por isso somam-se duas vezes. A altura calcula-se da mesma forma, com os lados de cima e de baixo. `width/height` são **só** o conteúdo: é o comportamento por omissão, `box-sizing: content-box`†. Com `box-sizing: border-box`, `width` passa a ser a largura **até ao bordo** (o conteúdo encolhe para caber o `padding` e o `border` nos 200px).

Cor, texto e tipo de letra

Propriedades coloríveis: `color` (o que é desenhado, tipicamente o texto), `background-color` (fundo) e `border-color` (bordo).

Uma cor escreve-se por **nome** (`red`), em **RGB**, em hexadecimal (`#FF0000`) ou em função (`rgb(255,0,0)`), ou em **HSL** (*Hue Saturation Lightness*: matiz, saturação, luminosidade; `hsl(0,100%,50%)` é vermelho). Em HSL é fácil escurecer uma cor à mão†: baixa-se só a luminosidade. RGB e HSL aceitam um canal **alpha** de opacidade: `rgba(255,0,0,0.5)`, `hsla(0,100%,50%,0.5)`.

A propriedade `opacity` (de `0`, transparente, a `1`, opaco) aplica-se ao **elemento inteiro**, com conteúdo e descendentes. Um `rgba(..., 0.5)` no `background-color` só torna o fundo transparente†; o texto fica opaco.

Definição

- `word-spacing` / `letter-spacing`: espaço extra entre **palavras** / **letras** (comprimento, ex. `3mm`, ou `normal`).
- `text-align` (em **blocos**): `left` (omissão), `right`, `center`, `justify` (todas as linhas à largura toda, exceto a última).
- `vertical-align` (em elementos **em linha**, posição em relação à linha de texto): `baseline`, `sub`, `super`, `top`, `text-top`, `middle`, `bottom`, `text-bottom`, ou uma distância/percentagem (positiva sobe, negativa desce).
- `text-decoration`: `underline`, `overline`, `line-through` (rasurado), `blink` (obsoleto), `none` (tira o sublinhado dos *links*).
- `line-height`: altura de cada linha; `normal`, um fator (`1.5` = 1,5 vezes o tamanho de letra), uma percentagem ou um comprimento (`15px`).

Definição

- `font-family`: uma família **genérica** (`serif`, `sans-serif`, `cursive`, `fantasy`, `monospace`) ou **específica** (`times`, `arial`). Normalmente uma **lista**†, da preferida para a genérica: `font-family: "Helvetica", Arial, sans-serif;` (usa a primeira que existir).
- `font-style`: `normal`, `italic` (a versão itálica desenhada para a fonte) ou `oblique` (inclinação gerada pelo navegador).
- `font-weight`: `normal`, `bold`, `bolder`, `lighter`, ou 100 a 900 (`400` = `normal`, `700` = `bold`).
- `font-size`: `xx-small`, `x-small`, `small`, `large`, `x-large`, `xx-large`; uma percentagem (do tamanho do progenitor); ou um comprimento (`24pt`).
- `font-variant`: `normal` ou `small-caps` (as minúsculas passam a maiúsculas pequenas, "versaletes").

Posicionamento clássico

O posicionamento do CSS1/CSS2: **flutuar**, **classificação** e dimensões, **coordenadas** e **referenciais**, **sobreposição**, **transbordo** e **centrar**.

Definição

`float: left` / `right` tira o elemento do fluxo e faz-o **flutuar** para esse lado, com o resto do conteúdo a correr à volta. Foi pensado para **imagens** e **tabelas**, que são blocos e ocupariam a largura toda.
Vários elementos seguidos com `float: left` ficam **lado a lado**: é assim que se fazem **colunas** com `div`s. Com `left` o primeiro fica mais à esquerda; com `right` o primeiro fica mais à direita. Podem misturar-se: os `left` acumulam da esquerda, os `right` da direita.
Um elemento **não flutuante** a seguir pode **sobrepor-se** aos flutuantes. A propriedade `clear` (`left`, `right`, `both`) obriga-o a começar abaixo dos flutuantes desse lado. É comum pôr um `div` vazio com `clear: both` depois do último flutuante.

```
<div style="float: left; width: 45%;"> Coluna esquerda... </div>
<div style="float: left; width: 45%;"> Coluna direita... </div>
```

Duas colunas: o segundo `div` encaixa à direita do primeiro em vez de ir para baixo.

Definição

`display` diz como o elemento entra no fluxo, e daí como se controlam as **dimensões**:

- **`inline`**: segue o fluxo do texto, lado a lado; as dimensões são as do **conteúdo** (`width`/`height` são ignorados);
- **`block`**: ocupa a **largura disponível** e quebra a linha antes e depois; respeita `width`/`height`;
- **`inline-block`**: posiciona-se como `inline` (lado a lado) e dimensiona-se como `block` (respeita `width`/`height`). Faz colunas sem `float`.

Outros valores imitam elementos HTML: `table`, `table-row`, `table-cell` (como `<table>`, `<tr>`, `<td>`), `list-item` (como `<li>`), e os do CSS3 `flex`, `inline-flex`, `grid`, `inline-grid`.

Três `div`s com `width: 100px; height: 300px;`: com `block` ficam empilhados (mesmo com 100px de largura); com `inline` ficam lado a lado, mas com o tamanho do conteúdo; com `inline-block` ficam três colunas de 100×300px lado a lado.

Esconder um elemento: `display: none` **remove-o** da formatação (as dimensões ficam nulas e o espaço é libertado; é o habitual para esconder com JavaScript, ex. diálogos); `visibility: hidden` torna-o **invisível** mas **mantém o espaço** (nada "salta").

Coordenadas e referenciais

Um elemento **posicionado** (`position` diferente de `static`) desloca-se com `top`, `right`, `bottom` e `left`: a distância entre cada lado da caixa e o mesmo lado do **referencial**. Não é preciso dar as quatro (`top` e `left` chegam). O referencial depende de `position`:

`position`	Referencial de `top`/`right`/`bottom`/`left`	Mantém o espaço?
`static`	nenhum: as coordenadas são ignoradas	sim (é o fluxo normal)
`relative`	a própria posição estática	sim
`absolute`	o primeiro antepassado posicionado (ou o documento)	não
`fixed`	a janela do navegador (não se mexe com o *scroll*)	não
`sticky`	`relative` enquanto visível; depois `fixed` no contentor com *scroll*	sim

Definição

- **`static`** (omissão): posição dada pelo `display` e pela ordem no documento; `top`/`right`/`bottom`/`left` são **ignorados**.
- **`relative`**: o elemento **mantém** o espaço no fluxo e é **deslocado** a partir da sua posição estática. É o que implementa o `vertical-align: sub/super`.
- **`absolute`**: sai do fluxo (o espaço **não** é mantido). Referencial: o **primeiro antepassado posicionado** (com `position` diferente de `static`); se não houver, o documento.
- **`fixed`**: sai do fluxo; o referencial é a **janela** (*viewport*), por isso não se mexe com o *scroll*.
- **`sticky`**: comporta-se como `relative` enquanto a sua posição está visível no contentor; quando o *scroll* a tiraria da vista, passa a `fixed` em relação ao **contentor com *scroll***, e fica "colado" a essa borda (útil em cabeçalhos de secções e tabelas).

Para usar um progenitor como referencial de um descendente `absolute`, dá-se-lhe `position: relative` **sem** `top`/`left`†: fica posicionado sem se mexer.

Sobreposição. `z-index` dá a cada elemento posicionado um nível numa pilha: **mais baixo** fica **por baixo**. Só funciona em elementos **posicionados**; num `static` é ignorado.

Transbordar. Quando o conteúdo excede a caixa, `overflow` decide: `visible` (omissão; sai da caixa), `hidden` (cortado), `scroll` (barra de deslocamento sempre visível) ou `auto` (barra só se for preciso).

Centrar

Exemplo resolvido

Texto numa caixa de 200×100px:

```
div.botao {
  width: 200px; height: 100px;
  text-align: center;   /* horizontal */
  line-height: 100px;   /* vertical   */
}
```

Uma linha de texto fica centrada na vertical dentro da sua altura de linha. Com `line-height` **igual** à `height` da caixa, o texto fica a meio da caixa; com valores diferentes, ficaria acima ou abaixo.
Um bloco de 100×100px no contentor:

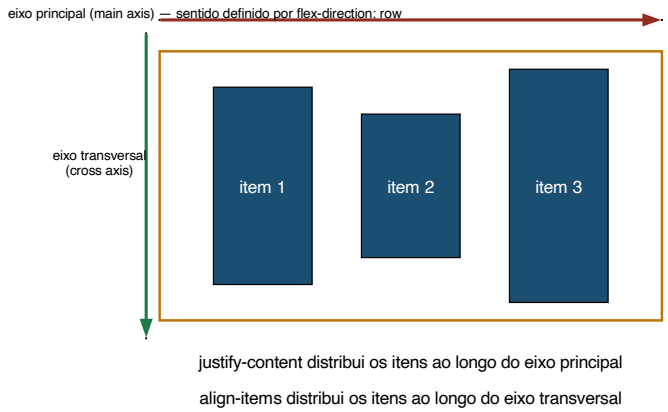
```
div.botao {
  width: 100px; height: 100px;
  margin: auto;        /* 1: centra na horizontal         */
  position: relative;  /* 2: posicionado, para usar top   */
  top: 50%;            /* 3: desce 50% da altura do CONTENTOR */
  margin-top: -50px;   /* 4: sobe metade da altura do botão */
}
```

1. `margin: auto` reparte a margem esquerda e direita por igual.
2. `position: relative` torna-o posicionado, para o `top` ter efeito.
3. `top: 50%` põe o **topo** do botão a meio do contentor: o botão fica meio botão abaixo do centro.
4. `margin-top: -50px` sobe metade da **altura do próprio botão** (100/2 = 50px): o **centro** do botão fica a meio do contentor.

Flexbox

Definição

O **flexbox** posiciona em **fluxo**, com ideias dos *toolkits* gráficos (preencher espaço, quebrar, alinhar). Há dois papéis: o **contentor** (`display: flex`) e os **itens** (os seus filhos diretos). Os itens dispõem-se ao longo do **eixo principal** (direção de `flex-direction`); o **eixo transversal** é o perpendicular.
Com `flex-direction: row` (omissão) o eixo principal é **horizontal**; com `column` é **vertical**†. Por isso `justify-content` (eixo principal) distribui na horizontal com `row` e na vertical com `column`, e `align-items` (eixo transversal) ao contrário.



Contentor	Valores
`display`	`flex` (o contentor é um bloco), `inline-flex` (em linha)
`flex-direction`	`row` (esq.→dir., omissão), `row-reverse`, `column` (cima→baixo), `column-reverse`
`flex-wrap`	`nowrap` (omissão: os itens **encolhem** para caber numa linha), `wrap` (quebra para a linha seguinte), `wrap-reverse` (para a anterior)
`flex-flow`	atalho: direção + quebra, ex. `column-reverse wrap`
`justify-content`	distribuição no eixo **principal**
`align-items`	alinhamento de cada item no eixo **transversal**
`align-content`	distribuição das **linhas** no eixo transversal

O prefixo `flex-` aparece quando já existe uma propriedade genérica com o mesmo nome.

Exemplo resolvido

Com `row` e três itens, `justify-content`:

- `flex-start`: itens encostados ao **início** (esquerda), espaço livre no fim;
- `flex-end`: encostados ao **fim** (direita);
- `center`: agrupados no **centro**, espaço igual antes e depois;
- `space-between`: o primeiro no início, o último no fim, o espaço livre **entre** os itens em partes iguais (nada nas pontas);
- `space-around`: cada item recebe a mesma margem **dos dois lados**; entre itens o espaço fica o **dobro** do das pontas.

`align-items` (com `row`, eixo transversal vertical):

- `stretch` (omissão): os itens **esticam** à altura do contentor (se não tiverem `height` própria);
- `flex-start` / `flex-end`: encostados ao **topo** / **fundo**;
- `center`: centrados na **vertical**;
- `baseline`: alinhados pela **linha de base do texto** (útil com tamanhos de letra diferentes).

`align-content` só atua com **várias linhas** (`flex-wrap: wrap` e altura suficiente): distribui as **linhas inteiras** no eixo transversal, e aceita também `space-around`.

Item	Efeito
`flex-basis`	tamanho **base** no eixo principal, antes de crescer/encolher; `auto` = tamanho do conteúdo; tem prioridade sobre `width` (`row`) ou `height` (`column`)
`flex-grow`	capacidade de **crescer** para o espaço livre; omissão `0`
`flex-shrink`	capacidade de **encolher** se as bases não couberem; `0` não encolhe, `1` omissão, `n` encolhe *n* vezes mais
`flex`	atalho `grow shrink basis`, omissão `0 1 auto`; ex. `flex: 1 0 200px`, `flex: 2` (só o *grow*)
`order`	ordem visual sem mexer no HTML; omissão `0`, menores primeiro, aceita negativos
`align-self`	o `align-items` só deste item (mesmos valores)

Exemplo resolvido

`flex-grow`. O espaço livre reparte-se **proporcionalmente**:

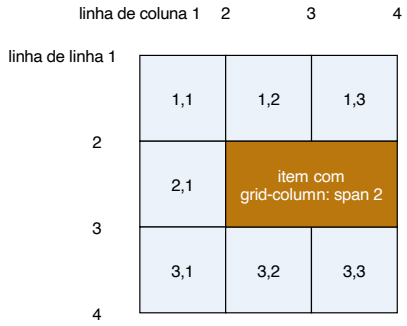
- três itens com `flex-grow: 1`: três partes iguais;
- itens com 1, 2 e 3: $1+2+3=6$ partes; o primeiro recebe 1/6 do espaço livre, o segundo 2/6, o terceiro 3/6.

Com os itens a crescer não sobra espaço livre, e `justify-content` deixa de ter efeito.
`flex-shrink`. Bases de 100px, 200px e 300px com `flex-shrink` 1, 2 e 3: se o conjunto não couber, o terceiro perde proporcionalmente mais largura do que o primeiro.

CSS Grid

Definição

O **CSS Grid** posiciona em **duas dimensões**: uma grelha de **linhas** e **colunas** (o flexbox é uma dimensão). Dois papéis: o **contentor** (`display: grid`) e os **itens** (filhos diretos, colocados nas células).
As fronteiras entre células são as **linhas de grelha**, numeradas a partir de **1**: 3 colunas têm 4 linhas de grelha verticais.



Numeros a cinza = linhas de grelha (fronteiras); células a laranja = item expandido por 2 colunas

Contentor	Efeito
`display`	`grid` (bloco) ou `inline-grid` (em linha)
`grid-template-columns`, `grid-template-rows`	**número** e **dimensão** das colunas/linhas, uma por valor
`row-gap`, `column-gap`	"corredor" entre linhas / colunas (não nas bordas); comprimento ou `normal`
`grid-auto-flow`	ordem de colocação automática: `row` (linha a linha, omissão) ou `column`
`grid-auto-rows`, `grid-auto-columns`	dimensão das linhas/colunas **implícitas**
`grid-template`, `gap`, `grid`	atalhos: modelos; separações; tudo

Em `grid-template-columns: 100px 40% auto;` há **três** colunas: 100px, 40% do contentor, e o resto. Cada valor pode ser um **comprimento**, `none` (omissão: sem modelo, as colunas criam-se conforme preciso), `auto` (pelo conteúdo), `max-content` / `min-content` (o maior / menor elemento), `initial` / `inherit`.

As linhas/colunas **implícitas** são as criadas quando há mais itens do que células no modelo†. Com `grid-template-rows: auto auto;` e itens que chegam a uma terceira linha, a altura dessa linha vem de `grid-auto-rows`, não do modelo. Aceita comprimento, `auto` (omissão), `max-content`, `min-content`.

Definição

Um **item** posiciona-se pelos **números das linhas de grelha** (as fronteiras, não as células): `grid-row-start`, `grid-row-end`, `grid-column-start`, `grid-column-end`. `grid-row-start: 2` com `grid-column-start: 2` põe o item na célula da 2.ª linha e 2.ª coluna.
Expandir: `span n` ocupa *n* células: `grid-row-start: span 3` dá 3 linhas de altura, `grid-column-start: span 2` duas colunas de largura.
Atalhos: `grid-row` (início/fim de linha) e `grid-column` (de coluna); `grid-row: span 3` é o mesmo que `grid-row-start: span 3`.

5 Tema 5 JavaScript: a linguagem

Executar código e mostrar resultados

O JavaScript entra no HTML em elementos `<script>`, no `body` ou no `head`. A posição decide **quando** corre:

- no `body`, corre **durante** a formatação, no ponto em que aparece (o que escrever fica ali, no meio do HTML);
- no `head`, corre **antes** de o corpo ser formatado;
- mais tarde, **em reação a eventos** (*event handlers*), ex.: o atributo `onclick="..."` corre JavaScript quando o elemento é clicado.

Definição **As três formas de output**

1. **`document.write(x)`** / **`document.writeln(x)`** escrevem na **própria formatação** da página (o `writeln` junta `\n`). As anotações HTML no texto (`<h3>Olá</h3>`) são formatadas.
2. **`alert(x)`** mostra uma caixa de diálogo, tipicamente em resposta a eventos. Da mesma família: **`prompt(mensagem)`** pede texto e **devolve** a *string* escrita (ou `null` se cancelar), e **`confirm(mensagem)`** pergunta OK/Cancelar e **devolve um booleano**. Nenhuma das três formata HTML.
3. **`console.log(x)`** escreve na **consola** do navegador (ferramentas de programador, invisível ao utilizador): serve para **depurar**. Na consola também se executam comandos interativamente.

Sintaxe básica

Definição

Tipos primitivos: `Number` (`123`; inteiros e decimais não se distinguem), `String` (`"Olá"`) e `Boolean` (`true/false`).
As **expressões** são como em Java (operadores, precedências, associatividades). Comentários `//` e `/* */`, **blocos** `{ }`, e instruções separadas por **`;`**, que **pode ser omitido** (ver ASI abaixo).
Variáveis: a primeira **atribuição** cria uma variável **global** implícita, mesmo dentro de uma função (evitar). Devem ser **declaradas**:

- **`const`**: não pode ser reatribuída; usar sempre que possível;
- **`let`**: âmbito do **bloco** `{ }` onde é declarada;
- **`var`**: âmbito da **função** (ou global); evitar.

Podem ser inicializadas na declaração, e o **tipo** é **dinâmico**: a mesma variável pode guardar valores de tipos diferentes.

Exemplo resolvido

```
document.writeln(1 + 1);        // 2
document.writeln("1" + "1");    // "11"
document.writeln(1 && 1);       // 1
document.writeln(1 && true);    // true
document.writeln(1 == "1");     // true
document.writeln(1 === "1");    // false
```

- `1 + 1`: dois `Number`, soma: `2`.
- `"1" + "1"`: com uma `String`, o `+` **concatena**: `"11"`.
- `1 && 1`: o `&&` **não** converte para booleano. `a && b` devolve `a` se `a` for *falsy* (sem avaliar `b`), senão devolve `b`†. `1` é *truthy*, logo devolve o segundo operando, `1`.
- `1 && true`: pela mesma regra, devolve `true`.
- `1 == "1"`: igualdade **fraca**, converte `"1"` para `1`: `true`.
- `1 === "1"`: igualdade **estrita**, tipos diferentes: `false`.

O `||` é simétrico†: `a || b` devolve `a` se for *truthy*, senão `b`. Dá valores por omissão: `nome = nomeDado || "Anónimo"`.

```
x = 1;            // sem declaração: global implícita
const y = 2;
let   z;          // undefined
var   w;          // undefined
z = 3;
w = 4;
console.log(x + y + z + w);   // 1+2+3+4 = 10
```

Em `{ let x = 1; let y = 2; { let x = 3; } }`, o `x = 3` é **outra** variável, que só existe no bloco interior; ao sair, o `x` de fora continua `1`.

Inserção automática de `;` (ASI)

O motor infere o fim de uma instrução pela mudança de linha, mas **só** quando a linha seguinte tornaria a instrução **inválida**.

Exemplo resolvido

```
const a = 1
const b = 2
const c = a + b
(a + b).toString()
```

Lança `TypeError: b is not a function`. A linha 4 começa por `(`, e `a + b (a + b).toString()` é válido (uma chamada de função), por isso **não** se insere `;` e o motor lê:

```
const c = a + b(a + b).toString()
```

`b` é o número `2`, que não é uma função: erro.

O `;` é inserido quando: a linha seguinte tornaria a instrução inválida; a linha seguinte começa por `{`; a linha tem só `return`, `break`, `throw` ou `continue` (o `;` vai logo a seguir, e uma expressão na linha seguinte a um `return` é **ignorada**); ou no fim do ficheiro. Boa prática: usar sempre `;` como **terminador**.

Controlo de fluxo e funções

O controlo de fluxo é como em Java: `if/else`, `while/do`, `switch/case/break`, `for/break/continue`. As variáveis de controlo dos ciclos devem ser `let`. Há ainda o ciclo **estendido** `for (variável in objeto)` (ver "Objetos e arrays").

```
for (let x = 1; x < 6; x++) {
   document.write(x + " é ");
   if (x % 2 == 0) {
     document.writeln("par");
   } else {
     document.writeln("ímpar");
   }
}
```

Escreve `1 é ímpar`, `2 é par`, `3 é ímpar`, `4 é par`, `5 é ímpar`; com `x = 6` a condição falha e o ciclo acaba.

> **Definição**
> **Declaração**: `function nome(parâmetros) { definição }`, sem `;` (é uma declaração, não uma instrução). Os parâmetros não têm tipo. A definição pode ter `return`, cujo valor fica para quem invoca.
> **Invocação**: `nome(argumentos)`. Os argumentos atribuem-se **por posição** aos parâmetros, avalia-se a definição, e o valor de retorno **substitui** a expressão de invocação. Ex.: em `soma(2, 3)` com `return a + b`, `a = 2`, `b = 3`, e `soma(2, 3)` vale `5`.

Âmbito de variáveis

Uma variável é **local** à função onde é declarada, ou **global**. Cada função cria um **novo âmbito**, e as locais **escondem** (*shadowing*) as globais com o mesmo nome. O **`var` ignora blocos** (o seu âmbito é a função); `let`/`const` respeitam o bloco.

Exemplo resolvido

```
function testarAmbito() {
  var a = "função";
  let b = "função-let";
  if (true) {
    var a = "bloco";      // a MESMA 'a' (var é de função)
    let b = "bloco-let";  // uma 'b' NOVA (let é de bloco)
    console.log("dentro do bloco:", a, b);
  }
  console.log("fora do bloco:", a, b);
}
testarAmbito();
```

1. `var a = "função"`: `a` (âmbito de função) vale `"função"`.
2. `let b = "função-let"`: `b` do corpo da função.
3. Entra no `if`: novo âmbito de bloco.
4. `var a = "bloco"`: o `var` ignora o bloco, por isso **reatribui** a mesma `a`, que passa a `"bloco"` em toda a função.
5. `let b = "bloco-let"`: cria uma `b` **nova**, só do bloco, que esconde a de fora.
6. Escreve `dentro do bloco: bloco bloco-let`.
7. Sai do bloco: a `b` do bloco desaparece; a de fora nunca mudou.
8. Escreve `fora do bloco: bloco função-let`.

É por isto que se evita o `var`: uma variável de ciclo "vaza" para fora do `if`/`for`/`while`.

Elevação (*hoisting*)

As declarações `var` e `function` são **elevadas** para o início do seu âmbito: é como se estivessem lá escritas, mas a **inicialização** fica onde está. Pode-se usar uma variável ou função **antes** da linha da declaração, mas a declaração tem de existir algures.

Exemplo resolvido

```
document.writeln(x);
document.writeln(getX());
x = 1;
document.writeln(x);

function getX() {
    var x = 2;
    return x;
}

var x;
```

O `var x` sobe **sem valor**; a `function getX` sobe **inteira** (as declarações de função sobem completas). O motor vê:

```
var x;                                  // undefined
function getX() { var x = 2; return x; }
document.writeln(x);
document.writeln(getX());
x = 1;
document.writeln(x);
```

1. `x` existe mas não tem valor: escreve `undefined`.
2. `getX()` usa o seu `x` **local** (`var x = 2`, outro âmbito): escreve `2`.
3. `x = 1` atribui ao `x` global.
4. Escreve `1`.

Resultado: `undefined`, `2`, `1`. Sem `var x` em lado nenhum, a primeira leitura daria `ReferenceError`. `let`/`const`† também são elevadas, mas usá-las antes da linha da declaração dá erro (*temporal dead zone*).

Funções anónimas

Uma função pode ser uma **expressão**, sem nome: atribui-se a uma variável ou **invoca-se logo** (IIFE, *Immediately Invoked Function Expression*), o que também cria um âmbito próprio para variáveis temporárias. Há ainda a sintaxe **seta**, `=>`.

```
let somar = function(a, b) { return a + b; };
document.writeln(somar(2, 3));                              // 5
document.writeln((function(a, b) { return a + b; })(2, 3)); // 5 (IIFE)
document.writeln(((a, b) => a + b)(2, 3));                  // 5 (seta)
```

Na seta, um corpo que é uma só expressão é devolvido sem `return` nem chavetas. Os parêntesis à volta da função† fazem dela uma **expressão**; sem eles, `function` seria lida como o início de uma declaração e a chamada a seguir daria erro de sintaxe.

Modo estrito

A diretiva `"use strict"` (uma *string*, que motores antigos ignoram) torna a linguagem mais estrita. **Global**, no início do ficheiro, ou **local**, no início de uma função. Proíbe: usar variáveis e funções **sem as declarar** (`x = 1` dá erro); **remover** (`delete`) variáveis e funções; a construção `with`; e usar como identificadores palavras reservadas para versões futuras.

Objetos e arrays

Os objetos criam-se com `new`. Há classes predefinidas, cada uma com **métodos** (`hoje.getFullYear()`) e **propriedades** (`'ola mundo'.length`): `String`, `Date`, `RegExp`, `Object`. As aspas de `'ola mundo'` são açúcar sintático para `new String('ola mundo')`.

> **Definição**
>
> - **`undefined`**: variável ou propriedade **nunca atribuída**, ou propriedade que **não existe**.
> - **`null`**: objeto **explicitamente** marcado como inexistente (é o programador que o atribui).
>
> `null == undefined` é `true` (regra especial), mas `null === undefined` é `false`. Um objeto real nunca é == a `null`.

> **Definição**
>
> **Propriedades** criam-se **dinamicamente** ao inicializá-las, sem declaração, e só existem no objeto onde foram criadas. Ler uma que não existe dá `undefined`. O operador **`in`** diz se ela existe.
> **Arrays** são objetos da classe `Array`; a dimensão é opcional na construção. Acedem-se com `[ ]`, e índices **para lá** da dimensão fazem o array crescer. Índices não atribuídos dão `undefined`.
> **Propriedade = índice**: `obj.nome`, `obj['nome']` e `obj[prop]` (com `prop = 'nome'`) são a **mesma** propriedade. Com `.` a chave é um **literal** com as regras dos nomes de variáveis; com `[ ]` é uma **expressão** qualquer, convertida para *string*.
> **`delete`** remove uma propriedade ou índice (`delete obj['nome']`, `delete obj.estado`); depois disso dá `undefined` e `in` dá `false`. Num array, `delete` deixa um **buraco**† (o `length` não muda); para tirar e fechar o buraco usa-se `array.splice(indice, 1)`.

Exemplo resolvido

```
const obj1 = new Object();
const obj2 = new Object();
obj1.nome = 'Objeto um';
document.writeln(obj1.nome);       // Objeto um
document.writeln(obj2.nome);       // undefined
document.writeln('nome' in obj1);  // true
document.writeln('nome' in obj2);  // false

const array = new Array(2);
array[0] = 'Início';
array[1] = 123;
array[3] = true;                   // salta o 2: o array cresce
for (let i = 0; i < 5; i++)
  document.writeln(i + ':' + array[i]);
```

O ciclo escreve `0:Início`, `1:123`, `2:undefined` (nunca atribuído), `3:true` e `4:undefined` (para lá do que foi atribuído).

Igualdade: == e ===

O === só dá `true` com o **mesmo tipo** e o mesmo valor; o == **converte** um dos lados primeiro.

Expressão	==	===	Porquê
`1` e `"1"`	true	false	`"1"` convertida para `1`
`null` e `undefined`	true	false	regra especial
`0` e `""`	true	false	`""` convertida para `0`
`NaN` e `NaN`	false	false	`NaN` nunca é igual a nada, nem a si próprio

`NaN` (*Not a Number*, ex. `0/0`) testa-se com `Number.isNaN(x)`†, nunca com `x == NaN`.

Iteração

> **Definição**
>
> - **`for (variável in objeto)`** percorre as **chaves**. A variável deve ser declarada com `let` (ou já existir).
> - **`for (variável of iterável)`** percorre os **valores** de uma coleção iterável (arrays, `Maps`, *strings*).
>
> Num array, `in` dá os **índices** e `of` os **valores**.

```
const pessoa = new Object();
pessoa.nome  = 'Ninguém';
pessoa.idade = 99;
pessoa.ativo = true;
for (let campo in pessoa)
   document.writeln(campo + ' = ' + pessoa[campo]);
// nome = Ninguém / idade = 99 / ativo = true

const numeros = [null, 'primeiro', 'segundo', 'terceiro'];
for (let indice in numeros) document.writeln(indice);  // 0 1 2 3
for (let valor of numeros) document.writeln(valor);    // null primeiro segundo terceiro
```

Os arrays têm também **métodos de iteração**†, que recebem uma função:

```
const numeros = [1, 2, 3, 4];
numeros.forEach(n => document.writeln(n * 10));        // 10 20 30 40, não devolve nada
const dobros = numeros.map(n => n * 2);                // [2,4,6,8]
const pares  = numeros.filter(n => n % 2 == 0);        // [2,4]
const soma   = numeros.reduce((acc, n) => acc + n, 0); // 0+1+2+3+4 = 10
```

`forEach(f)` chama `f(valor, indice, array)` por elemento; `map(f)` faz um array **novo** com os resultados; `filter(f)` um array novo com os elementos em que `f` dá *truthy*; `reduce(f, inicial)` acumula tudo num valor. Nenhum altera o array original.

Map

A classe `Map` cria arrays associativos previsíveis (as chaves de um objeto podem incluir, por exemplo, métodos herdados).

```
const pessoa = new Map();
pessoa.set('nome', 'Ninguém');
pessoa.set('idade', 99);
pessoa.set('ativo', true);
for (let dado of pessoa)
    document.writeln(dado[0] + " = " + dado[1] + "\n" + pessoa.get(dado[0]));
```

O `for-of` dá, em cada volta, um par `[chave, valor]`; `get(chave)` lê um valor. Escreve `nome = Ninguém`, `Ninguém`, `idade = 99`, `99`, `ativo = true`, `true`. O `for-in` **não** funciona em `Maps`.

	`Map`	Objeto
Chaves	**qualquer valor** (objetos, funções)	só tipos simples (*strings*, símbolos)
Ordem	a de **inserção**, sempre	"lexicográfica"†: na prática, chaves que parecem inteiros primeiro (por ordem numérica), depois as outras pela ordem de inserção
Tipo	um `Map` é um objeto	um objeto não é um `Map` (não tem `set`/`get`)

Notação de objetos, construção e desconstrução

Objetos escrevem-se `{ nome: valor, ... }` e arrays `[ valor, ... ]`, e um valor estruturado segue o mesmo formato:

```
const pessoa = {
    nome: 'Fulano de tal',
    idade: 99,
    morada: { endereco: 'Rua Nova', codPostal: '4000-001 PORTO' },
    hobbies: ['sky diving', 'graffiti']
};
document.writeln(pessoa.morada.endereco);   // Rua Nova
document.writeln(pessoa.hobbies[1]);        // graffiti
```

Construção abreviada: se as variáveis têm o nome das propriedades, `{ nome, idade, morada }` é o mesmo que `{ nome: nome, idade: idade, morada: morada }`.

A **desconstrução** (*destructuring*) faz o inverso: um "molde" à esquerda de uma atribuição extrai variáveis de uma estrutura.

Exemplo resolvido

```
let hobbies = ['sky diving', 'graffiti'];
let [primeiro, segundo] = hobbies;
console.log(primeiro, segundo);              // sky diving graffiti
[segundo, primeiro] = [primeiro, segundo];   // troca, SEM let
console.log(primeiro, segundo);              // graffiti sky diving
```

1. Nos **arrays** emparelha-se **pela posição**: `primeiro = hobbies[0]`, `segundo = hobbies[1]`.
2. Na troca, avalia-se primeiro o lado direito com os valores atuais, `['sky diving', 'graffiti']`, e depois desconstrói-se: `segundo` fica `'sky diving'` e `primeiro` fica `'graffiti'`. Leva **sem** `let`: `let` de novo daria `SyntaxError` (variável já declarada).

```
const pessoa = { nome: 'Fulano de tal', idade: 99,
    morada: { endereco: 'Rua Nova', codPostal: '4000-001 Porto' } };
let { nome, idade, morada } = pessoa;
let { nome: name, morada: { endereco: street, codPostal: zip } } = pessoa;
console.log(name, street, zip);   // Fulano de tal Rua Nova 4000-001 Porto
```

Nos **objetos** emparelha-se **pelo nome**. `nome: name` **renomeia** (cria `name` com o valor de `pessoa.nome`), e o molde pode **aninhar-se** para descer em `morada`. Propriedades desnecessárias omitem-se.

Programação orientada a objetos

> **Definição**
> O ES6 (ECMAScript 2015) trouxe a sintaxe `class`, para **estruturação** (hierarquia de componentes), **reutilização** (herança) e **encapsulação**: definir classes, instanciar objetos, associar **campos** e **métodos**, e **estender** classes.
> Mas `class` é **açúcar sintático** sobre os objetos e protótipos que já existiam: as classes são **opcionais**, **não há encapsulação real** (tudo o que está em `this` é acessível de fora), e só se reutilizam **definições**, não declarações (interfaces).

Um objeto **sem classe** escreve-se em notação de objetos, com métodos como propriedades-função. Serve para objetos únicos:

```
let joao = {
    nome: "João",
    nasce: new Date(1999, 3 - 1, 15),   // meses contam de 0: 2 = março
    hobbies: ["programação", "mergulho"],
    idade: function() {
        var agora = new Date();
        var msAno = 1000 * 60 * 60 * 24 * 365.25;
        return Math.floor((agora.getTime() - this.nasce.getTime()) / msAno);
    }
};
```

Em `joao.idade()`, `this` é o próprio `joao`.

> **Definição**
> - `class Nome { ... }` define uma classe.
> - O método `constructor(...)` inicializa cada nova instância.
> - Nos métodos, **`this`** é o **objeto corrente**.
> - Os **campos** são propriedades atribuídas a `this` (normalmente no `constructor`); não há declaração separada de campos.
> - `new Nome(argumentos)` cria uma instância e passa os argumentos ao `constructor`.
> - **Métodos** são funções da classe; usam campos e outros métodos através de `this` e podem devolver um valor.

Exemplo resolvido

```
class Pessoa {
    constructor(nome, ano, mes, dia) {
        this.nome  = nome;
        this.nasce = new Date(ano, mes - 1, dia);
    }
    saudacao(outro) {
        return 'Olá ' + this.nome + ', eu sou o ' + outro.nome;
    }
    saudar(outro) {
        console.log(this.saudacao(outro));
    }
}
let joao  = new Pessoa("João", 1999, 3, 15);
let maria = new Pessoa("Maria", 2000, 5, 21);
joao.saudar(maria);
```

1. `new Pessoa("João", 1999, 3, 15)` cria um objeto e corre o `constructor` com `this` = esse objeto: `this.nome = "João"`, `this.nasce` = 15 de março de 1999 (`mes - 1 = 2`, porque o `Date` conta os meses a partir de 0). O mesmo para `maria`.
2. `joao.saudar(maria)`: `this = joao`, `outro = maria`.
3. `this.saudacao(outro)` mantém `this = joao`. Devolve `'Olá ' + joao.nome + ', eu sou o ' + maria.nome`, ou seja, `"Olá João, eu sou o Maria"`.
4. `saudar` escreve isso na consola.

Ler um dado de outra instância (`outro.nome`) é uma **troca de mensagens** entre objetos, a ideia central da orientação a objetos.

Acessores e membros estáticos

Os **acessores** `get`/`set` são métodos que medeiam o acesso a uma propriedade, mas **usam-se como a propriedade**, sem parêntesis.

Exemplo resolvido

```
const MS_ANO = 1000 * 60 * 60 * 24 * 365.25;
class Pessoa { // ...
  get idade() {
      let agora = new Date();
      return Math.floor((agora.getTime() - this.nasce.getTime()) / MS_ANO);
  }
  set idade(anos) {
      let agora = new Date();
      this.nasce = new Date(agora.getTime() - anos * MS_ANO);
  }
}
joao.idade = 99;                                          // SET
document.writeln("O " + joao.nome + " tem " + joao.idade);  // GET
```

`joao.idade = 99` não cria a propriedade `idade`: chama `set idade(99)`, que recalcula `this.nasce` para quem tem 99 anos hoje. Ler `joao.idade` chama `get idade()`, que calcula a idade a partir de `this.nasce`: dá 99. Um acessor guarda o estado **noutro campo** (`nasce`): se o `get idade` fizesse `this.idade = ...`, chamaria o `set`, que podia chamar o `get`, num ciclo infinito.

Um método **`static`** pertence à **classe**, não a uma instância: não tem `this` e invoca-se com o **nome da classe**.

```
class Pessoa {
    static duracao(inicio, fim) {
        return (fim.getTime() - inicio.getTime()) / MS_ANO;
    }
    idade() {
        return Pessoa.duracao(new Date(), this.nasce);
    }
}
```

Em `joao.idade()`, `this = joao`; `Pessoa.duracao(...)` corre só com os parâmetros `inicio` e `fim`, sem `this`.

Herança

`class B extends A` cria uma **subclasse** que herda tudo de `A`, pode **redefinir** métodos (*override*) e acrescentar novos. **`super`** dá acesso às definições da superclasse **imediata**.

Exemplo resolvido **Estender a Pessoa (slides)**

```
class Aluno extends Pessoa {
  saudacao(outro) {
     return 'Oi ' + this.nome + ', tudo em cima? Aqui ' + outro.nome + '!';
  }
  saudarCareta(outro) {
     console.log(super.saudacao(outro));
  }
}
let marco = new Aluno("Marco", 2001, 1, 13);
marco.saudar(joao);        // Oi Marco, tudo em cima? Aqui João!
marco.saudarCareta(joao);  // Olá Marco, eu sou o João
```

1. `Aluno` não tem `constructor`: usa o de `Pessoa`.
2. `marco.saudar(joao)`: `saudar` é **herdado** de `Pessoa` e chama `this.saudacao(outro)`. Como `this = marco` é um `Aluno`, corre a versão **redefinida** em `Aluno`: `"Oi Marco, tudo em cima? Aqui João!"`.
3. `marco.saudarCareta(joao)`: `super.saudacao(outro)` salta a redefinição e corre a de `Pessoa`: `"Olá Marco, eu sou o João"`.

Exemplo resolvido **Dois níveis de extends**

```
class Pessoa {
  constructor(nome) { this.nome = nome; }
  saudacao() { return `Olá, sou ${this.nome}`; }
}
class Aluno extends Pessoa {
  constructor(nome, curso) {
    super(nome);                // constructor de Pessoa
    this.curso = curso;
  }
  saudacao() { return `${super.saudacao()} e estudo ${this.curso}`; }
}
class Finalista extends Aluno {
  constructor(nome, curso, anoFinal) {
    super(nome, curso);         // constructor de Aluno
    this.anoFinal = anoFinal;
  }
  saudacao() { return `${super.saudacao()} (ano final: ${this.anoFinal})`; }
}
let f = new Finalista("Marco", "Eng. Informática", 2026);
```

Construção:

1. `new Finalista(...)` cria um objeto `f` e corre o `constructor` de `Finalista` com `this = f`.
2. `super(nome, curso)` corre o `constructor` de `Aluno` com o **mesmo** `this = f`.
3. Lá dentro, `super(nome)` corre o de `Pessoa`: `f.nome = "Marco"`.
4. De volta a `Aluno`: `f.curso = "Eng. Informática"`.
5. De volta a `Finalista`: `f.anoFinal = 2026`.

`f` é **um** objeto com os três campos.

Exemplo resolvido

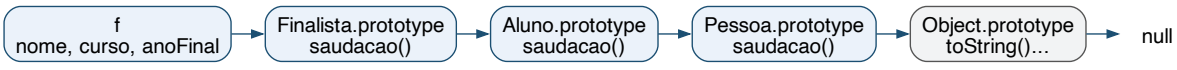
`f.saudacao()`:

1. A procura começa em `Finalista`, que define `saudacao`. Aí `this = f` (o objeto à esquerda do ponto, em qualquer nível).
2. `super.saudacao()` em `Finalista` chama a versão de `Aluno`, ainda com `this = f`.
3. `super.saudacao()` em `Aluno` chama a de `Pessoa`: devolve `"Olá, sou Marco"`.
4. `Aluno` junta `" e estudo Eng. Informática"`.
5. `Finalista` junta `" (ano final: 2026)"`:

```
Olá, sou Marco e estudo Eng. Informática (ano final: 2026)
```

O `this` é **sempre** o objeto que recebeu a chamada. O `super` é a classe **acima daquela onde está escrito**: em `Finalista` vai a `Aluno`, em `Aluno` vai a `Pessoa`.

Por baixo†, cada objeto aponta (`__proto__`) para o `.prototype` da sua classe, e cada `.prototype` para o da superclasse, até `Object.prototype` e `null`. Procurar um método é subir esta **cadeia de protótipos** até à primeira definição.



Callbacks e classes anónimas

Uma função pode passar-se como argumento: `setTimeout(funcao, ms)` chama `funcao` ao fim de `ms` milissegundos.

```
class Pessoa { // ...
   alerta(quando, mensagem) {
       let callback = () => this.alertar(mensagem);
       let millisecs = (quando.getTime() - (new Date()).getTime());
       setTimeout(callback, millisecs);
   }
   alertar(mensagem) {
       alert("Olá " + this.nome + "\nSão horas de " + mensagem);
   }
}
```

O `callback` tem de ser uma **seta**†. Uma função normal recebe o `this` de **como é invocada**, e o `setTimeout` chama-a solta (sem objeto à esquerda do ponto): o `this` seria `undefined` (modo estrito) ou o objeto global. A seta **não tem `this` próprio**: usa o do sítio onde foi **escrita** (o método `alerta`, onde `this` é a `Pessoa`).

As classes também podem ser **anónimas**, como expressões: `let Pet = class { constructor(nome) { this.nome = nome; } };` e depois `new Pet("Bola")`. Só existem através da variável que as guarda.

6 Tema 6 JavaScript: DOM, CSSOM e eventos

Reflexão

Definição

A **reflexão** representa a formatação da página como **objetos**, que o código pode **ler** e **alterar**. Há três modelos:

- **widgets**: o mais antigo, só para formulários; evitar;
- **CSSOM** (*CSS Object Model*): o estilo de cada elemento;
- **DOM** (*Document Object Model*): a estrutura inteira do documento.

Widgets. Cada campo de formulário (`input`, `select`, `textarea`) é um objeto com `name` e `value`. Num *callback*, `this` é o elemento corrente; `this.form` é o seu formulário, que tem uma propriedade por **nome** de campo:

```
<form>
  <input name=a size=2 value=1> +
  <input name=b size=2 value=1>
  <input type=button value="=" onclick="
     this.form.c.value =
       parseInt(this.form.a.value) +
       parseInt(this.form.b.value);">
  <input name=c size=2>
</form>
```

Ao clicar em "=": `this` é o botão, `this.form.a` e `this.form.b` são os campos pelo `name`; os valores (*strings*) passam a inteiros com `parseInt` e somam-se, e o resultado vai para `this.form.c.value` (mostrado como texto).

CSSOM. Todo o elemento tem a propriedade **`style`**, um objeto com o estilo **em linha** do elemento, que se pode **inspecionar** e **alterar**: `element.style.background = "orange"`. Os nomes **simples** são iguais aos do CSS (`color`, `width`); os nomes **com hífen** passam a **lowerCamelCase** (`font-size` → `fontSize`, `border-style` → `borderStyle`), porque `style.font-size` seria lido como a subtração `style.font - size`†.

```
<select onchange="this.form.style.color = this.value;">
  <option></option> <option>orange</option> <option>DarkBlue</option>
</select>
<select onchange="this.form.style.fontSize = this.value;">
  <option>24pt</option> <option>36pt</option> <option>12pt</option>
</select>
```

Ao escolher "orange", `this` é o `<select>` e `this.value` é `"orange"`: o formulário fica com `style.color = "orange"` e o navegador repinta-o. O segundo `<select>` muda `style.fontSize` (o `font-size` do CSS).

A árvore DOM

Definição

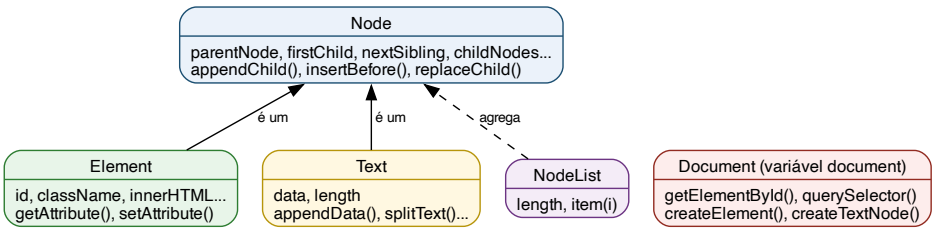
O **DOM** reflete o documento **inteiro** como uma árvore de objetos, com uma API para a consultar e alterar. Serve também para XML (o HTML só acrescenta algumas propriedades). Tipos:

- `Document`: o documento como um todo;
- `Node`: o tipo **mais geral** de nó;
- `Element`: nós criados por pares de anotações;
- `Text`: nós de texto pendurados nos elementos;
- `NodeList`: uma **coleção** de nós (não é um nó).

Cada nó tem **um** progenitor (exceto a raiz). `Element` e `Text` são especializações de `Node` (herdam as suas propriedades e métodos). Os **espaços em branco** entre anotações (mudanças de linha, indentação) também dão nós de texto.

```
<html>
  <head> <title>Uma página</title> </head>
  <body>
    <h1>Uma página</h1>
    <p>Uma linha</p>
    <p>Ainda <b>outra</b> linha</p>
  </body>
</html>
```

`html` tem os filhos `head` e `body`; `head` tem `title`, com o texto `"Uma página"`; `body` tem `h1` (texto `"Uma página"`), um `p` (texto `"Uma linha"`) e outro `p` com três filhos: o texto `"Ainda "`, o elemento `b` (texto `"outra"`) e o texto `" linha"`.



`Document`

A variável global **`document`** dá acesso à árvore, para **obter** elementos e **criar** nós.

- `document.getElementById(id)`: o elemento com esse `id` (ou `null`).
- `querySelector(s)`: o **primeiro** elemento que corresponde ao seletor CSS `s`; `querySelectorAll(s)`: **todos**, numa `NodeList`. Também existem nos **elementos**, para procurar só lá dentro.
- `document.createElement(tag)` cria um `Element`; `document.createTextNode(texto)` cria um `Text`. Os nós criados ficam **soltos**: só aparecem depois de ligados à árvore (`appendChild`, …).

```
document.getElementById("base").style.background = color;
const firstParagraph     = document.querySelector('div > p');    // o 1.º
const allFirstParagraphs = document.querySelectorAll('div > p'); // todos
const dentro = algumDiv.querySelector('p');           // só dentro de algumDiv
```

Node

`Node` tem as propriedades e métodos **estruturais**, comuns a todos os nós:

Propriedade ou método	Significado
`parentNode`	o progenitor
`firstChild`, `lastChild`	o primeiro / último filho
`nextSibling`, `previousSibling`	o irmão seguinte / anterior
`childNodes`	`NodeList` dos filhos
`innerText` (HTML)	o texto do nó e dos descendentes
`appendChild(no)`	junta `no` como **último** filho
`insertBefore(no, ref)`	insere `no` **antes** do filho `ref` (com `ref = null`, no fim)
`replaceChild(depois, antes)`	troca o filho `antes` por `depois`

Exemplo resolvido

Cada clique numa caixa cria outra igual **no fim** de `#base`:

```
function create(color) {
  const base = document.getElementById("base");
  const box = document.createElement("div");
  const space = document.createTextNode(" ");
  box.style.background = color;
  box.style.width = "1em";
  box.style.margin = "auto";
  box.onclick = function() { create(color); };
  box.appendChild(space);
  base.appendChild(box);
  base.appendChild(document.createTextNode("\n"));
}
```

1. `base` é obtido com `getElementById`.
2. `box` (um `div`) e `space` (o texto `" "`) são criados, ainda soltos.
3. O estilo de `box` é posto pelo CSSOM.
4. `box.onclick` recebe uma função que chama `create` com a mesma cor: a caixa nova também é clicável.
5. `box.appendChild(space)`: `space` passa a filho de `box`.
6. `base.appendChild(box)`: `box` passa a **último filho** de `base`; só aqui aparece no ecrã.
7. Junta-se um texto `"\n"` depois da caixa.

Exemplo resolvido

Inserir a caixa nova **a seguir à clicada**, registando `onclick="create('#f90', this);"` em cada caixa:

```
function create(color, current) {
  const base = document.getElementById("base");
  const box = document.createElement("div");
  const space = document.createTextNode(" ");
  const nl = document.createTextNode("\n");
  const next = current.nextSibling.nextSibling;
  box.style.background = color;
  box.style.width = "1em";
  box.style.margin = "auto";
  box.onclick = function() { create(color); };
  box.appendChild(space);
  base.insertBefore(box, next);
  base.insertBefore(nl, next);
}
```

1. `next = current.nextSibling.nextSibling`: cada caixa é seguida de um texto `"\n"`, por isso avança-se **dois** irmãos para chegar à caixa seguinte (ou a `null`, se `current` é a última).
2. `insertBefore(box, next)` põe `box` logo a seguir a `current`; se `next` é `null`, no fim (o mesmo que `appendChild`).
3. `insertBefore(nl, next)` põe o `"\n"` entre a caixa nova e `next`.

`NodeList`, `Element` e `Text`

Uma **`NodeList`** agrega nós: os filhos de um elemento, ou o resultado de uma pesquisa (`getElementsByTagName`, `querySelectorAll`). Tem `length` e `item(indice)` (a partir de 0).

```
let conta = 0;
function contar() {
    const base = document.getElementById("base");
    const lista = base.getElementsByTagName("div");
    conta++;
    for (let i = 0; i < lista.length; i++)
        lista.item(i).innerText = conta;
}
```

Registada como `onclick` das caixas: a `NodeList` tem todos os `div` de `base`; a global `conta` sobe 1 a cada clique, e o ciclo escreve o novo valor em **todas** as caixas, não só na clicada.

Os **`Element`** são os nós estruturais (os `Text` são folhas). Obtêm-se com as pesquisas de `Document` ou com `createElement`.

`Element`	
`getAttribute(nome)`	lê um atributo
`setAttribute(nome, valor)`	altera (ou cria) um atributo
`getElementsByTagName(n)`	`NodeList` dos descendentes com a anotação `n`
`className`	as classes CSS (*string*, como o atributo `class`)
`id`	o identificador
`clientHeight`, `clientWidth` (HTML)	altura / largura **interna**
`innerHTML` (HTML)	o HTML do **conteúdo**, como *string*
`appendChild`, `childNodes`, `firstChild`, `lastChild`	herdados de `Node`

```
box.setAttribute("style", "background: " + color + "; width: 1em; margin: auto;");
box.setAttribute("onclick", "create('" + color + "');");
```

Com `setAttribute` o `style` é escrito como texto CSS e o `onclick` como uma *string* de código, tal como no HTML. Atribuir uma função à propriedade (`box.onclick = function() {...}`) é preferível†: não mistura código em *strings*.

Um **`Text`** tem `data` (o texto) e `length`. Métodos, com posições a partir de 0: `substringData(offset, count)` (devolve `count` carateres a partir de `offset`), `appendData(string)`, `insertData(offset, string)`, `deleteData(offset, count)`, `replaceData(offset, count, string)` e `splitText(offset)` (parte o nó em dois: o original fica com o início e o novo nó, devolvido, com o resto, como irmão seguinte).

Eventos

> **Definição**
>
> A programação tradicional é **proativa**: começa numa função (`main`), que invoca outras e recebe o controlo de volta. A programação **por eventos** é **reativa**: um troço de código só corre **em resposta a um evento**, representado por um **objeto** passado à função **registada** para esse tipo de evento nessa origem.
> Um registo tem três partes: o **elemento**, o **tipo** de evento e o ***callback***. Os tipos (janela, teclado, rato, *widgets*, …) dão origem a **atributos** HTML e **propriedades** DOM com o prefixo `on`, e ao primeiro **argumento** de `addEventListener`.

Modo 1: atributo HTML. O atributo `on` + evento (`onclick`) tem como valor uma *string* de código. Qualquer elemento os pode ter, e `this` é o elemento.

```
<div style="..." onclick="with(this.style) {
     let tmp = color;
     color = background;
     background = tmp;
   }"> Don't click me! </div>
```

Modo 2: propriedade do elemento. Separa o HTML do JavaScript: uma **função** (normalmente anónima) vai para a propriedade com o mesmo nome do atributo. O elemento obtém-se com `getElementById`, e por isso só depois de a janela carregar (`window.onload`).

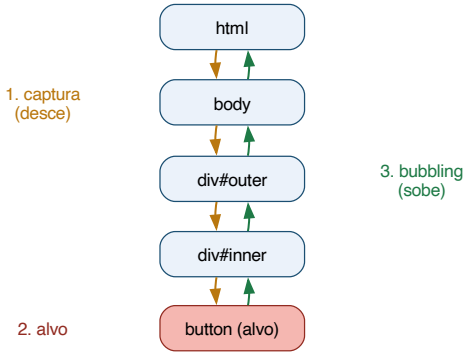
```
window.onload = function() {
   const button = document.getElementById("button");
   button.onclick = function() {
      const tmp = button.style.color;
      button.style.color = button.style.background;
      button.style.background = tmp;
   };
};
```

Nos modos 1 e 2, um novo *callback* **substitui** o anterior†: só há um por evento e elemento.

Modo 3: `addEventListener` (DOM Level 2). Permite **vários** *callbacks* no mesmo elemento e evento. Argumentos: o **nome** do evento **sem** `on` (`"click"`), o ***callback***, e a **fase** (`true` = *capturing*, `false` = *bubbling*).

```
window.addEventListener("load", function() {
   const button = document.getElementById("button");
   button.addEventListener("click", function() {
      const tmp = button.style.color;
      button.style.color = button.style.background;
      button.style.background = tmp;
   }, false);
}, false);
```

Fases. Os elementos sobrepõem-se (cada um contém os descendentes), por isso o evento é entregue a **todos** os elementos no caminho entre a raiz e o alvo. Havia duas convenções: ***capturing***, do mais **externo** para o mais **interno** (Netscape), e ***bubbling***, do mais **interno** para o mais **externo** (Microsoft). O DOM **combina-as**: a captura desce da raiz ao alvo, o alvo processa, e o *bubbling* sobe de volta. O terceiro argumento de `addEventListener` escolhe a fase do *callback*; o habitual é `false` (também a omissão†); a captura serve para **intercetar** um evento antes de chegar ao alvo.



O objeto `Event`, passado ao *callback*, tem informação sobre o evento e controla a propagação. Propriedades (exemplos): `bubble` (está na fase de *bubbling*), `cancelBubble` (cancela o *bubbling*). Métodos: `preventDefault()` (cancela a **ação por omissão**, se possível: navegar num *link*, começar uma seleção de texto) e `stopPropagation()` (**para** a propagação). Há **classes especializadas** por categoria de evento.

Janela. O evento `load` (a página e os recursos acabaram de carregar) é da janela: `window.onload = ...` ou `window.addEventListener("load", ...)`. Espera-se por ele para obter elementos com `getElementById`.

Evento	Quando
Rato	propriedades `x`, `y`, `button`, `ctrlKey`
`click`	pressão e fim no mesmo sítio
`doubleclick`	dois cliques seguidos
`mousedown`, `mouseup`	início / fim da pressão de um botão
`mousemove`	movimento
Teclado	propriedades `key`, `keyCode`, `ctrlKey`, `shiftKey`, `altKey`
`keypress`	tecla que gera um carácter
`keydown`, `keyup`	início / fim da pressão de uma tecla

Exemplo resolvido **Arrastar com o rato**

```
window.onload = function() {
   const dot = document.getElementById("dot");
   let x;
   let y;
   let moving = false;
   dot.onmousedown = function(event) {
      x = event.x;
      y = event.y;
      moving = true;
      event.preventDefault();
   };
   dot.onmousemove = function(event) {
      if (!moving) return;
      dot.style.left = (parseInt(dot.style.left) + (event.x - x)) + "px";
      dot.style.top = (parseInt(dot.style.top) - (y - event.y)) + "px";
      x = event.x;
      y = event.y;
      event.preventDefault();
   };
   dot.onmouseup = function() { moving = false; };
};
```

O contexto é criado com a página carregada (`dot` já existe); `x` e `y` guardam a última posição do cursor e `moving` se o ponto está a ser arrastado.

1. **`mousedown`**: guarda a posição do cursor, `moving = true`, e `preventDefault()` evita que o navegador comece uma seleção de texto.
2. Cada **`mousemove`** com `moving = true`: o deslocamento horizontal é `event.x − x`; soma-se ao `left` atual (`"75px"` passa a número com `parseInt`) e escreve-se de volta com `"px"`. O mesmo para `top` (o `y` do ecrã cresce para baixo). Depois guarda a posição atual, para o próximo movimento contar a partir daqui.
3. **`mouseup`**: `moving = false`; os `mousemove` seguintes saem logo no `return`. Sem ele, o ponto ficava colado ao cursor.

Exemplo resolvido **Mover com o teclado**

```
const dot = document.getElementById("dot");
document.onkeypress = function(event) {
   let left = parseInt(dot.style.left);
   let top = parseInt(dot.style.top);
   switch (event.key) {
      case "k": case "K": left += 10; break;
      case "j": case "J": left -= 10; break;
      case "i": case "I": top -= 10; break;
      case "m": case "M": top += 10; break;
   }
   dot.style.left = left + "px";
   dot.style.top = top + "px";
};
```

O teclado não tem posição no ecrã, por isso regista-se em `document`. Ao premir “K”: lê-se `left/top` do ponto como inteiros; `event.key` é `"k"` ou `"K"` (com *shift*), e ambos os casos somam 10 a `left`; o ponto anda 10 pixels para a direita.

Exemplos completos

Exemplo resolvido **Contador**

Um *widget* feito só com JavaScript e DOM sobre um `<div id=base>` vazio, com o aspeto numa folha CSS (`className`) e métodos como *callbacks*:

```
class Counter {
    constructor(parentId) {
        const parent = document.getElementById(parentId);
        const counter = document.createElement("div");
        const incr = document.createElement("div");
        const reset = document.createElement("div");
        this.display = document.createElement("div");
        this.count = 0;
        parent.appendChild(counter);
        counter.appendChild(incr);
        counter.appendChild(reset);
        counter.appendChild(this.display);
        counter.className = "counter";
        incr.className = "button";
        reset.className = "button";
        this.display.className = "display";
        incr.innerText = " Incr ";
        reset.innerText = " Reset ";
        this.display.innerText = this.count;
        incr.onclick = () => this.increment();
        reset.onclick = () => this.reset();
    }
```

Exemplo resolvido **Contador (continuação)**

```
    increment() { this.count++; this.display.innerText = this.count; }
    reset()     { this.count = 0; this.display.innerText = this.count; }
}
window.onload = function() { const counter = new Counter("base"); };
```

```
<script src="counter.js"></script>
<link rel="stylesheet" href="counter.css" type="text/css">
<div id=base></div>
```

Só `display` e `count` ficam na instância; `counter`, `incr` e `reset` não são precisos depois de construídos. Os slides mostram duas formas de ligar os métodos: `incr.onclick = this.increment.bind(this)` e `incr.onclick = () => this.increment()`. Um método passado diretamente (`incr.onclick = this.increment`) teria `this` = o elemento clicado; o `bind(this)` cria uma função com o `this` fixo, e a seta usa o `this` do construtor (a instância).

1. **Carregamento**: `new Counter("base")` cria os quatro `div`, liga-os (`incr`, `reset` e `display` dentro de `counter`, `counter` dentro de `#base`), dá-lhes as classes CSS e o texto inicial (visor a `0`) e regista os dois `onclick`.
2. **Clique em "Incr"**: a seta chama `this.increment()` na instância: `count` passa a 1 e o visor mostra `1`.
3. **Outro clique**: `count` passa a 2, o visor mostra `2`.
4. **"Reset"**: `count = 0`, o visor mostra `0`.

Exemplo resolvido **Jogo do galo**

O mesmo método (classe + DOM + CSS), mas cada célula tem de saber a **sua posição** quando é clicada.

```
class TicTacToe {
    constructor(id) {
        const parent = document.getElementById(id);
        const board = document.createElement("div");
        board.className = "board";
        parent.appendChild(board);
        this.content = new Array(9);
        this.board = new Array(9);
        this.current = "X";
        for (let i = 0; i < 9; i++) {
            let cell = document.createElement("div");
            cell.className = "cell";
            board.appendChild(cell);
            cell.onclick = () => this.play(i);
            this.board[i] = cell;
        }
    }
    play(pos) {
        this.content[pos] = this.current;
        this.board[pos].innerHTML = this.current;
        this.current = (this.current == "X" ? "O" : "X");
    }
}
window.onload = function() { const ticTacToe = new TicTacToe("base"); };
```

Exemplo resolvido **Jogo do galo (continuação)**

```css
.board {
    width: 295px; height: 295px;
    display: grid;
    grid-template-columns: auto auto auto;
    grid-template-rows: auto auto auto;
    gap: 5px;
    background: #228;
}
.cell {
    width: 95px; height: 95px;
    background: #FAFAFA;
    font-family: sans-serif; font-size: 95px; line-height: 95px;
    text-align: center; color: orange;
    cursor: pointer;
}
```

O estado são **duas listas paralelas** de 9 posições: `content` (o valor lógico, `"X"`, `"O"` ou vazio) e `board` (o elemento DOM de cada célula, para atualizar o ecrã). `current` alterna entre `"X"` e `"O"`. O tabuleiro é uma **grelha** CSS; o fundo azul do `.board` aparece nos `gap` de 5px e faz de linhas.

Cada célula guarda o **seu** `i` porque o ciclo usa **`let`**: cada iteração cria uma ligação nova de `i`, que a seta captura. Com `var` todas as setas partilhariam o mesmo `i`, que no fim vale 9, e todos os cliques chamariam `play(9)`, fora do array.

1. **Construção**: cria o `board` (grelha 3×3), as listas, `current = "X"`, e as 9 células, cada uma com o seu `onclick` e guardada em `board[i]`.
2. **Clique na célula do meio** (posição 4): `play(4)` põe `"X"` em `content[4]` e no `innerHTML` da célula, e `current` passa a `"O"`.
3. **Clique na posição 0**: marca `"O"`, `current` volta a `"X"`.

O código não impede jogar numa célula já ocupada nem deteta o vencedor.

7 Tema 7
O protocolo HTTP

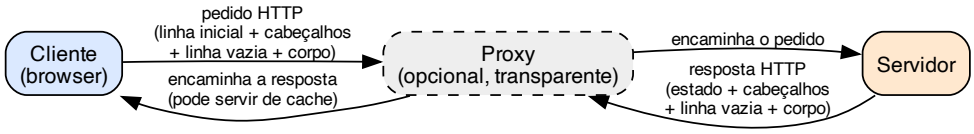
Introdução

O **HTTP** (*HyperText Transfer Protocol*) é o protocolo **aplicacional** da Web. Assenta no modelo cliente-servidor e troca **mensagens** sobre representações de **recursos** (páginas, imagens, dados). Vai na versão 2, com a 3 a ganhar adoção.

Há três tipos de **agentes**:

- **navegadores**, operados por pessoas, que pedem conteúdo em vários formatos: texto (HTML, CSS, JS), imagem e vídeo (PNG, JPEG, MPEG, Ogg), dados (JSON, XML, TXT);
- **servidores**, automáticos, que fornecem conteúdo **estático** (ficheiros em disco) ou **dinâmico** (gerado por programas no momento);
- **intermediários** (*proxies*), que encaminham pedidos e respostas e podem aplicar políticas de acesso, registar tráfego ou guardar cópias (**cache**).

O **ciclo pedido-resposta** é o átomo do HTTP: o **cliente** inicia sempre o pedido e o **servidor** responde de imediato. Os *proxies* pelo caminho são, em princípio, **transparentes**.



Definição

- **Sem conexão**: a conexão é, em princípio, refeita a cada pedido (não há um "diálogo" contínuo), o que facilita mudar de servidor entre pedidos. Desde o HTTP 1.1 a ligação TCP pode ser **reaproveitada** (*keep-alive*)†, mas isso é só uma otimização.
- **Sem estado**: o protocolo não guarda o estado da interação; só os agentes o guardam. Pode **simular-se** estado (*cookies*).
- **Independente de media**: transporta qualquer tipo de recurso que os agentes saibam interpretar; os cabeçalhos dizem o tipo.

Versões. 0.9: só `GET` (ainda suportada por compatibilidade). **1.0**: a generalidade dos métodos atuais, cabeçalhos com metainformação e segurança. **1.1**: vários pedidos pela mesma conexão e clarificações. **2.0**: baseada no SPDY da Google; mantém a semântica da 1.1 (métodos, cabeçalhos, códigos), mas transmite **pacotes binários** em vez de texto, o que permite multiplexar pedidos.

Mensagens

Definição

As mensagens são **texto** (o corpo pode ser binário), com quatro partes:

1. **linha inicial**, terminada por EOL;
2. **cabeçalhos**, um por linha, `nome: valor`;
3. **linha vazia** (dois EOL seguidos);
4. **corpo**, texto ou binário, possivelmente vazio.

O EOL do HTTP é **CR+LF** (`\r\n`)†.
Pedido: linha inicial `MÉTODO URL VERSÃO` (ex. `GET /~zp/about/index.html HTTP/1.1`); cabeçalhos com as capacidades do navegador (`Accept*`), as suas características (`User-Agent`) e a página de onde se veio (`Referer`); corpo com dados (ex. de um formulário).
Resposta: linha inicial `VERSÃO CÓDIGO DESCRIÇÃO` (ex. `HTTP/1.1 200 OK`); cabeçalhos como `Date`, `Content-Type`, `Server`; corpo (pode não haver) com o recurso.

```
GET /~zp/about/index.html HTTP/1.1
Accept: */*
Accept-Encoding: gzip, deflate, sdch, br
Accept-Language: en-US,en;q=0.8,pt;q=0.6,pt-PT;q=0.4
Connection: keep-alive
Host: localhost
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
```

```
HTTP/1.1 200 OK
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
Date: Tue, 24 Oct 2017 17:51:01 GMT
Keep-Alive: timeout=5, max=100
Server: Apache/2.4.25 (Fedora)
Transfer-Encoding: chunked

<!DOCTYPE html>
<html>
<head>
   <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
   <title>José Paulo Leal</title>
   ...
```

O pedido `GET` não tem corpo: acaba na linha vazia. A resposta tem a linha inicial, os cabeçalhos, a linha vazia e o HTML.

Examinar HTTP

- **Navegador**: nas ferramentas de programador, separador **Network**, vê-se para cada pedido a linha inicial, os cabeçalhos de pedido e de resposta e os recursos recebidos (se forem texto).
- **telnet**: `telnet servidor porta` liga à porta; escreve-se o pedido à mão e termina-se com dois EOL; a resposta aparece na consola.
- **curl**: transfere um URL. `-X` (`--request`) escolhe o método; `-D ficheiro` (`--dump-header`) guarda os cabeçalhos da resposta, e com `-D -` mostra-os no terminal. Ver `man curl`.

```
$ telnet www.up.pt 80
Trying 193.137.55.13...
Connected to www.up.pt.
Escape character is '^]'.
OPTIONS / HTTP/1.0

HTTP/1.1 200 OK
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache
Allow: OPTIONS,GET,HEAD,POST
Content-Length: 0
Connection: close
Content-Type: text/html

Connection closed by foreign host.
```

Escreveu-se `OPTIONS / HTTP/1.0` e uma linha vazia; o cabeçalho `Allow` da resposta lista os métodos suportados. O mesmo com `curl -D - -X OPTIONS http://www.up.pt/`, sem escrever o pedido à mão.

Recursos

As mensagens **atuam** sobre recursos do servidor: **obter**, **criar**, **modificar** ou **remover**. Os recursos são **referidos** por URLs (na linha de pedido ou em cabeçalhos) ou **transferidos** no corpo. Os dados usam **formatos** próprios do HTTP, e os **metadados** (tamanho, tipo) vão nos cabeçalhos.

Exemplo resolvido

O **URL** `http://localhost:8080/regista?nome=zp&id=123`:

Parte	Valor
protocolo (esquema)	`http`
servidor	`localhost`
porta (omissão: 80)	`8080`
caminho (localiza o recurso, como um ficheiro)	`/regista`
query string (depois de `?`; dados, ex. de formulários)	`nome=zp&id=123`

Na linha inicial de um pedido, o protocolo, o servidor e a porta são **omitidos**, exceto nos pedidos a um *proxy*, que precisa do URL completo. Pode haver ainda um **fragmento**† (`#seccao2`), uma posição **dentro** do recurso, tratada só pelo cliente e **nunca enviada** ao servidor.

Um ***media type*** (ou tipo **MIME**) tem **tipo** e **subtipo** separados por `/` (`text/html`). É o metadado do tipo de recurso, em pedidos (`Accept`) e respostas (`Content-Type`). Exemplos: `application/ javascript`, `json`, `x-www-form-urlencoded`, `msword`, `vnd.ms-excel`; `audio/ mpeg`, `vorbis`; `multipart/form-data`; `text/ css`, `html`, `plain`; `image/ png`, `jpeg`, `gif`.

Codificações de dados. Os dados vêm de **âncoras** (pedidos `GET`) ou **formulários** (`GET` ou `POST`), e vão no **URL** (`GET`) ou no **corpo** (`POST`), num de dois formatos:

Definição

`application/x-www-form-urlencoded` (a **omissão**): tudo numa *string*. Nos valores, espaço → `+`, letras e dígitos ficam iguais, outros carateres → `%` + valor hexadecimal. Os nomes e valores separam-se por `=` e os pares por `&`. Em JavaScript: `encodeURI`/`decodeURI` e `encodeURIComponent`/`decodeURIComponent`.
`multipart/form-data`: para dados complexos, sobretudo **ficheiros**. O corpo tem várias partes separadas por uma *string* aleatória (*boundary*): cada parte começa por `--boundary` e tem os seus metadados (nome do campo e, opcionalmente, tipo e nome de ficheiro); a mensagem acaba em `--boundary--`.

Exemplo resolvido

Campos `de` = "José Paulo Leal", `para` = "Mundo", `texto` = "Olá Mundo". Em *urlencoded* (os espaços passam a `+`; é e á passam aos seus bytes UTF-8 em `%`):

```
de=Jos%C3%A9+Paulo+Leal&para=Mundo&texto=Ol%C3%A1+Mundo
```

Em *multipart*, com mais um campo `foto` (um ficheiro, impraticável em *urlencoded*):

```
Content-Type: multipart/form-data; boundary=AaB03x

--AaB03x
Content-Disposition: form-data; name="de"

José Paulo Leal
--AaB03x
Content-Disposition: form-data; name="para"

Mundo
--AaB03x
Content-Disposition: form-data; name="texto"

Olá Mundo
--AaB03x
Content-Disposition: form-data; name="foto"; filename="foto.jpeg"
Content-Type: image/jpeg

... conteúdo da imagem ...
--AaB03x--
```

Cada parte descreve-se a si própria (`Content-Disposition` e, no ficheiro, `Content-Type`).

Métodos

O método é o **verbo** do pedido. `GET`, `HEAD`, `POST` e `OPTIONS` são os **básicos**: existem em todos os servidores e todos os navegadores os usam. Os outros podem não estar disponíveis por omissão.

Método	O que faz
`GET`	o mais usado: pede o recurso do URL, que vem no corpo da resposta; não deve ter efeitos colaterais
`HEAD`	como `GET`, mas a resposta **não tem corpo**: só metadados
`POST`	envia **dados** no corpo (*urlencoded* ou *multipart*) relacionados com o recurso: critério de pesquisa, dados a acrescentar ou modificar, ou para criar um recurso novo
`OPTIONS`	devolve no cabeçalho `Allow` os métodos suportados para o recurso
`PUT`	guarda o recurso do corpo no servidor, no URL indicado
`DELETE`	remove o recurso do URL
`TRACE`	ecoa o pedido recebido (mostra o que os *proxies* mudaram)
`CONNECT`	estabelece comunicações SSL/TLS através de um *proxy*
`PATCH`	modifica **parcialmente** um recurso

```
POST /regista HTTP/1.1
Accept: */*
Host: localhost
Referer: http://www.dcc.fc.up.pt/~zp/about/index.html
User-Agent: Mozilla/5.0 (X11; Linux x86_64)
Content-Length: 56
Content-Type: application/x-www-form-urlencoded; charset=UTF-8

de=Jos%C3%A9+Paulo+Leal&para=Mundo&texto=Ol%C3%A1+Mundo
```

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
Date: Tue, 24 Oct 2017 17:51:01 GMT
Server: Apache/2.4.25 (Fedora)
Transfer-Encoding: chunked

<!DOCTYPE html>
<html><head><title>Recebido</title></head>
<body><h1>Recebido</h1></body>
</html>
```

Um `OPTIONS / HTTP/1.1` ao mesmo servidor responde `Allow: HEAD,HEAD,GET,HEAD,POST,OPTIONS,TRACE` (com `Content-Type: httpd/unix-directory`).

Definição

Três propriedades dos métodos, que o protocolo **não impõe** (são convenções):

- **Seguro**†: só serve para obter informação, **não deve** alterar o servidor. É uma intenção, não uma garantia (um `GET` pode correr um programa com efeitos; muitos `GET` podem negar o serviço).
- **Idempotente**†: repetir o **mesmo** pedido tem o **mesmo efeito** que fazê-lo uma vez. `PUT` e `DELETE` alteram o servidor mas são idempotentes (repetir o `DELETE` deixa o recurso removido); o `POST` não é (repetir uma compra cria duas encomendas), e por isso o navegador avisa antes de reenviar um formulário.
- **Cachável**†: a resposta pode ser guardada (no cliente ou num *proxy*) e reutilizada para pedidos iguais, segundo os cabeçalhos de cache.

Método	Seguro	Idempotente	Cachável
`GET`, `HEAD`	sim	sim	sim
`POST`	não	não	sob condições
`PUT`, `DELETE`	não	sim	não
`OPTIONS`, `TRACE`	sim	sim	não
`CONNECT`, `PATCH`	não	não	não

Códigos de estado

Três dígitos, em 5 classes pelas centenas: **1XX** informativos, **2XX** sucesso, **3XX** redireção, **4XX** erros do **cliente**, **5XX** erros do **servidor**. A mensagem associada é **opcional** e pode mudar; só o **número** é normativo. A lista não é exaustiva.

Código	Quando
100 Continue	os cabeçalhos estão bem, o cliente pode enviar o corpo
101 Switching Protocols	o servidor aceita mudar de protocolo (ex. *WebSockets*)
102 Processing	recebido, ainda em processamento
200 OK	sucesso; a resposta tem o resultado
201 Created	sucesso, criou um recurso novo
202 Accepted	aceite, mas o processamento ainda não acabou
204 No Content	sucesso, sem corpo
300 Multiple Choices	há várias representações; o cliente escolhe
301 Moved Permanently	movido **de vez** para o URL do `Location`
302 Found	temporariamente noutro URL
303 See Other	a resposta está noutro URL, a obter com `GET` (comum depois de um `POST`)
307 Temporary Redirect	temporário, repetindo o pedido com o **mesmo método**
400 Bad Request	erro de sintaxe no pedido
401 Unauthorized	falta autenticação (ou é inválida)
402 Payment Required	reservado para pagamentos
403 Forbidden	percebido, mas recusado
404 Not Found	o recurso não existe
500 Internal Server Error	erro interno do servidor (ex. exceção num programa)
501 Not Implemented	método não reconhecido ou não suportado
502 Bad Gateway	um *gateway/proxy* recebeu uma resposta inválida
503 Service Unavailable	temporariamente indisponível (sobrecarga, manutenção)
504 Gateway Timeout	um *gateway/proxy* não recebeu resposta a tempo

```
HTTP/1.1 200 OK
Date: Fri, 21 Apr 2017 15:23:55 GMT
Server: Apache/2.4.25 (Fedora)
Last-Modified: Fri, 21 Apr 2017 15:23:55 GMT
ETag: "116f-54daed884be86"
Accept-Ranges: bytes
Content-Length: 4463
Connection: close
Content-Type: text/html; charset=UTF-8

HTTP/1.1 301 Moved Permanently
Location: http://khato.dcc.fc.up.pt/~zp/
Content-Type: text/html; charset=iso-8859-1

HTTP/1.1 404 Not Found
HTTP/1.1 500 Internal Server Error
```

Cabeçalhos

Alguns cabeçalhos são só de pedidos, outros só de respostas, outros de ambos; alguns formam "diálogos" (cache, autenticação, CORS).

> **Definição**
>
> - **Identificação**: `User-Agent` (pedido: navegador, sistema operativo, sistema de janelas), `Server` (resposta), `Referer` (pedido: página de origem), `Date` (resposta).
> - **Metadados do corpo** (pedidos e respostas): `Content-Type` (o *media type*; nos pedidos, *urlencoded* ou *multipart*), `Content-Length` (bytes), `Transfer-Encoding: chunked` (o corpo vai aos bocados, cada um com o seu tamanho, quando não se sabe o total de antemão).
> - **Redireção**: corrige URLs relativos, indica novas localizações ou alternativas (traduções). Usa um código 3XX e o novo URL no `Location`.

Um pedido a uma diretoria sem `/` final é corrigido por redireção:

```
GET /~zp HTTP/1.1

HTTP/1.1 301 Moved Permanently
Location: http://khato.dcc.fc.up.pt/~zp/

GET /~zp/ HTTP/1.1
```

Cookies

O HTTP não guarda estado; os ***cookies*** simulam-no, para seguir utilizadores, personalizar conteúdo ou criar sessões.

- `Set-Cookie: nome=valor` vai na **resposta** e aparece **primeiro**.
- `Cookie: nome=valor` vai nos **pedidos** seguintes; vários *cookies* separam-se por `;`.
- `Expires` dá a validade; sem ele, o *cookie* morre quando se fecha o navegador. `Path` restringe-o a uma sub-árvore do servidor.

Exemplo resolvido

```
GET /index.html HTTP/1.1

200 OK
Set-cookie: id=123; Expires=Wed, 22 Sep 2027 10:00:00 GMT
Set-cookie: sessao=12345
```

1. O cliente guarda os dois e envia-os daí em diante: `GET /loja/index.html` com `Cookie: id=123; sessao=12345`.
2. A resposta traz `Set-cookie: prefere=A; Path=/loja/`.
3. `GET /loja/regista` leva `Cookie: id=123; sessao=12345; prefere=A`.
4. `GET /index.html` (fora de `/loja/`) leva só `Cookie: id=123; sessao=12345`.
5. Fechado e reaberto o navegador, `sessao` (sem `Expires`) desapareceu: `GET /index.html` leva `Cookie: id=123`, e a resposta cria uma sessão nova com `Set-cookie: sessao=4321`.

Cache

- `Cache-control`: `max-age` (validade em segundos), `no-store` (não guardar; dados sensíveis), `no-cache` (guardar, mas confirmar antes de reutilizar), `private` (reutilizar exige autenticação), `public` (não exige).
- Por *hash*: a resposta traz `ETag` (identifica a versão); o pedido seguinte manda `If-None-Match` com esse valor.
- Por data: `If-Modified-Since`.

Exemplo resolvido

1. `GET /index.html` recebe `200 OK` com `Cache-control: max-age=120` e `ETag: xpto`.
2. Passados os 120 s, o cliente revalida: `GET /index.html` com `If-None-Match: xpto`.
3. O servidor responde `304 Not Modified`, sem corpo: o recurso não mudou e o cliente usa a cópia que tem.

CORS

Os pedidos de uma página vão normalmente ao servidor de onde ela veio. Os pedidos **para outra origem** são muito usados em ataques, e os navegadores **limitam-nos**. Para servidores que servem legitimamente várias origens há o **CORS** (*Cross-Origin Resource Sharing*): a resposta traz `Access-Control-Allow-Origin` com $*$ (qualquer origem) ou a lista de origens autorizadas.

Antes de um pedido entre origens "não simples" (ex. `DELETE`, ou cabeçalhos próprios), o navegador faz um ***preflight***, um `OPTIONS` com:

- `Access-Control-Request-Method`: o método do pedido real;
- `Access-Control-Request-Headers`: os cabeçalhos do pedido real;
- `Origin`: esquema, servidor e porta de origem.

A resposta traz `Access-Control-Allow-Origin` e `Access-Control-Allow-Methods`. Só se for aceite é que o pedido real segue.

```
OPTIONS /recurso HTTP/1.1
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: origin, x-requested-with
Origin: http://my.server.org

HTTP/1.1 200 OK
Access-Control-Allow-Origin: http://my.server.org
Access-Control-Allow-Methods: POST, GET, OPTIONS, DELETE
```

A origem coincide e o `DELETE` está nos métodos aceites: o navegador envia então o `DELETE /recurso` real (com `Origin`), e a resposta traz de novo `Access-Control-Allow-Origin`.

Autenticação

1. O cliente pede um recurso protegido (`GET /segredos/index.html`).
2. O servidor responde `401 Unauthorized` com `WWW-Authenticate: Basic realm="Pequenos segredos"` (o **tipo**, normalmente `Basic`, e o **reino**, que descreve o motivo).
3. O navegador mostra uma caixa de autenticação e repete o pedido com `Authorization: Basic enA6c291IGV1Cg==` (mesmo tipo; no `Basic`, as credenciais em **base64**).
4. O servidor responde `200 OK`.

O base64 **não é cifra**†: decodifica-se com `base64 -d`. O `Basic` só deve ser usado sobre HTTPS.

Segurança

O HTTP é **inseguro**: as mensagens de texto são **visíveis** a quem as intercetar (incluindo a autenticação), o que permite ataques *man-in-the-middle*, e as linhas iniciais (com os dados de um `GET`) ficam em *logs* intermédios.

O **HTTPS** (`https://`) é o HTTP sobre uma camada de transporte segura: um **canal cifrado** entre cliente e servidor, que pode passar por *proxies* (que já não o leem). A cifra impede **ler** e **alterar** as mensagens pelo caminho. Protege transações financeiras, palavras-passe, a identidade e a privacidade, e hoje é a norma.

Definição

O HTTPS assenta em **certificados digitais**:

- **de servidor**: específicos do **domínio** e **validáveis**; provam a **autenticidade** do servidor. São validados por **autoridades certificadoras**, em **cadeia** (uma autoridade certifica outra); as de topo vêm **pré-configuradas** nos navegadores;
- **de cliente**: normalmente **gerados** pelo navegador, ou **pessoais**, carregados à mão; autenticam melhor do que o `Basic`.

No *handshake* TLS†, o servidor apresenta o certificado (com a sua chave pública, assinado por uma autoridade); o cliente valida a assinatura; e os dois negoceiam com chave pública uma **chave simétrica**, que cifra o resto do tráfego (é mais rápida). O certificado garante a **identidade** do servidor e a **confidencialidade** do canal.

8 Tema 8
Comunicação assíncrona cliente-servidor

A forma clássica: formulários e páginas inteiras

No ciclo pedido-resposta básico, a resposta **substitui a página inteira**. Foi pensado para navegar em hipertexto, não para aplicações interativas.

Definição

Âncoras. `<a href="http://some.site.com">Um sítio qualquer</a>` gera um `GET` para o URL do `href`, e a página HTML da resposta substitui a atual.

Dados no URL. Um `GET` leva dados na *query string*, em *urlencoded*: `<a href="http://some.site.com/execute?command=remove&id=123">`. Só serve para poucos pares, não tem *widgets* para o utilizador escrever, e o `GET` deve ser **seguro**, **idempotente** e **cachável**: não serve para alterar dados no servidor.

Formulários. Dão mais controlo:

- `method`: `POST` sempre que se altera o servidor; `GET` é a omissão;
- os pares **nome-valor** vêm dos *widgets* (`name` e `value` ou o que o utilizador escreveu); `type=hidden` inclui valores fixos invisíveis;
- `enctype` escolhe a codificação: `application/x-www-form-urlencoded` (omissão), `multipart/form-data` (obrigatória para enviar ficheiros com `type=file`; só com `POST`) ou `text/plain` (sem codificação, só os espaços passam a +).

Frames. Antes do DOM, os *frames* permitiam atualizar uma zona da página **independentemente**. `<iframe name=frame>` cria uma sub-janela de navegação (com `name` e opcionalmente `src`); âncoras e formulários com `target=frame` mandam a resposta para lá em vez de substituir a página.

```
<form action=http://some.site.com/execute method=post>
    <input type=hidden name=command value=remove>
    <input type=hidden name=id value=123>
    <input type=submit value=Remover>
</form>
<form action=http://some.site.com/execute method=post enctype=multipart/form-data>
    <input type=hidden name=command value=insert>
    <input type=text name=name>
    <input type=file name=photo>
    <input type=submit value=Inserir>
</form>
<a href=p1.html target=frame>Uma página</a>
<a href=p2.html target=frame>Outra página</a>
<iframe name=frame></iframe>
```

Evitar atualizar a formatação inteira. Substituir a página toda dá mudanças **abruptas**. Com o DOM fazem-se **mudanças localizadas**, só com os dados recebidos. Antes do XHR e do `fetch` havia truques: um **`iframe` invisível** com `script`, e o **JSONP**: criar pelo DOM um `<script>` cujo `src` é um URL do servidor, que devolve `application/javascript` com uma chamada a uma função do cliente, passando-lhe um objeto JSON. Contorna a *same-origin policy* (os *scripts* carregam-se de qualquer origem), mas obriga a confiar totalmente no servidor, que passa a executar código no cliente.

Serialização de dados

> **Definição**
> O corpo HTTP é uma **sequência de bytes** (com um *media type*), mas os dados de um programa são uma **rede de referências** espalhadas pela memória, com representações binárias que dependem da plataforma. Para os passar entre processos (de linguagens e máquinas diferentes) é preciso **serializá-los** numa forma linear, normalmente **texto**: independente da plataforma e legível.

Formato	Exemplo	Vantagem	Desvantagem
texto simples	`Fulano 99`	muito simples	não estruturado (dados separados por um carácter)
XML	`<data> <name>Fulano</name> <age>99</age> </data>`	estruturado; define linguagens e valida	mais complexo de ler e gerar
JSON	`{"name": "Fulano", "age": 99}`	estruturado e compacto	mais difícil de normalizar e validar

> **Definição**
> O **JSON** (*JavaScript Object Notation*) usa a sintaxe dos objetos de JavaScript: `{ "name": "Fulano", "age": 99, "active": true, "lucky numbers": [7, 13] }`.
> Tipos: **números** (inteiros ou vírgula flutuante), ***strings*** (Unicode, **sempre entre aspas duplas**, com escapes), **booleanos**, **listas** `[ ]`, **objetos** `{ }` e `null`. As chaves são *strings* com aspas, seguidas de `:`; elementos e pares separam-se por `,`. Não é um subconjunto exato de JS†: chaves sem aspas e vírgulas finais não são JSON válido.

```
let objt = { a: 1, b: [2, 3] };
let json = JSON.stringify(objt);   // '{"a":1,"b":[2,3]}'   (string)
let copy = JSON.parse(json);       // { a: 1, b: [2, 3] }   (objeto novo)
```

`JSON.stringify` passa um valor a texto JSON; `JSON.parse` faz o inverso. Antigamente usava-se `eval("(" + jsonString + ")")`, que é **inseguro**: se o texto for código, é executado. `JSON.parse(JSON.stringify(objt))` dá ainda uma **cópia profunda**† (sem referências partilhadas).

XMLHttpRequest

Cenário: um contador **distribuído**, com botões `Incr` e `Reset` e um visor, mas com o valor num **servidor central**, partilhado por vários clientes. O JavaScript tem de comunicar com o servidor **sem recarregar a página**: é para isso o `XMLHttpRequest` (XHR). Há três operações: `POST incr`, `POST reset` e obter o valor.

Definição

- `new XMLHttpRequest()`: testa-se antes se existe (`if (!XMLHttpRequest) ...`). Cada instância serve para **uma só** comunicação e dá o contexto (métodos, propriedades).
- `xhr.open(método, url, assíncrono)`: **configura** o pedido (método, URL absoluto, e `true` = assíncrono, a omissão). Não envia.
- `xhr.send([dados])`: **envia**, com dados opcionais no corpo. Num pedido **síncrono** só retorna quando chega a resposta (bloqueia); num **assíncrono** retorna **logo**.
- `xhr.readyState`: `0` por enviar, `1` enviado, `2` cabeçalhos recebidos, `3` corpo a chegar, `4` **completo**. `xhr.onreadystatechange` é chamado a cada mudança; reage-se quando é 4.
- Depois da resposta (antes são indefinidas): `xhr.status` (código), `xhr.responseText` (corpo em texto), `xhr.responseXML` (corpo como documento DOM).
- `xhr.setRequestHeader(nome, valor)` (antes do `send`) e `xhr.getResponseHeader(nome)` (depois da resposta).
- **CORS**: a *same-origin policy* só deixa pedir a outro domínio se o servidor o permitir, e mesmo assim ***sem cookies***; `xhr.withCredentials = true` envia-os. Sem precisar de *cookies*, deixa-se `false`.

```
// Síncrono
increment() { this.do('incr'); }
reset()     { this.do('reset'); }
do(command) {
    if (!XMLHttpRequest) { console.log('XHR não é suportado'); return; }
    const xhr = new XMLHttpRequest();
    xhr.open('POST', 'http://' + host + ':' + port + '/' + command, false);
    xhr.send();
    if (xhr.status == 200)
        this.display.innerText = xhr.responseText;
}
```

Quando o `send()` síncrono retorna, a resposta **já chegou**: lê-se como uma chamada de função. Mas enquanto espera, o programa **não processa eventos** (a página "pendura"), o que degrada a experiência do utilizador. Por isso o XHR síncrono está ***deprecated*** na *thread* principal; só se usa em *web workers* (outras *threads*) ou no `onbeforeunload` (esperar que um registo termine). A alternativa é o pedido **assíncrono**, com um *callback*:

```
// Assíncrono
do(command) {
    const xhr = new XMLHttpRequest();
    const display = this.display;
    xhr.open('POST', 'http://' + host + ':' + port + '/' + command, true);
    xhr.onreadystatechange = function() {
        if (xhr.readyState < 4) return;
        if (xhr.status == 200)
            display.innerText = xhr.responseText;
    };
    xhr.send();
}
```

Exemplo resolvido

Enviar e receber JSON:

```
do(command, value) {
    const xhr = new XMLHttpRequest();
    const display = this.display;
    xhr.open('POST', 'http://' + host + ':' + port + '/process', true);
    xhr.onreadystatechange = function() {
        if (xhr.readyState == 4 && xhr.status == 200) {
            const data = JSON.parse(xhr.responseText);
            display.innerText = data.value;
        }
    };
    xhr.send(JSON.stringify({ command: command, value: value }));
}
```

O corpo do pedido é o objeto serializado com `JSON.stringify` (em ES6, `{command, value}`); a resposta volta a objeto com `JSON.parse`. O XHR só transporta texto: o formato JSON é um acordo entre cliente e servidor.

Encadear pedidos

Muitas vezes um segundo pedido depende do resultado do primeiro. Com pedidos **síncronos**, a ordem do código é a da execução, e uma função pode **devolver** o resultado:

```
function callRemote(value) {
    let res = null;
    const xhr = new XMLHttpRequest();
    xhr.open('POST', url, false);
    xhr.send(value);
    if (xhr.status == 200)
        res = xhr.responseText;
    return res;
}
y = callRemote(x);
z = continuar(y);
```

Com pedidos **assíncronos**, a função retorna antes da resposta e não pode devolver o valor: quem continua é o *callback* (`continuar(xhr.responseText)` dentro do `onreadystatechange`). Ao encadear dois pedidos, a ordem do código deixa de ser a das invocações:

```
const xhr1 = new XMLHttpRequest();
xhr1.open('POST', url1, true);
xhr1.onreadystatechange = function() {
    // ...
    const xhr2 = new XMLHttpRequest();
    xhr2.open('POST', url2, true);
    xhr2.onreadystatechange = function() {
        // ...
        continuar(xhr2.responseText);
    };
    xhr2.send(xhr1.responseText);
};
xhr1.send(JSON.stringify(x));
```

O `xhr2` só é criado no *callback* do `xhr1`, e `continuar` só corre no do `xhr2`. Cada pedido encadeado é mais um nível de funções aninhadas: com 4 ou 5, e erros a tratar em cada um, fica ilegível (*callback hell*†). As *promises* resolvem isto.

`fetch`, `Request` e `Response`

O **`fetch(url, request)`** é a alternativa moderna ao XHR e devolve uma **promessa de um `Response`**. O segundo argumento, opcional, é o **`Request`**: `method`, `headers` (objeto com os cabeçalhos) e `body`.

```
fetch('http://some.site.org/authenticate', {
    method: 'POST',
    headers: { 'Content-type': 'application/x-www-form-urlencoded; charset=UTF-8' },
    body: 'nick=cave&pass=word'
})
  .then(response => console.log(response))
  .catch(console.log);
```

> **Definição**
>
> O **`Response`** tem as propriedades `ok` (estado em [200, 300[), `status`, `statusText`, `headers` e `type` (`basic` ou `cors`), e os métodos `json()` e `text()`, que devolvem **promessas** do corpo (ler o corpo também é assíncrono).
> Uma **`Promise`** representa o resultado futuro de uma operação assíncrona. Constrói-se com uma função que recebe duas **continuações**, `resolve` (sucesso) e `reject` (falha), e chama uma delas no fim. Métodos: `.then(f)` regista a continuação de sucesso e `.catch(f)` a de falha. `Promise.resolve(v)` e `Promise.reject(e)` criam promessas já resolvidas / rejeitadas.
> O `.then()` devolve **sempre uma nova `Promise`**, por isso **encadeia-se**; se a função passada devolve uma promessa, o `.then` seguinte espera por ela. Uma rejeição em qualquer ponto salta para o `.catch` seguinte.

```
fetch('http://some.site.org/resource')
    .then(function(response) {
        if (response.ok) response.text().then(console.log);
        else console.log('erro: ' + response.statusText);
    })
    .catch(console.log);

function vamosSairLogo(quando) {
    return new Promise(function(resolve, reject) {
        setTimeout(function() {
            if (aindaApetece()) resolve(); else reject();
        }, quando.getTime() - (new Date()).getTime());
    });
}
vamosSairLogo(new Date("..."))
    .then(() => irASitioAltamente())
    .catch(() => console.log('És sempre o mesmo :-('));
```

Exemplo resolvido

```
function status(response) {
    if (response.ok) return Promise.resolve(response);
    else return Promise.reject(new Error('Invalid status'));
}
console.log('A');
fetch(url)
    .then(status)
    .then(response => response.json())
    .then(json => console.log('D', json))
    .catch(console.log);
console.log('B');
```

1. **Síncrono**: escreve `A`. O `fetch` **inicia** o pedido (fora da *thread* de JS) e devolve logo uma promessa pendente. Os `.then` e o `.catch` só **registam** continuações. Escreve `B`.
2. Chega a resposta: a promessa do `fetch` resolve com o `Response`, e `status(response)` entra na **fila de *microtasks***, que corre quando a *thread* fica livre.
3. `status` devolve `Promise.resolve(response)`: a segunda promessa resolve com o `Response` (mais uma *microtask*).
4. Corre `response.json()`, que lê o corpo (assíncrono) e devolve outra promessa pendente.
5. Lido o corpo, a terceira promessa resolve com o objeto, e escreve `D`.
6. Se o `fetch` falhar ou `status` rejeitar (`!response.ok`), salta-se para o `.catch`.

Ordem: `A`, `B`, `D`. O código dos `.then` corre **sempre depois** do código síncrono a seguir ao `fetch`.

`async` e `await`

> **Definição**
> Uma função **`async`** devolve sempre uma `Promise`: `return v` passa a `Promise.resolve(v)` e `throw e` a `Promise.reject(e)`. É açúcar sintático sobre as promessas.
> **`await promessa`**, só dentro de uma função `async`, **espera** pela promessa (sem bloquear a *thread*) e dá o valor do `resolve`, ou **lança** o do `reject` (apanha-se com `try/catch`).

```
async function status(response) {
    if (response.ok) return response;
    else throw new Error('Invalid status');
}
async function message(response) {
    if (response.ok) {
        let message = await response.json();
        if ('error' in message) throw new Error(message.error);
        else return message;
    } else
        throw new Error('Invalid status');
}
```

Exemplo resolvido

Dois pedidos encadeados, com `.then` e com `await`:

```
function combina(x) {
    return fetch(url1, { method: 'POST', body: JSON.stringify(x) })
        .then(response => response.json())
        .then(y => fetch(url2, { method: 'POST', body: JSON.stringify(y) }))
        .then(response => response.json());
}
async function combina2(x) {
    let respostaY = await fetch(url1, { method: 'POST', body: JSON.stringify(x) });
    let y = await respostaY.json();
    let respostaZ = await fetch(url2, { method: 'POST', body: JSON.stringify(y) });
    let z = await respostaZ.json();
    return z;
}
combina(x).then(continuar);
combina2(x).then(continuar);
```

Cada `await` faz o papel de um `.then`: suspende `combina2` até a promessa resolver. O `return z` passa a `Promise.resolve(z)`, por isso usa-se com `.then(continuar)` como a versão com promessas. O código lê-se de cima para baixo, sem funções aninhadas.

Comparação†	*Callbacks* (XHR)	*Promises*	`async/await`
Encadear	aninhamento crescente	uma cadeia de `.then`	linear, como código síncrono
Erros	verificar em cada nível	um `.catch` para a cadeia	`try/catch` à volta dos `await`
Valor devolvido	nenhum: usa-se uma continuação	uma `Promise`	uma `Promise` (automática)

Tempo real: SSE e WebSockets

No pedido-resposta é sempre o **cliente** que começa: o servidor não o pode avisar de novidades (uma jogada de outro utilizador, uma mensagem). Antes do HTML5:

- ***polling***: pedidos regulares a perguntar se há novidades; quase todos em vão, desperdiçando tempo de ligação e largura de banda;
- ***long polling***: usa a mesma ligação HTTP para repetir pedidos;
- **Comet**: o nome para estas técnicas.

O HTML5 traz duas APIs com o mesmo padrão: um **construtor**, **propriedades** com o estado da ligação e **eventos** para receber (e, nos WebSockets, enviar) dados.

Definição

Server-Sent Events. `new EventSource(url)` liga com um `GET`. Propriedades (só de leitura): `withCredentials` (enviar credenciais em CORS), `url`, `readyState` (`0` CONNECTING, `1` OPEN, `2` CLOSED). Eventos: `onopen`, `onmessage` (uma vez por mensagem) e `onerror`. Os dados vêm como **texto** em `event.data` (`JSON.parse` se preciso). A ligação mantém-se **indefinidamente** e é **restabelecida automaticamente** se cair; como o número de ligações ao mesmo servidor é limitado, fecha-se no fim com `close()`.
Sobre HTTP: o cliente faz um `GET` com `Accept: text/event-stream`; o servidor responde `200 OK` com `Content-Type: text/event-stream`, `Cache-Control: no-cache`, `Connection: keep-alive` e não fecha. O corpo são **partes** separadas por uma **linha vazia**, com linhas `data:` (conteúdo, pode ocupar várias linhas) ou `id:` (identificador, para retomar). Cada parte dispara um `onmessage`.

```
const eventSource = new EventSource(url);
eventSource.onmessage = function(event) {
    const data = JSON.parse(event.data);
};
eventSource.close();
```

```
data: 0

data: 1

data: 2
```

Definição

WebSockets. `new WebSocket(url)`, com URL `ws://` ou `wss://` (sobre TLS, como o `https`); o segundo argumento, opcional, lista **sub-protocolos** a negociar. Propriedades: `protocol` (o escolhido pelo servidor) e `readyState` (como no SSE). Eventos `onopen`, `onmessage`, `onerror`, com os dados em `event.data`. **Envia-se** a qualquer momento com `send(dados)` (sempre *strings*, normalmente `JSON.stringify`). As mensagens trocadas definem um **protocolo aplicacional** próprio. Se a ligação cair, **não** é reaberta; fecha-se com `close()`.
Sobre HTTP: o cliente faz um `GET` com `Upgrade: websocket`, `Connection: Upgrade`, `Sec-WebSocket-Version` e uma chave aleatória `Sec-WebSocket-Key`. Se aceitar, o servidor responde **`101 Switching Protocols`** com `Upgrade`, `Connection` e `Sec-WebSocket-Accept` (calculado da chave). Daí em diante a ligação troca **mensagens WebSocket** nos dois sentidos (as ferramentas do navegador mostram-nas).

```
const url = "ws://" + host + ":" + port + "/";
const webSocket = new WebSocket(url);
webSocket.onmessage = function(event) {
    const data = JSON.parse(event.data);
};
webSocket.send(JSON.stringify(data));
webSocket.close();
```

	Server-Sent Events	WebSockets
Sentido	servidor → cliente (o `fetch` complementa)	bidirecional
Protocolo	integrado no HTTP	negoceia o seu protocolo
Ligação	reconexão automática	sem controlo de conexão
Duplex	*half-duplex*	*full-duplex*
Dados	só texto UTF-8	texto UTF-8 e binários
Servidor	mais simples	mais complexo
Exemplo	*ticker*	*chat*

Exemplos completos

Exemplo resolvido **Contador distribuído (fetch + SSE)**

```
class Counter {
    constructor(parentId) {
        const parent = document.getElementById(parentId);
        const counter = document.createElement("div");
        const incr = document.createElement("div");
        const reset = document.createElement("div");
        this.display = document.createElement("div");
        parent.appendChild(counter);
        counter.appendChild(incr);
        counter.appendChild(reset);
        counter.appendChild(this.display);
        counter.className = "counter";
        incr.className = "button";
        reset.className = "button";
        this.display.className = "display";
        incr.innerText = " Incr ";
        reset.innerText = " Reset ";
        incr.onclick = this.increment.bind(this);
        reset.onclick = this.reset.bind(this);
        const source = new EventSource("http://" + host + ":" + port + "/update");
        source.onmessage = event => this.display.innerText = event.data;
    }
    increment() { this.do("incr"); }
    reset()     { this.do("reset"); }
    do(command) {
        fetch("http://" + host + ":" + port + "/" + command, { method: 'POST' })
            .catch(console.log);
    }
}
```

Exemplo resolvido **Contador distribuído (continuação)**

A contagem está no **servidor**: os **comandos** vão por `fetch` (sem ler a resposta) e as **atualizações** chegam por SSE, também quando outro cliente mudou o valor. A seta em `source.onmessage` mantém `this` = o `Counter`. Clientes A e B ligados por SSE, contador a 5:

1. A clica em "Incr": `increment()` chama `do('incr')`.
2. O `fetch` **inicia** um `POST /incr` e devolve logo uma promessa; A ignora a resposta (o `.catch` só atua se falhar).
3. O servidor passa o contador a 6 e responde `200 OK`.
4. O servidor envia `data: 6` por SSE a **todos** os clientes em `/update`.
5. Em A, `onmessage` põe `6` no visor.
6. Em B, que não fez nada, o **seu** `onmessage` também põe `6`.

O valor nunca vem da resposta do `fetch`: só pelo SSE, como para qualquer cliente.

Exemplo resolvido **Jogo do galo multiutilizador (WebSockets)**

```
class TicTacToe {
    constructor(id) {
        const parent = document.getElementById(id);
        const board = document.createElement("div");
        board.className = "board";
        parent.appendChild(board);
        this.cells = new Array(9);
        for (let i = 0; i < 9; i++) {
            const cell = document.createElement("div");
            cell.className = "cell" + (((i + 1) % 3) == 0 ? " line" : "");
            board.appendChild(cell);
            cell.innerHTML = " ";
            cell.onclick = () =>
                webSocket.send(JSON.stringify({ command: "play", pos: i }));
            this.cells[i] = cell;
        }
        const reset = document.createElement("div");
        reset.className = 'button';
        parent.appendChild(reset);
        reset.onclick = function() {
            webSocket.send(JSON.stringify({ command: "reset" }));
        };
    }
}
```

Exemplo resolvido **Jogo do galo multiutilizador (continuação)**

```
const url = "ws://" + host + ":" + port + "/";
const webSocket = new WebSocket(url);
webSocket.onmessage = function(event) {
    const message = JSON.parse(event.data);
    if ('board' in message)
        for (const b in message.board)
            ticTacToe.cells[b].innerHTML = message.board[b];
    if ('pos' in message && 'piece' in message)
        ticTacToe.cells[message.pos].innerHTML = message.piece;
    if ('current' in message)
        showCurrent.innerHTML = "Playing " + message.current;
};
const ticTacToe = new TicTacToe("base");
```

As mensagens JSON formam um **protocolo** (típico de arquiteturas orientadas a serviços). Pedidos: `{"command": "play", "pos": 2}`, `{"command": "reset"}`. Respostas, com o estado completo ou só a alteração: `{"board": ["X", " ", "O"], "current": "O"}`, `{"pos": 2, "piece": "X"}`.
Clientes A ("X", a jogar) e B:

1. A clica na posição 2: envia `{"command":"play","pos":2}`. O tabuleiro de A **não** muda ainda.
2. O servidor valida (posição livre, vez de "X"), atualiza o estado e passa a vez a "O".
3. Difunde a todos `{"pos": 2, "piece": "X"}` e `{"current": "O"}`.
4. Em A, `onmessage` põe `"X"` na célula 2 e mostra "Playing O".
5. Em B, o mesmo, sem ter clicado.
6. Jogadas inválidas de B são o **servidor** que as recusa.

Tal como no contador, o clique nunca muda a interface: envia um comando, e é a mensagem de volta que atualiza o DOM de todos.

9 Tema 9
Outras APIs do navegador

O HTML5 trouxe APIs de JavaScript independentes do HTTP e da estrutura da página: a **Web Storage** (persistência no cliente) e o **Canvas** (desenho 2D imperativo).

Web Storage

Definição

A **Web Storage** guarda dados no cliente, sem as limitações dos *cookies*:

- **5 a 10 MB** por origem (os *cookies* têm poucos KB);
- **sem transferência automática** para o servidor (os *cookies* vão em todos os pedidos);
- acessível em JavaScript por um objeto simples;
- um dicionário de **chaves** (*strings*) para **valores** simples ou objetos serializados.

Há duas variantes com a **mesma API**:

	`localStorage`	`sessionStorage`
Duração	indefinida: sobrevive a fechar o navegador	a sessão do **separador**
Âmbito	partilhado pelos separadores da mesma origem	próprio de cada separador† (um separador novo começa vazio)
Recarregar a página	mantém	mantém
Fechar o separador	mantém	apaga

```
if (typeof(Storage) === 'undefined') {
  // Web Storage indisponível: usar alternativa (cookies, ou não guardar)
}
localStorage.setItem('name', 'fulano de tal');
localStorage.setItem('age', 99);
localStorage.getItem('name');   // 'fulano de tal'
localStorage.getItem('age');    // '99' (string!)
```

Testa-se primeiro se existe o tipo `Storage`. `setItem(chave, valor)` converte os dois argumentos para *string* (`toString()`); `getItem(chave)` devolve sempre uma *string*, ou `null` se a chave não existe. Com `sessionStorage` é igual.

Um objeto guardado diretamente ficava `"[object Object]"`: **serializa-se em JSON**.

```
const data = [{ name: 'fulano', age: 99 }];
localStorage.setItem('data', JSON.stringify(data));
// ... mais tarde, mesmo depois de recarregar ...
const data2 = JSON.parse(localStorage.getItem('data'));
```

Funções, `undefined`, referências circulares, `Date` e `Map` não sobrevivem ao JSON†: uma `Date` volta como *string* (refaz-se com `new Date(string)`).

Canvas: fundamentos

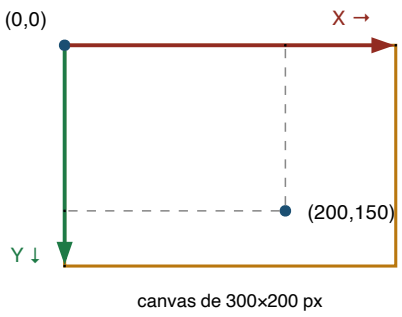
Definição

O `<canvas>` é uma **tela** onde se pinta de forma **imperativa** (o SVG é declarativo), chamando funções **JavaScript** sobre um **contexto gráfico**, cujas propriedades têm sintaxe parecida com o CSS (cores, letra).

- O elemento é **vazio**: o conteúdo entre `<canvas>` e `</canvas>` só aparece em navegadores sem *canvas*.
- As dimensões dão-se com os **atributos** `width` e `height`, **nunca** em CSS: em CSS a resolução interna fica 300×150 e o desenho é esticado e desfocado†. Mudar as dimensões (mesmo para o mesmo valor) **limpa** o *canvas* e repõe o contexto.
- O `id` serve para o obter com `getElementById`.
- `canvas.getContext('2d')` dá o contexto 2D (o usado aqui; `'webgl'` é 3D). O contexto tem **propriedades** (estado: `fillStyle`, `font`, `lineWidth`), que valem para tudo o que se desenha até mudarem, e **métodos** (primitivas: `fillText`, `fillRect`, `arc`).
- **Coordenadas**† em píxeis, com a origem no **canto superior esquerdo**: X cresce para a direita e **Y para baixo**. Por isso um ângulo positivo roda no sentido dos ponteiros do relógio.

```html
<canvas id="tela" width="300" height="200"></canvas>
```

```js
const tela = document.getElementById('tela');
const gc = tela.getContext('2d');
gc.fillStyle = '#228';
gc.font = '48px serif';
gc.fillText('Olá mundo!', 100, 100);
```



Retângulos: `strokeRect(x, y, width, height)` (contorno), `fillRect(...)` (preenchido) e `clearRect(...)` (limpa, fica transparente), com `(x, y)` o canto superior esquerdo. As propriedades `canvas.width` e `canvas.height` dão as dimensões.

```js
gc.strokeRect(40, 40, 100, 40);    // só o contorno
gc.fillRect(40, 100, 100, 40);     // preenchido
gc.fillRect(160, 40, 100, 100);    // um quadrado preenchido...
gc.clearRect(180, 60, 60, 60);     // ...com um buraco de 60x60 por cima
const xc = canvas.width / 2, yc = canvas.height / 2;
gc.fillRect(xc - 10, yc - 10, 20, 20);   // quadrado 20x20 centrado
```

Cores de `fillStyle` (preenchimento) e `strokeStyle` (traço): como no CSS, por nome (`'orange'`), RGB hexadecimal (`'#228'`), `rgb/rgba` ou `hsl/hsla`.

```
for (let hue = 0; hue < 240; hue++) {
  gc.fillStyle = 'hsl(' + hue + ',100%,50%)';
  gc.fillRect(30 + hue, 30, 1, 140);
}
gc.strokeStyle = '#228';
gc.strokeRect(30, 30, 240, 140);
```

São 240 retângulos de 1 px, cada um com outra matiz: um arco-íris.

Canvas: texto

Definição

- `fillText(texto, x, y)` **preenche** as letras; `strokeText(texto, x, y)` **traça** o contorno.
- `font` tem a sintaxe do `font` do CSS, por esta ordem: `font-style`, `font-variant`, `font-weight`, `font-size` e `font-family` (estes dois obrigatórios). Ex.: `gc.font = '32pt Arial'`.
- `textAlign` diz que ponto do texto fica no `x`: `"left"` (omissão, início), `"center"`, `"right"` (fim), `"start"`/`"end"` (como `left`/`right` em texto da esquerda para a direita, trocados em árabe).
- `textBaseline`† faz o mesmo na vertical, para o `y`: `"top"`, `"middle"`, `"alphabetic"` (omissão: a linha de base; "g" e "j" descem abaixo dela).
- `measureText(texto).width` dá a largura que o texto ocuparia com a letra atual (não desenha nada).

```
gc.font = '32pt Arial';
gc.fillStyle = 'orange';
gc.fillText('the quick', 30, 60);
gc.strokeText('brown fox', 30, 100);
gc.textAlign = 'center';
gc.fillText('jumped over', 120, 140);    // centrado em x = 120
gc.textAlign = 'right';
gc.fillText('the lazy dog', 120, 180);   // acaba em x = 120
gc.textAlign = 'left';
const w = gc.measureText('the quick').width;
gc.strokeRect(30 + w, 40, 20, 20);       // quadrado logo a seguir a "the quick"
```

Canvas: caminhos

Definição

Um **caminho** é uma sequência de segmentos que o *canvas* acumula e que **só aparece** quando é traçado ou preenchido.

- `beginPath()`: começa um caminho vazio (sem ele, os segmentos novos ligam-se ao caminho anterior).
- `moveTo(x, y)`: move a "caneta" sem desenhar (ponto inicial).
- `lineTo(x, y)`: segmento de reta desde o último ponto.
- `closePath()`: liga o **último** ponto ao **primeiro**. Só importa para o `stroke()`.
- `stroke()`: traça o caminho ativo com as propriedades de linha atuais.
- `fill()`: preenche o interior. Considera **sempre** o caminho fechado, mesmo sem `closePath()`. Se o caminho se cruza (uma estrela), a regra de preenchimento decide o que é "dentro"†: `nonzero` (omissão) ou `fill("evenodd")`.
- `rect(x, y, width, height)`: junta um retângulo ao caminho, sem `moveTo`; ao contrário de `strokeRect`/`fillRect`, combina-se com outros segmentos do mesmo caminho.

```
gc.beginPath();
gc.moveTo(20, 20);
gc.lineTo(20, 140);
gc.lineTo(140, 20);
gc.stroke();       // dois lados: (20,20)–(20,140)–(140,20), aberto
gc.closePath();
gc.stroke();       // agora com o 3.º lado, de (140,20) a (20,20)

gc.beginPath();
gc.moveTo(40, 40); gc.lineTo(40, 160); gc.lineTo(160, 40);
gc.fill();         // triângulo preenchido, sem precisar de closePath

gc.beginPath();
gc.rect(40, 40, 100, 120);
gc.stroke();       // o mesmo que strokeRect(40, 40, 100, 120)
```

Canvas: propriedades de linha

Definição

Valem para tudo o que se traça com `stroke()`, até mudarem.

- `lineWidth`: largura em píxeis.
- `lineJoin`, as **junções** entre segmentos (vê-se com linhas largas): `"miter"` (bico, omissão), `"round"` (arredondada), `"bevel"` (cortada).
- `lineCap`, as **pontas**: `"butt"` (corta na ponta, omissão), `"round"` (meia circunferência para lá da ponta), `"square"` (meia largura para lá da ponta, com corte reto).
- `setLineDash([cheio, vazio, ...])`: tracejado, alternando troços desenhados e vazios; com um número ímpar de elementos a lista é duplicada (`[3,1,3]` → `[3,1,3,3,1,3]`); `[]` volta à linha contínua.

```
gc.lineWidth = 10;
gc.beginPath(); gc.moveTo(40,20); gc.lineTo(200,60); gc.lineTo(40,100);
gc.lineJoin = 'bevel'; gc.stroke();       // vértice cortado
gc.beginPath(); gc.moveTo(40,60); gc.lineTo(200,100); gc.lineTo(40,140);
gc.lineJoin = 'round'; gc.stroke();       // arredondado
gc.beginPath(); gc.moveTo(40,100); gc.lineTo(200,140); gc.lineTo(40,180);
gc.lineJoin = 'miter'; gc.stroke();       // bico
```

```
gc.lineWidth = 20;
gc.beginPath(); gc.moveTo(40,60); gc.lineTo(200,60);
gc.lineCap = 'butt'; gc.stroke();
gc.beginPath(); gc.moveTo(40,100); gc.lineTo(200,100);
gc.lineCap = 'round'; gc.stroke();        // parece 10 px mais comprida
gc.beginPath(); gc.moveTo(40,140); gc.lineTo(200,140);
gc.lineCap = 'square'; gc.stroke();       // idem, mas com cantos retos
gc.setLineDash([1,1]);                    // ponteado fino
gc.setLineDash([3,1,3]);                  // repete-se como [3,1,3,3,1,3]
gc.setLineDash([3,2,5,4]);                // padrão irregular
```

Antisserrilhamento. A largura e a nitidez de uma linha nem sempre são as pedidas: nas margens usam-se cores intermédias para esconder o aspeto em escada. Com coordenadas **inteiras** há sempre antisserrilhamento (uma linha de 1 px fica "entre" dois píxeis); deslocando **meio píxel** (`x + 0.5`) as linhas horizontais e verticais ficam nítidas.

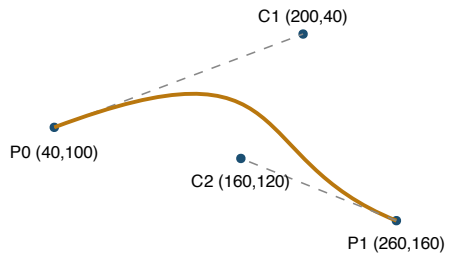
Canvas: curvas

Definição

- `arcTo(xc, yc, x, y, raio)`: arco desde o último ponto até `(x, y)`, com ponto de **controlo** `(xc, yc)` e o raio dado. É **tangente** à reta início–controlo no início e à reta controlo–fim no fim.
- `quadraticCurveTo(xc, yc, x, y)`: curva com **um** ponto de controlo; tangente às retas início–controlo e fim–controlo.
- `bezierCurveTo(xc1, yc1, xc2, yc2, x, y)`: Bézier **cúbica**, com **dois** pontos de controlo; tangente a início–C1 e a fim–C2.
- `arc(x, y, raio, ânguloInicial, ânguloFinal, sentido)`: arco de centro `(x, y)`, sem `moveTo`. Ângulos em **radianos**, 0 à direita, a crescer no sentido dos ponteiros do relógio; `sentido = true` percorre ao contrário (anti-horário), `false` ou omitido no sentido horário. Com `2*Math.PI` é a circunferência completa; `stroke()` dá a circunferência e `fill()` o círculo (ou a "fatia").

```
gc.beginPath(); gc.moveTo(20, 20);
gc.arcTo(140, 20, 140, 140, 100); gc.stroke();
gc.beginPath(); gc.moveTo(40, 100);
gc.quadraticCurveTo(200, 40, 260, 160); gc.stroke();
gc.beginPath(); gc.moveTo(40, 100);
gc.bezierCurveTo(200, 40, 160, 120, 260, 160); gc.stroke();
```

- `arcTo`: sai de (20,20) na horizontal (para o controlo (140,20)) e chega a (140,140) na vertical: um canto arredondado de raio 100.
- `quadraticCurveTo`: o controlo (200,40) **puxa** a curva sem ela lá passar: sai de (40,100) na direção dele e chega a (260,160) vinda da mesma direção.
- `bezierCurveTo`: sai na direção de C1 = (200,40) e chega vinda de C2 = (160,120); com dois controlos pode fazer um "S".




```
gc.beginPath(); gc.arc(80, 80, 40, 0, 2*Math.PI, true);   gc.stroke(); // circunferência
gc.beginPath(); gc.arc(180, 80, 40, 0, Math.PI, true);    gc.fill();   // metade de cima
gc.beginPath(); gc.arc(180, 100, 40, 0, Math.PI, false);  gc.fill();   // metade de baixo
```

De 0 a `Math.PI` é meia volta: o `sentido` decide qual das metades.

Canvas: animação

> **Definição**
>
> Animar é **redesenhar tudo**, frame a frame, depressa o suficiente para parecer contínuo (o *canvas* não tem objetos que se movam). Base de tempo:
>
> - `id = setInterval(f, ms)`: chama `f` de `ms` em `ms`.
> - `id = setTimeout(f, ms)` / `clearTimeout(id)`: chama `f` **uma vez**; chamado dentro de `f`, controla o tempo até ao frame seguinte.
> - `window.requestAnimationFrame(f)`: chama `f` antes do próximo repintar; chamado **recursivamente** dentro de `f`, anima. É o preferido: o intervalo ideal depende do dispositivo e da carga, e é o navegador que o sabe (e pausa em separadores escondidos†).
>
> Cada frame começa por **limpar**: `canvas.width = canvas.width` limpa **e** repõe o contexto (transformações, cores); `gc.clearRect(0, 0, canvas.width, canvas.height)` só limpa, por isso exige que o contexto já esteja no estado inicial (com `save`/`restore`).
> Nunca um ciclo `for`/`while` sem fim: bloquearia a *thread*, sem repintar nem tratar eventos.

- O relógio só muda de segundo a segundo: `setInterval`.
- O catavento é uma animação contínua: `requestAnimationFrame`. A cada frame é redesenhado do zero com as pás rodadas mais $\pi/100$ (1,8°): `alpha` = 0, −0,0314, −0,0628, … rad.
- O `bind(this)` é preciso porque `this.play`, passada como referência, perderia o `this`.

```
class Watch {
  constructor(canvasId) {
    // ...
    this.show();
    setInterval(this.show.bind(this), 1000);   // um frame por segundo
  }
}
class WindMill {
  play() {
    this.base.width = this.base.width;          // novo frame
    this.show(this.gc);
    this.alpha -= WindMill.radialVelocity;
    requestAnimationFrame(this.play.bind(this));
  }
}
```

Canvas: embeber e eventos

O *canvas* integra-se no HTML, CSS e JavaScript à volta (as *applets* e *plugins* eram caixas fechadas). É **transparente** por omissão, e com `position: absolute` dentro de um contentor `position: relative` pode ficar **por cima** de outro conteúdo, para desenhar sobre ele:

```
<div style="position: relative;">
  <div style="position: absolute; top: 0; left: 0;
              font: 20px Arial; width: 350px; height: 250px;
              line-height: 22px;">
    Lorem ipsum dolor sit amet, consectetur...
  </div>
  <canvas id="tela" width="350" height="250"
          style="position: absolute; top: 0; left: 0;"></canvas>
</div>
```

Exemplo resolvido

Arrastar para **destacar** texto (o *canvas* recebe eventos de rato como qualquer elemento):

```
let start = null;
canvas.onmousedown = function(e) { start = relativeCoords(e); };
canvas.onmousemove = function(e) {
  if (start !== null) {
    const p = relativeCoords(e);
    canvas.width = canvas.width;      // limpa
    lines();                          // redesenha as linhas guia
    gc.fillStyle = 'rgba(255,165,0,0.6)';
    gc.fillRect(start.x, (start.y / 20) * 19, p.x - start.x, 22);
  }
};
canvas.onmouseup = function(e) { start = null; };

function relativeCoords(event) {
  const bounds = event.target.getBoundingClientRect();
  const x = event.clientX - bounds.left;
  const y = event.clientY - bounds.top;
  return { x, y };
}
```

`mousedown` guarda o ponto inicial; cada `mousemove` com o botão premido redesenha um retângulo semi-transparente do início à posição atual; `mouseup` acaba. As coordenadas do evento (`clientX`, `clientY`) são da **janela**, e as do *canvas* são do **elemento**: `relativeCoords` subtrai a posição do *canvas* dada por `getBoundingClientRect()`. Com o *canvas* em `left = 20`, `top = 50` e um clique em (120, 90): `x = 120 − 20 = 100`, `y = 90 − 50 = 40`.

Canvas: transformações

Definição

As transformações mudam o **sistema de coordenadas** dos desenhos seguintes (não desenham nada). Cada uma **compõe-se** com as anteriores.

- `translate(dx, dy)`: desloca a origem; `translate(a,b)` seguido de `translate(c,d)` é `translate(a+c, b+d)`.
- `rotate(ângulo)`: roda o referencial em radianos (positivo: sentido dos ponteiros, por causa do Y para baixo); `rotate(a)` e `rotate(b)` dão `rotate(a+b)`.
- `scale(sx, sy)`: escala cada eixo (`1` igual, `2` dobro, `0.5` metade, negativo **espelha**).
- `transform(a, b, c, d, e, f)` **compõe** uma matriz; `setTransform(a, b, c, d, e, f)` **substitui** a atual. `a`, `d`: escala horizontal e vertical; `b`, `c`: torção horizontal e vertical; `e`, `f`: deslocamento horizontal e vertical.
- `save()` **empilha** uma cópia do estado do contexto (transformações, cores, linha, letra); `restore()` **desempilha** e repõe-no, desfazendo tudo desde o `save` correspondente (pilha: podem aninhar-se).

Exemplo resolvido

Cada `sample(...)` desenha na origem do referencial **atual**:

```
sample('teste 1');
gc.translate(0, 60);    sample('teste 2');   // origem em (0,60)
gc.translate(140, 0);   sample('teste 3');   // (140,60)
gc.translate(0, 60);    sample('teste 4');   // (140,120)
gc.translate(-140, 0);  sample('teste 5');   // (0,120)
```

```
sample('teste 1');
gc.translate(140, 120);    sample('teste 2');
gc.rotate(-Math.PI/2);     sample('teste 3');  // roda em torno de (140,120)
gc.rotate(19/12*Math.PI);  sample('teste 5');  // acumula
```

O `translate` veio antes, por isso o centro da rotação é (140,120), não o (0,0) do *canvas*.

```
sample('teste 1');
gc.translate(0, 40);     gc.scale(2, 2);      sample('teste 2'); // dobro
gc.translate(60, -20);   gc.scale(0.5, 0.5);  sample('teste 3');
gc.translate(-120, 120); gc.scale(1, 0.5);    sample('teste 4');
gc.translate(240, 0);    gc.scale(-0.5, 2);   sample('teste 5'); // X espelhado
gc.transform(1, Math.tan(Math.PI/6), 0, 1, 0, 0);   // torção de tan(30°)
```

Depois de `scale(2, 2)`, o `translate(60, -20)` também é escalado: anda 120 e −40 píxeis reais. O `scale(0.5, 0.5)` volta à escala 1 (2 × 0,5). O último `transform` mantém escala 1 e deslocamento 0, mas inclina o desenho (torção horizontal).

Exemplo resolvido

A ordem importa: o mesmo ponto local (50, 0), com as mesmas duas transformações.

A: `translate(100, 0)` e depois `rotate(Math.PI/2)`.

1. A origem passa para (100, 0).
2. O referencial roda 90°: o X local aponta para baixo.
3. (50, 0) local são 50 para baixo a partir de (100, 0): **(100, 50)**.

B: `rotate(Math.PI/2)` e depois `translate(100, 0)`.

1. A origem fica em (0, 0), mas o X local já aponta para baixo.
2. O `translate(100, 0)` anda 100 ao longo do X **rodado**: a origem vai para (0, 100).
3. (50, 0) local são mais 50 para baixo: **(0, 150)**.

```
sample('teste 1');
gc.save();
gc.translate(0, 40);
sample('teste 2');     // 40 px abaixo
gc.restore();          // desfaz o translate(0, 40)
gc.translate(120, 0);
sample('teste 3');     // 120 px à direita da posição ORIGINAL
```

Sem o `save`/`restore`, as duas translações somavam-se. O padrão `save` → alterar → desenhar → `restore` aplica uma transformação (ou cor, ou letra) só a uma parte do desenho; os exemplos seguintes usam-no para cada marca do mostrador e cada pá.

Exemplos completos

Exemplo resolvido **Relógio**

```
class Watch {
  static numberSize = 14;
  static largeMarkSize = 10;
  static smallMarkSize = 5;
  constructor(canvasId) {
    this.base = document.getElementById(canvasId);
    this.gc = this.base.getContext('2d');
    this.center = { x: this.base.width / 2, y: this.base.height / 2 };
    this.length = Math.min(this.center.x, this.center.y);
    this.show();
    setInterval(this.show.bind(this), 1000);
  }
  show() {
    const gc = this.gc;
    const now = new Date();
    this.base.width = this.base.width;           // limpa
    gc.translate(this.center.x, this.center.y);  // origem no centro
    gc.textAlign = 'center';
    gc.textBaseline = 'middle';
    gc.font = Watch.numberSize + 'pt Arial bold';
    this.face(gc);
    gc.strokeStyle = '#228';
    this.hand(gc, now.getHours() + now.getMinutes()/60, 12, this.length/2, 3);
    this.hand(gc, now.getMinutes(), 60, this.length - Watch.numberSize, 2);
    this.hand(gc, now.getSeconds(), 60, this.length - Watch.numberSize, 1);
  }
```

- **Construtor**: guarda o *canvas* e o contexto, o centro e o comprimento (o menor de meia largura e meia altura, para caber), desenha e agenda `show` a cada segundo.
- **`show`**: limpa, põe a origem no centro (tudo o resto desenha-se em relação ao centro), centra o texto na horizontal e na vertical, e desenha o mostrador e os três ponteiros.

Exemplo resolvido **Relógio (continuação)**

```
  face(gc) {
    gc.strokeStyle = '#228';
    gc.fillStyle = '#228';
    for (let minutes = 1; minutes <= 60; minutes++) {
      const angle = minutes * Math.PI/30 - Math.PI/2;
      const isHour = (minutes % 5) === 0;
      const len = this.length - Watch.numberSize - Watch.largeMarkSize;
      if (isHour) {
        gc.save();
        gc.fillStyle = 'orange';
        gc.fillText(minutes / 5, len*Math.cos(angle), len*Math.sin(angle));
        gc.restore();
      }
      gc.save();
      gc.rotate(angle);
      gc.moveTo(this.length, 0);
      gc.lineTo(this.length - (isHour ? Watch.largeMarkSize : Watch.smallMarkSize), 0);
      gc.stroke();
      gc.restore();
    }
  }
```

- Uma volta de 60 minutos são 2π, por isso cada minuto vale $2\pi/60 = \pi/30$; o $-\pi/2$ põe o minuto 0 no **topo** (0 rad é à direita).
- De 5 em 5 minutos escreve a hora (com `save`/`restore` à volta, para mudar o `fillStyle` só ali) e faz uma marca maior.
- Cada marca desenha-se sempre **na horizontal** e roda-se o referencial: é mais simples do que calcular as coordenadas rodadas.

Exemplo resolvido **Relógio (continuação)**

```
  hand(gc, value, max, length, width) {
    const angle = (2*(value % max)/max - 1) * Math.PI;
    gc.save();
    gc.rotate(angle);
    gc.moveTo(width/2, 0);
    gc.lineTo(0, length);
    gc.lineTo(-width/2, 0);
    gc.lineWidth = width;
    gc.stroke();
    gc.restore();
  }
}
const watch = new Watch('base');
```

Um ponteiro genérico: `value` é a posição atual (ex.: minutos), `max` o total (60), e `length`/`width` o comprimento e a largura da base. O ângulo é a fração `(value % max)/max` de uma volta; roda-se o referencial e desenha-se o ponteiro sempre na "vertical" local, de `(width/2, 0)` a `(0, length)` a `(-width/2, 0)`.

Exemplo resolvido **Catavento**

```
class WindMill {
  static radialVelocity = Math.PI/100;
  constructor(canvasId) {
    this.base = document.getElementById(canvasId);
    this.gc = this.base.getContext('2d');
    this.gc.fillStyle = 'orange';
    this.center = { x: this.base.width/2, y: this.base.height/2 };
    this.length = Math.min(this.center.x, this.center.y);
    this.alpha = 0;
    this.play();
  }
  play() {
    this.base.width = this.base.width;      // novo frame
    this.show(this.gc);
    this.alpha -= WindMill.radialVelocity;  // roda um pouco
    requestAnimationFrame(this.play.bind(this));
  }
  show(gc) {
    this.stick(gc);
    for (let beta = this.alpha; beta < this.alpha + 2*Math.PI; beta += Math.PI/2)
      this.sail(gc, beta);                  // 4 pás a 90°
    this.centerCircle(gc, this.length/5, '#228');
    this.centerCircle(gc, this.length/9, 'orange');
    this.centerCircle(gc, this.length/30, '#228');
  }
```

- `radialVelocity` é **estático** (igual para todas as instâncias); `base`, `gc`, `center`, `length` e `alpha` são da **instância**.
- O construtor arranca logo a animação com `play()`.
- `show` desenha a vara, as 4 pás e 3 círculos no centro, **por esta ordem**: os círculos ficam por cima e escondem a junção das pás.

Exemplo resolvido **Catavento (continuação)**

```
  stick(gc) {
    gc.beginPath();
    gc.moveTo(this.center.x, 2*this.center.y);   // fundo do canvas
    gc.lineTo(this.center.x, this.center.y);     // até ao centro
    gc.strokeStyle = '#228';
    gc.lineWidth = 10;
    gc.stroke();
  }
  centerCircle(gc, length, color) {
    gc.beginPath();
    gc.arc(this.center.x, this.center.y, length, 0, 2*Math.PI);
    gc.fillStyle = color;
    gc.fill();
  }
  static inLine(p, a, l) {
    return { x: p.x + l*Math.cos(a), y: p.y + l*Math.sin(a) };
  }
```

A vara é uma linha grossa do fundo ao centro. `inLine(p, a, l)` dá o ponto à distância `l` de `p` na direção `a` (coordenadas **polares**); é **estático** porque só faz contas com os argumentos.

Exemplo resolvido **Catavento (continuação)**

```
  sail(gc, beta) {
    const center = this.center, length = this.length;
    const a  = WindMill.inLine(center, beta, length);
    const b  = WindMill.inLine(center, beta - Math.PI/2, length/2);
    const c1 = WindMill.inLine(a, beta - 3*Math.PI/4, length/2);
    const c2 = WindMill.inLine(b, beta, length/2);
    const c3 = WindMill.inLine(center, beta - Math.PI/4, length/2);
    gc.beginPath();                              // metade de fora
    gc.moveTo(center.x, center.y);
    gc.lineTo(a.x, a.y);
    gc.bezierCurveTo(c1.x, c1.y, c2.x, c2.y, b.x, b.y);
    gc.closePath();
    gc.fillStyle = '#228';
    gc.fill();
    gc.beginPath();                              // metade de dentro
    gc.moveTo(center.x, center.y);
    gc.lineTo(a.x, a.y);
    gc.bezierCurveTo(c1.x, c1.y, c3.x, c3.y, center.x, center.y);
    gc.fillStyle = 'orange';
    gc.fill();
  }
}
const windmill = new WindMill('base');
```

Uma pá na direção `beta`: as duas metades partilham o segmento do centro à ponta `a`. A de fora vai de `a` a `b` (controlos `c1`, `c2`) e fecha no centro, a azul; a de dentro volta de `a` ao centro (controlos `c1`, `c3`), a laranja. As duas cores dão o efeito de torção da pá.

10 Tema 10
O servidor: Node.js

Primeiro servidor

Definição

O **Node.js** corre JavaScript **fora do navegador**: na linha de comandos ou como **servidor web**. É:

- **multiplataforma** (Linux, Windows, macOS);
- ***single threaded***: o código da aplicação corre numa **só** *thread*;
- de ***I/O* não bloqueante** (*event driven*): ler um ficheiro ou esperar pela rede não bloqueia a *thread*; a operação é pedida e o resultado chega mais tarde a um ***callback*** ou evento;
- baseado no motor **V8** (o do Chrome);
- com **gestão de pacotes** pelo `npm`.

Por haver uma só *thread*†, uma operação que bloqueasse pararia o servidor inteiro. Por isso pede-se a operação e devolve-se logo o controlo ao ***event loop***; quando o sistema operativo a termina, o Node agenda o *callback*. É daí que vêm os *callbacks*, os eventos `data`/`end` e a preferência pelas versões assíncronas do `fs`.

```
const http = require('http');
const server = http.createServer(function (request, response) {
    response.writeHead(200, {'Content-Type': 'text/plain'});
    response.end('Olá mundo\n');
});
server.listen(8008);
```

- `require('http')` inclui o módulo interno **`http`**, que implementa o protocolo.
- `http.createServer(callback)` cria o servidor; o *callback* processa o **ciclo pedido-resposta** e é chamado **uma vez por pedido**, com o `request` (o pedido) e a `response` (a resposta a construir).
- `server.listen(porta)` **lança** o servidor, que fica à escuta indefinidamente.

```
$ node ola.js
```

```
# noutro terminal:
$ curl 'http://localhost:8008/'
Olá mundo
```

`node ola.js` corre o servidor até `CTRL+C`. Testa-se noutro terminal com `curl`: `localhost` é a própria máquina, a porta é a do `listen`, e as plicas evitam que a *shell* interprete `?` e `&`.

Módulos

Um **módulo** implementa funcionalidades para o resto da aplicação, **partilhando** só o que interessa e mantendo o resto **restrito**. Incluem-se com **`require()`**: módulos **instalados** (internos ou do `npm`) pelo **nome** (`require('http')`), módulos **próprios** pelo **caminho** (`require('./modulo.js')`). O `require` devolve o que o módulo **exportou** em `module.exports`.

Há cinco formas de partilhar, da mais simples à mais expressiva:

Exemplo resolvido

```
// 1. Variáveis globais (evitar): só valores e funções anónimas
// atribuídas a nomes globais; function, class, let e const ficam no módulo.
f = function() { /* ... */ }                        // modulo.js
x = 1
require('./modulo.js'); f(); x;                     // aplicacao.js

// 2. Uma função anónima: um só valor por módulo, sem poluir o global.
module.exports = function() { /* ... */ }           // modulo.js
const f = require('./modulo.js'); f();              // aplicacao.js

// 3. Funções nomeadas: vários valores, como propriedades.
module.exports.f = function() { /* ... */ };        // modulo.js
module.exports.x = 1;
const mod = require('./modulo.js'); mod.f(); mod.x;  // aplicacao.js
```

Exemplo resolvido

```
// 4. Uma classe anónima: orientação a objetos, uma classe por módulo.
module.exports = class {                            // modulo.js
    constructor() { this.c = 1; }
    m() { /* ... */ }
}
const T = require('./modulo.js');                   // aplicacao.js
const t = new T(); t.m(); t.c;

// 5. Classes nomeadas: vários tipos de objetos por módulo.
module.exports.Type1 = class { m() { /* ... */ } };  // modulo.js
module.exports.Type2 = class { m() { /* ... */ } };
const mod = require('./modulo.js');                 // aplicacao.js
const t1 = new mod.Type1(); t1.m();
const t2 = new mod.Type2(); t2.m();
```

As globais potenciam colisões de nomes entre módulos. Com classes exportam-se construtores e usa-se `new` na aplicação.

Módulos internos

O Node traz **módulos internos** (ver a API do Node). Para um servidor HTTP básico: **`http`** (pedidos e respostas), **`url`** (analisar URLs), **`stream`** (fluxos de dados) e **`fs`** (ficheiros, ao estilo POSIX).

`http`

Implementa o HTTP a **baixo nível**: dá os cabeçalhos já como objetos JS (sem *parsing*), trata o corpo como *stream*, suporta corpos grandes (*chunked*) e permite negociar outros protocolos (*WebSockets*). Tem vários tipos (`Agent`, gestão de ligações; `Client`, pedidos como cliente); aqui interessa o **`Server`**.

```
const server = http.createServer(function (request, response) {
    response.writeHead(200, {'Content-Type': 'text/plain'});
    response.write('Método: ' + request.method + '\n');
    response.write('URL: ' + request.url + '\n');
    response.end();
});
server.listen(8008);
```

Definição

- `request.method`: o método (`'GET'`, `'POST'`, …).
- `request.url`: o URL **relativo ao servidor**: `http://servidor.up.pt/hello?nome=valor` chega como `'/hello?nome=valor'`.
- O `request` é também um ***stream***: é assim que se lê o corpo.
- `response.writeHead(estado, cabeçalhos)`: escreve o código de estado e um objeto com os cabeçalhos (`{'Content-Type': 'text/plain'}`).
- `response.write(dados)`: escreve uma parte do **corpo**; pode chamar-se várias vezes.
- `response.end([dados])`: escreve o último pedaço (opcional) e **fecha** a resposta.

O ciclo de um pedido no servidor:

1. chega uma ligação TCP à porta (ex.: 8008);
2. o `http` lê o pedido (linha inicial e cabeçalhos) e cria o `request` e a `response`;
3. o *event loop* chama o *callback* do `createServer` (o corpo, se for lido, chega depois, por eventos);
4. o *callback* decide a resposta (ex.: lê um ficheiro com `fs`) e chama `writeHead`, `write` (zero ou mais vezes) e `end`;
5. o processo fica livre para outros pedidos; nunca bloqueou à espera de *I/O*.

`url`

`url.parse(request.url, true)` devolve um objeto com as partes do URL (protocolo, servidor, porta, caminho, *query-string*), também para URLs relativos. Com o segundo argumento `true`, a *query-string* vem decomposta num objeto (sem ele, vem como *string*).

- `pathname`: o caminho, sempre a começar por `/`; serve para **encaminhar** o pedido;
- `query`: um objeto com uma propriedade por parâmetro.

Exemplo resolvido

```
const url = require('url');
http.createServer(function (request, response) {
    const parsedUrl = url.parse(request.url, true);
    const query = parsedUrl.query;
    const name = query.name === undefined ? 'amig@' : query.name;
    let mensagem;
    switch (parsedUrl.pathname) {
    case '/hello': mensagem = 'Olá ' + name;   break;
    case '/bye':   mensagem = 'Adeus ' + name; break;
    default:       mensagem = 'Podes repetir?';
    }
    // ... escrever a mensagem na resposta
}).listen(8003);
```

- Pedido `http://exemplo.up.pt/hello?name=Rui`: `request.url` é `'/hello?name=Rui'`, `pathname` é `'/hello'` e `query` é `{ name: 'Rui' }`: a mensagem é `'Olá Rui'`.
- Pedido `/hello` sem *query-string*: `query.name` é `undefined`, e fica `'Olá amig@'`.

`stream`

Um interface **abstrato** para fluxos de dados, suportado por objetos como o `request`. É preciso para ler o **corpo** de um `POST`, que chega aos bocados (*chunks*), de forma assíncrona, por **eventos**.

> **Definição**
>
> `objeto.on(evento, callback)` regista um *callback* e **devolve o próprio objeto**, para encadear. Eventos:
>
> - `'data'`: chegou um bloco (*string* ou *buffer*; um *buffer* passa a *string* ao concatenar com `+=`);
> - `'end'`: a leitura **acabou**: é aqui que se processa tudo;
> - `'error'`: houve um erro; regista-se no *log* ou responde-se com um erro HTTP (ex.: 400).
>
> O `fs.readFile`† também lê por blocos, mas só mostra o resultado final num *callback* `(err, data)`.

Exemplo resolvido

```
let body = '';
switch (request.method) {
case 'POST':
    request
        .on('data', (chunk) => { body += chunk; })
        .on('end', () => {
            try { query = JSON.parse(body); /* processar query */ }
            catch (err) { /* erros de JSON */ }
        })
        .on('error', (err) => { console.log(err.message); });
    break;
// ...
}
```

`POST /users` com o corpo `{"nome":"Ana","idade":25}`, entregue em dois blocos:

1. O `case 'POST'` regista os três *callbacks* e devolve logo o controlo ao *event loop*.
2. Chega o bloco 1: `'data'` com `'{"nome":"Ana"'`; `body` fica igual.
3. Chega o bloco 2: `'data'` com `',"idade":25}'`; `body` fica com o JSON completo.
4. Acabou: `'end'` faz `JSON.parse(body)` e dá `{nome: 'Ana', idade: 25}`; um corpo malformado ia para o `catch`.
5. `'error'` não dispara; dispararia se a ligação caísse a meio.

`fs`

```
const fs  = require('fs');
const fsp = require('fs').promises;
```

Lê e escreve ficheiros e dá estatísticas sobre eles, de forma **assíncrona** (preferida, também com promessas, em `fsp`) ou **síncrona**. (Há mais: listar diretorias, gerir ficheiros.)

Definição

- `fs.readFile(caminho, codificação, callback)`: a **codificação** é opcional (sem ela, os dados vêm como *buffer*). O *callback* recebe `(err, data)`: só se processa `data` se **não** houver `err`. Com `fsp.readFile`, os dados chegam ao `then` e os erros ao `catch`.
- `fs.writeFile(caminho, dados, callback)`: os dados são *string* ou *buffer* (`JSON.stringify` para objetos); o *callback* recebe um eventual erro (com `fsp`, o `catch`).
- `readFileSync` / `writeFileSync`: versões **síncronas**; bloqueiam a única *thread* e o servidor inteiro. Preferir as assíncronas, ou `await` sobre as de `fsp` (estilo sequencial, mas não bloqueante).
- `fs.stat(caminho, callback)`: dá um `Stat` com `size` (bytes), `blocks`, `birthtime` (criação), `ctime`/`mtime`/`atime` (do *inode*) e os métodos `isFile()` e `isDirectory()`.

```
fs.readFile('dados.txt', 'utf8', function (err, data) {
    if (!err) { /* processar data (string) */ }
});
fsp.readFile('dados.txt', 'utf8').then((data) => { /* processar */ });
fs.readFile('dados.json', function (err, data) {
    if (!err) { const dados = JSON.parse(data.toString()); /* ... */ }
});
fs.writeFile('dados.txt', dados_string, (err) => { if (err) { /* erro */ } });
fsp.writeFile('dados.txt', dados).catch(() => { /* erro */ });
```

```
let dados = fs.readFileSync('dados.txt');
fs.writeFileSync('dados.txt', dados);
(async () => {
    let dados = await fsp.readFile('dados.txt');
    await fsp.writeFile('dados.txt', dados);
})();
fs.stat(pathname, (err, stats) => {
    if (err) throw err;
    console.log(pathname + ' tem ' + stats.size + ' bytes');
});
```

O Node atende tudo num **só processo** que nunca bloqueia (o *I/O* é delegado e continua num *callback*); o **CGI** (Tema 11) cria um **processo novo por pedido**: mais simples de raciocinar, mas paga a criação de um processo a cada pedido.

Exemplos completos

Exemplo resolvido **Servidor de páginas estáticas**

Serve **ficheiros e diretorias** do disco, só a `GET` sem *query-string*, com estados 2XX, 3XX, 4XX e 5XX.

```
const http = require('http');
const path = require('path');
const url  = require('url');
const fs   = require('fs');
const conf = require('./conf.js');
```

```
// conf.js
module.exports.documentRoot = '/home/zp/public_html/about';
module.exports.defaultIndex = 'index.html';
module.exports.port         = 8007;
module.exports.mediaTypes   = {
    'txt': 'text/plain', 'html': 'text/html', 'css': 'text/css',
    'js': 'application/javascript',
    'png': 'image/png', 'jpeg': 'image/jpeg', 'jpg': 'image/jpeg',
};
```

A configuração (raiz dos documentos, índice das diretorias, porta, tipos por extensão) fica num módulo à parte; um servidor real tem muitas mais.

Exemplo resolvido **Servidor de páginas estáticas (continuação)**

```
function getPathname(request) {
    const purl = url.parse(request.url);
    let pathname = path.normalize(conf.documentRoot + purl.pathname);
    if (!pathname.startsWith(conf.documentRoot))
        pathname = null;
    return pathname;
}
function getMediaType(pathname) {
    const pos = pathname.lastIndexOf('.');
    let mediaType;
    if (pos !== -1)
        mediaType = conf.mediaTypes[pathname.substring(pos + 1)];
    if (mediaType === undefined)
        mediaType = 'text/plain';
    return mediaType;
}
function isText(mediaType) {
    return !mediaType.startsWith('image');
}
```

- `getPathname` junta o caminho do URL à raiz e **normaliza-o** (`path.normalize` tira os `.` e `..`). Se deixar de começar pela raiz, é inválido (`null`): `/../../etc/passwd` não sai da diretoria servida.
- `getMediaType` deduz o tipo pela **extensão**, ou `text/plain`.
- `isText`: tudo o que não é imagem é texto (decide a codificação).

Exemplo resolvido **Servidor de páginas estáticas (continuação)**

```
function doGetPathname(pathname, response) {
    const mediaType = getMediaType(pathname);
    const encoding = isText(mediaType) ? 'utf8' : null;
    fs.readFile(pathname, encoding, (err, data) => {
        if (err) {
            response.writeHead(404);                    // Not Found
            response.end();
        } else {
            response.writeHead(200, { 'Content-Type': mediaType });
            response.end(data);
        }
    });
}
```

```
function doGetRequest(request, response) {
    const pathname = getPathname(request);
    if (pathname === null) {
        response.writeHead(403);                        // Forbidden
        response.end();
    } else
        fs.stat(pathname, (err, stats) => {
            if (err) {
                response.writeHead(500);                // Internal Server Error
                response.end();
            } else if (stats.isDirectory()) {
                if (pathname.endsWith('/'))
                    doGetPathname(pathname + conf.defaultIndex, response);
                else {
                    response.writeHead(301, { 'Location': pathname + '/' });
                    response.end();                     // Moved Permanently
                }
            } else
                doGetPathname(pathname, response);
        });
}
```

Exemplo resolvido **Servidor de páginas estáticas (continuação)**

```
http.createServer((request, response) => {
    switch (request.method) {
    case 'GET':
        doGetRequest(request, response);
        break;
    default:
        response.writeHead(501);                    // Not Implemented
        response.end();
    }
}).listen(conf.port);
```

- `doGetPathname` lê o ficheiro com a codificação do seu tipo: `404` se falhar, `200` com o `Content-Type` se correr bem.
- `doGetRequest`: `403` se o caminho é inválido; senão `fs.stat`: `500` se falhar; uma **diretoria** com `/` final recebe o índice, sem `/` é redirecionada (`301`); um ficheiro é servido.
- O servidor só aceita `GET`; o resto dá `501`.

`GET /pagina.html`, existente em `/home/zp/public_html/about/`:

1. É `GET`: `doGetRequest`.
2. `getPathname`: `pathname` do URL é `'/pagina.html'`; junto à raiz e normalizado dá `/home/zp/public_html/about/pagina.html`, que começa pela raiz: válido.
3. `fs.stat` é assíncrono (o *event loop* fica livre); responde sem erro e não é diretoria.
4. `doGetPathname`: a extensão `html` dá `'text/html'`, que é texto: `encoding = 'utf8'`.
5. `fs.readFile` acaba sem erro: `writeHead(200, {'Content-Type': 'text/html'})` e `end(data)`.

Se o ficheiro não existisse, o passo 5 teria `err`: resposta `404` sem corpo.

Exemplo resolvido **Servidor do contador partilhado**

Um contador partilhado por todos os clientes: `POST` incrementa ou repõe, e `GET /update` **subscreve** as atualizações por SSE (a resposta fica **aberta**, sem `end()`, e o servidor vai escrevendo `data: ...\n\n`).

```
"use strict";
let PORT = 8001;
let http    = require('http');
let url     = require('url');
let counter = require('./model.js');
let updater = require('./updater.js');
const headers = {
    plain: { 'Content-Type': 'application/json',
             'Cache-Control': 'no-cache',
             'Access-Control-Allow-Origin': '*' },
    sse:   { 'Content-Type': 'text/event-stream',
             'Cache-Control': 'no-cache',
             'Access-Control-Allow-Origin': '*',
             'Connection': 'keep-alive' }
};
```

`counter` é o **modelo** (separado do servidor) e `updater` gere as ligações abertas. Há cabeçalhos `plain` (respostas aos `POST`, que fecham) e `sse` (a subscrição, que fica aberta).

Exemplo resolvido **Servidor do contador partilhado (continuação)**

```
// model.js
let count = 0;
module.exports.incr  = function() { count++; };
module.exports.reset = function() { count = 0; };
module.exports.get   = function() { return count; };
```

```
// updater.js
let responses = [];
module.exports.remember = function(response) {
    responses.push(response);
};
module.exports.forget = function(response) {
    let pos = responses.findIndex((resp) => resp === response);
    if (pos > -1) responses.splice(pos, 1);
};
module.exports.update = function(message) {
    for (let response of responses)
        response.write('data: ' + message + '\n\n');
};
```

O *updater* guarda (`remember`) e esquece (`forget`) as respostas abertas, e **difunde** (`update`) uma linha `data:` a todas, com o **duplo** fim de linha do SSE.

Exemplo resolvido **Servidor do contador partilhado (continuação)**

```
function doGet(pathname, request, response) {
    let answer = {};
    switch (pathname) {
    case '/update':
        updater.remember(response);
        request.on('close', () => updater.forget(response));
        setImmediate(() => updater.update(counter.get()));
        answer.style = 'sse';
        break;
    default:
        answer.status = 400;
    }
    return answer;
}
function doPost(pathname) {
    let answer = {};
    switch (pathname) {
    case '/incr':  counter.incr();  updater.update(counter.get()); break;
    case '/reset': counter.reset(); updater.update(counter.get()); break;
    default:       answer.status = 400;
    }
    return answer;
}
```

- `doGet` só aceita `/update`: guarda a `response`, esquece-a quando o cliente fecha (`'close'`), e difunde o valor atual com `setImmediate` (como `setTimeout(f, 0)`: corre **depois** deste processamento, já com os cabeçalhos enviados). Marca `style = 'sse'`.
- `doPost` faz `/incr` e `/reset` e difunde o novo valor; o resto é `400`.

Exemplo resolvido **Servidor do contador partilhado (continuação)**

```
http.createServer(function (request, response) {
    const preq = url.parse(request.url, true);
    const pathname = preq.pathname;
    let answer = {};
    switch (request.method) {
    case 'GET':  answer = doGet(pathname, request, response); break;
    case 'POST': answer = doPost(pathname); break;
    default:     answer.status = 400;
    }
    if (answer.status === undefined) answer.status = 200;
    if (answer.style === undefined)  answer.style  = 'plain';
    response.writeHead(answer.status, headers[answer.style]);
    if (answer.style === 'plain')
        response.end();
}).listen(PORT);
```

Um **ponto único** escreve a resposta: por omissão `200` e `plain`; só fecha (`end()`) se **não** for SSE.
O cliente B já fez `GET /update`: a sua `response` está guardada no *updater*, recebeu os cabeçalhos `sse` e nunca fechou. O cliente A faz `POST /incr`:

1. `doPost('/incr')`: `counter.incr()` passa de 3 a 4, e `updater.update(4)` escreve `'data: 4\n\n'` na `response` de B, que o `EventSource` de B recebe sem ter pedido nada.
2. A resposta de A fica `200`, `plain`, e fecha-se.
3. Se B fechar o separador, o `'close'` do seu pedido chama `forget` e tira-o da lista.

Exemplo resolvido **Servidor do jogo do galo**

Um servidor **WebSockets** (outro protocolo, negociado a partir de um pedido HTTP com *upgrade*), que lê e difunde mensagens **JSON**.

```
const WebSocketServer = require('websocket').server;
const http = require('http');
const httpServer = http.createServer(() => {}).listen(PORT);
const webSocketServer = new WebSocketServer({ httpServer });

webSocketServer.on('request', function (request) {
    const connection = request.accept(null, request.origin);
    remember(connection);
    connection.sendUTF(JSON.stringify({ board, current }));
    connection.on('message', function (message) {
        if (message.type === 'utf8')
            process(JSON.parse(message.utf8Data));
        else
            console.log('Unsupported message type: ' + message.type);
    });
    connection.on('close', () => forget(connection));
});
```

- O `http` é necessário, mas não processa pedidos (o *callback* está vazio): serve de base ao `WebSocketServer` (biblioteca `websocket`).
- O servidor é um ***stream* de pedidos** (`'request'`); cada um dá uma `connection` com `accept`. A conexão é um ***stream* de mensagens** (`'message'`).
- Ao aceitar: guarda a conexão e envia-lhe o tabuleiro. Mensagens UTF-8 são JSON e vão para `process`. Ao fechar, esquece-a.
- Devia controlar-se a **origem** do pedido (*same origin policy*).

Exemplo resolvido **Servidor do jogo do galo (continuação)**

```
let connections = [];
function remember(connection) { connections.push(connection); }
function forget(connection) {
    let pos = connections.findIndex((conn) => conn === connection);
    if (pos > -1) connections.splice(pos, 1);
}
function broadcast(data) {
    let json = JSON.stringify(data);
    for (let connection of connections)
        connection.sendUTF(json);
}
```

```
let current = 'X';
let board = undefined;
function initBoard() {
    board = new Array(9);
    for (let b = 0; b < 9; b++) board[b] = ' ';
}
function play(pos) {
    let piece = current;
    current = (current == 'X' ? 'O' : 'X');
    board[pos] = piece;
    return piece;
}
initBoard();
```

Exemplo resolvido **Servidor do jogo do galo (continuação)**

```
function process(message) {
    switch (message.command) {
    case 'reset':
        initBoard();
        broadcast({ board, current });
        break;
    case 'play':
        if ('pos' in message) {
            const piece = play(message.pos);
            broadcast({ pos: message.pos, piece, current });
        } else
            console.log('pos expected in message');
        break;
    default:
        console.log('Invalid command: ' + message.command);
    }
}
```

`broadcast` é o `update` do contador sobre WebSockets. `process` reage ao `command`: `'reset'` difunde o tabuleiro **inteiro**; `'play'` exige `pos` e difunde só a **jogada**.
Clientes A e B ligados, `current = 'X'`; A envia `{"command":"play","pos":4}`:

1. O `'message'` de A chama `process({command: 'play', pos: 4})`.
2. `play(4)`: `piece = 'X'`, `current` passa a `'O'`, `board[4] = 'X'`, e devolve `'X'`.
3. `broadcast({pos: 4, piece: 'X', current: 'O'})` envia a A **e** a B.
4. Os dois põem `X` na posição 4 e sabem que joga `O`.

Com `{"command":"reset"}`, `initBoard()` limpa o tabuleiro e todos recebem `{board, current}` completo.

11 Tema 11 CGI e alternativas

O modelo CGI

O **CGI** (*Common Gateway Interface*) normalizou a interface entre os servidores HTTP (pensados para conteúdo **estático**) e os programas que geram conteúdo **dinâmico**. Começou para programas em C, mas serve **qualquer linguagem** que saiba **ler** do `stdin`, **escrever** no `stdout` e **ler variáveis de ambiente**.

> **Definição** **Um processo por pedido**
>
> - O servidor lança **um processo CGI novo por cada pedido**† (`fork()` + `exec()`: criar o processo, carregar o interpretador e destruí-lo no fim, o que é caro).
> - O **ambiente** do processo tem os cabeçalhos e a informação do pedido, como **variáveis de ambiente**.
> - O **`stdin`** recebe o **corpo** do pedido.
> - O **`stdout`** recebe o que o programa escreve: o fim do cabeçalho, uma linha em branco e o corpo da resposta.

```
#!/usr/bin/bash
echo "Content-type: text/html"
echo ""
echo "<h1>Olá mundo</h1>"
```

Quase todos os servidores HTTP suportam CGI (há até um módulo para Node.js). Há alternativas mais eficientes, **Fast-CGI** e **SCGI** (*Simple Common Gateway Interface*), que evitam o custo de um processo por pedido. No Node, um processo sempre vivo atende todos os pedidos (Tema 10).

Variáveis de ambiente

As **variáveis de ambiente** são pares nome/valor definidos pelo contexto de um processo (ex.: a *shell* que o lançou). O **CGI 1.0** tinha um conjunto restrito; o **CGI 1.1** acrescentou conjuntos novos, com prefixos próprios: cabeçalhos do pedido (`HTTP_`), o **cliente remoto** e o **servidor**.

Conjunto	Variável	Contém
Original (1.0)	`DOCUMENT_ROOT`	diretoria servida
	`GATEWAY_INTERFACE`	versão do CGI (`"CGI/1.1"`)
	`HOME`	casa do utilizador do servidor
	`PATH_INFO`	caminho pedido (relativo)
	`PATH_TRANSLATED`	o caminho no sistema de ficheiros
	`QUERY_STRING`	o que vem depois do `?`
HTTP (1.1)	`HTTP_ACCEPT`	tipos aceites pelo cliente
	`HTTP_ACCEPT_LANGUAGE`	línguas preferidas
	`HTTP_HOST`	o cabeçalho `Host`
	`HTTP_USER_AGENT`	o navegador
	`HTTP_CONNECTION`	tipo de ligação (`keep-alive`)
Cliente (1.1)	`REMOTE_ADDR`, `REMOTE_PORT`	IP e porta do cliente
	`REQUEST_METHOD`	método do pedido
	`REQUEST_URI`	URI pedido: `PATH_INFO` + `QUERY_STRING`
	`CONTENT_TYPE`, `CONTENT_LENGTH`	tipo e tamanho do corpo
	`REMOTE_USER`, `AUTH_TYPE`	utilizador e tipo de autenticação
Servidor (1.1)	`SERVER_ADDR`, `SERVER_NAME`	endereço e nome
	`SERVER_PORT`	porta
	`SERVER_PROTOCOL`	versão do HTTP
	`SERVER_SOFTWARE`	o software do servidor
	`SERVER_ADMIN`	contacto do administrador

Entrada e saída

Entrada. O **corpo** do pedido chega pelo `stdin` (é preciso para os `POST` de formulários) e tem de ser descodificado conforme a codificação, normalmente `multipart/form-data` ou `application/x-www-form-urlencoded`. `REQUEST_METHOD`, `CONTENT_TYPE` e `CONTENT_LENGTH` (quantos bytes ler) dizem como ler. Há bibliotecas para isto, mas dependem da linguagem do CGI.

Definição

Saída. O CGI escreve a resposta no `stdout`:

1. **linha inicial** (opcional), começada por `Status:`, se o código de estado não for o óbvio;
2. **cabeçalhos**: só o `Content-type` é obrigatório;
3. **linha em branco** (obrigatória);
4. **corpo** (opcional).

Exemplo resolvido

```
#!/usr/bin/bash
echo "Content-type: text/html"     # cabeçalho obrigatório
echo ""                            # linha em branco obrigatória
echo "<h1>Olá mundo</h1>"
echo "<p>Método: $REQUEST_METHOD</p>"
echo "<p>Caminho pedido: $PATH_INFO</p>"
echo "<p>Query string: $QUERY_STRING</p>"
echo "<p>Cliente: $REMOTE_ADDR</p>"
```

Para `GET /cgi-bin/ola.cgi?nome=Ana`:

1. O servidor lança o *script* com `REQUEST_METHOD="GET"`, `QUERY_STRING="nome=Ana"`, `REMOTE_ADDR` = IP do cliente, etc., no ambiente. Um `GET` não tem corpo: nada a ler do `stdin`.
2. O *script* escreve no `stdout` o `Content-type`, a linha em branco e o HTML com os valores das variáveis.
3. O servidor envia esse `stdout` como resposta HTTP e o processo acaba.

Num **`POST`**: lê-se `CONTENT_LENGTH` (ex.: `27`), lêem-se esses bytes do `stdin` (`nome=Ana&idade=25`), descodifica-se o *urlencoded* para `{nome: "Ana", idade: "25"}`, e escreve-se a resposta da mesma forma.